

EUGENIA BAHIT

PYTHON APLICADO

4ª Edición



PYTHON APLICADO

4ª Edición

EUGENIA BAHIT

Informática Teórica[†]
Elida Bahit Research Centre

EBRC Publisher

Londres • Buenos Aires
2022

[†] Directora del Elida Bahit Research Centre de Reino Unido. Contacto: www.elidabahit.co.uk

PYTHON APLICADO, 4ª Edición.
© 2022 EUGENIA BAHIT

Excepto donde se indique lo contrario, el contenido de este libro está distribuido bajo una licencia Creative Commons Atribución No Comercial Sin Derivadas 4.0 Internacional (CC BY-NC-ND 4.0). La copia y distribución sin fines de lucro se encuentran permitidas. Texto completo de la licencia: <https://creativecommons.org/licenses/by-nc-nd/4.0/deed.es>

ISBN: 978-1-8381901-8-7
e-ISBN: 978-1-8381901-9-4

Cuarta edición
Publicado en Reino Unido por EBRC Publisher
71-75 Shelton Street, Covent Garden, London, WC2H 9JQ
Editora: Eugenia Bahit
www.eugeniabahit.co.uk

Diseño original: Amanda Kraft
Adaptación de diseño de la 4ta. edición: EBRC Publisher



ÍNDICE DE CAPÍTULOS

1. Diferencia entre programar y codificar.....	15
2. Primer acercamiento al <i>Scripting</i>	19
3. Acerca de Python.....	21
4. Elementos del Lenguaje.....	24
5. Módulos e importación.....	29
6. Colecciones.....	32
7. Estructuras de Control de Flujo.....	36
8. Colecciones por comprensión.....	43
9. Funciones.....	48
10. Docstrings (cadenas de documentación).....	61
11. Manipulación de cadenas de texto.....	65
12. Manipulación de listas y tuplas.....	77
13. Manipulación de diccionarios.....	82
14. Manejo y manipulación de archivos.....	87
15. Manejo de archivos CSV.....	91
16. Expresiones regulares.....	97
17. Creando menús de opciones.....	100
18. Captura y manejo de excepciones.....	107
19. Generación de registros de sistema.....	118
20. Módulos del sistema (os, sys y subprocess).....	123
21. Conexiones remotas (HTTP, FTP y SSH).....	133
22. Bibliotecas para manejo avanzado de archivos en GNU/Linux.....	140
23. Paralelismo y concurrencia.....	145
24. Probabilidad y Estadística con Python.....	164
25. Estadística descriptiva con Python.....	182
26. Matemáticas nativas.....	189
27. Python como CGI para aplicaciones Web.....	207
28. Consumo y producción de APIs con arquitecturas REST/JSON.....	233
29. Conexiones a bases de datos con MySQL® y MariaDB.....	245

30. Generación y lectura de cookies HTTP.....	257
31. Sistemas de control de acceso en entornos HTTPS.....	261
32. Programación orientada a objetos con Python.....	267
33. Pruebas unitarias.....	278

TABLA DE CONTENIDOS

1. Diferencia entre programar y codificar.....	15
2. Primer acercamiento al <i>Scripting</i>	19
2.1 Convertir un script en comando del sistema.....	20
3. Acerca de Python.....	21
3.1 Glosario.....	21
3.2 La shell interactiva de Python.....	23
4. Elementos del Lenguaje.....	24
4.1 Variables.....	24
4.2 Entrada y salida.....	25
4.3 Tipos de datos.....	26
4.4 Codificación de caracteres (encoding).....	27
4.5 Operadores Aritméticos.....	27
4.6 Comentarios.....	28
5. Módulos e importación.....	29
6. Colecciones.....	32
6.1 Tuplas.....	32
6.2 Listas.....	33
6.3 Diccionarios.....	33
6.4 Series (arrays).....	34
7. Estructuras de Control de Flujo.....	36
7.1 Sangrado.....	37
7.2 Estructuras de control de flujo condicionales.....	37
7.2.1 Operadores lógicos.....	38
7.3 Estructuras de control iterativas.....	40
7.3.1 Bucle while.....	40

7.3.2 Bucle for.....	42
8. Colecciones por comprensión.....	43
8.1 Listas y conjuntos por comprensión.....	43
8.2 Diccionarios por comprensión.....	47
8.3 Sintaxis para colecciones por comprensión.....	48
9. Funciones.....	48
9.1 Funciones definidas por el usuario.....	49
9.1.1 Sobre los parámetros.....	50
9.1.2 Parámetros por omisión.....	51
9.1.3 Claves como argumentos.....	52
9.1.4 Parámetros arbitrarios.....	52
9.1.5 Desempaquetado de parámetros.....	53
9.1.6 Llamadas recursivas y de retorno.....	54
9.2 Sobre la finalidad de las funciones.....	54
9.3 Conceptos avanzados sobre funciones.....	55
9.3.1 Lambdas.....	55
9.3.2 Clausuras (closures).....	56
9.3.3 Envolturas (wrappers) y decoradores.....	58
9.3.3.1 Funcionamiento de las envolturas y decoradores.....	59
10. Docstrings (cadenas de documentación).....	61
11. Manipulación de cadenas de texto.....	65
11.1 Inyección de variables.....	66
11.2 Métodos de formato.....	67
11.2.1 Convertir a mayúscula la primera letra.....	67
11.2.2 Convertir una cadena a minúsculas.....	67
11.2.3 Convertir una cadena a mayúsculas.....	67
11.2.4 Convertir mayúsculas a minúsculas y viceversa.....	68
11.2.5 Convertir una cadena en Formato Título.....	68
11.2.6 Centrar un texto.....	68
11.2.7 Alinear texto a la izquierda.....	68
11.2.8 Alinear texto a la derecha.....	69
11.2.9 Rellenar un texto anteponiendo ceros.....	69
11.3 Métodos de Búsqueda.....	69

11.3.1 Contar cantidad de apariciones de un fragmento de texto.....	69
11.3.2 Buscar un fragmento de texto dentro de una cadena.....	70
11.4 Métodos de Validación.....	70
11.4.1 Saber si una cadena comienza por un texto determinado.....	70
11.4.2 Saber si una cadena finaliza con un texto determinado.....	70
11.4.3 Saber si una cadena es alfanumérica.....	71
11.4.4 Saber si una cadena es alfabética.....	71
11.4.5 Saber si una cadena es numérica.....	71
11.4.6 Saber si una cadena contiene solo minúsculas.....	72
11.4.7 Saber si una cadena contiene solo mayúsculas.....	72
11.4.8 Saber si una cadena contiene solo espacios en blanco.....	73
11.4.9 Saber si una cadena tiene formato tipo título.....	73
11.5 Métodos de Sustitución.....	73
11.5.1 Dar formato a una cadena, sustituyendo texto dinámicamente..	73
11.5.2 Reemplazar texto en una cadena.....	74
11.5.3 Eliminar caracteres a la izquierda y derecha de una cadena.....	74
11.5.4 Eliminar caracteres a la izquierda de una cadena.....	75
11.5.5 Eliminar caracteres a la derecha de una cadena.....	75
11.6 Métodos de unión y división.....	75
11.6.1 Unir una cadena de forma iterativa.....	75
11.6.2 Partir una cadena en tres partes, utilizando un separador.....	76
11.6.3 Partir una cadena en varias partes, utilizando un separador.....	76
11.6.4 Partir una cadena en en líneas.....	76
12. Manipulación de listas y tuplas.....	77
12.1 Métodos de agregado.....	77
12.1.1 Agregar un elemento al final de la lista.....	77
12.1.2 Agregar varios elementos al final de la lista.....	77
12.1.3 Agregar un elemento en una posición determinada.....	78
12.2 Métodos de eliminación.....	78
12.2.1 Eliminar el último elemento de la lista.....	78
12.2.2 Eliminar un elemento por su índice.....	78
12.2.3 Eliminar un elemento por su valor.....	78
12.3 Métodos de orden.....	79
12.3.1 Ordenar una lista en reversa (invertir orden).....	79

12.3.2 Ordenar una lista en forma ascendente.....	79
12.3.3 Ordenar una lista en forma descendente.....	79
12.4 Métodos de búsqueda.....	79
12.4.1 Contar cantidad de apariciones elementos.....	79
12.4.2 Obtener número de índice.....	80
12.5 Anexo sobre listas y tuplas.....	80
12.5.1 Conversión de tipos.....	80
12.5.2 Concatenación de colecciones.....	81
12.5.3 Valor máximo y mínimo.....	81
12.5.4 Contar elementos.....	81
13. Manipulación de diccionarios.....	82
13.1 Métodos de eliminación.....	82
13.1.1 Vaciar un diccionario.....	82
13.2 Métodos de agregado y creación.....	82
13.2.1 Copiar un diccionario.....	82
13.2.2 Crear un nuevo diccionario desde las claves de una secuencia. .	83
13.2.3 Concatenar diccionarios.....	83
13.2.4 Establecer una clave y valor por defecto.....	84
13.3 Métodos de retorno.....	84
13.3.1 Obtener el valor de una clave.....	84
13.3.2 Saber si una clave existe en el diccionario.....	85
13.3.3 Obtener las claves y valores de un diccionario.....	85
13.3.4 Obtener las claves de un diccionario.....	86
13.3.5 Obtener los valores de un diccionario.....	87
13.3.6 Obtener la cantidad de elementos de un diccionario.....	87
14. Manejo y manipulación de archivos.....	87
14.1 Modos de Apertura de un archivo.....	88
14.2 Algunos métodos del Objeto File.....	90
14.3 Acceso a archivos mediante la estructura with.....	90
15. Manejo de archivos CSV.....	91
15.1 Algunos ejemplos de archivos CSV.....	91
15.2 Trabajar con archivos CSV desde Python.....	93
15.2.1 Lectura de archivos CSV.....	93

15.2.2 Escritura de archivos CSV.....	95
16. Expresiones regulares.....	97
16.1 Búsqueda de patrones.....	98
17. Creando menús de opciones.....	100
17.1 Creación de un menú de opciones básico.....	100
17.2 Creación de un menú de opciones con argparse.....	102
17.2.1 Paso 1: Importación del módulo.....	102
17.2.2 Paso 2: Construcción de un objeto ArgumentParser.....	102
17.2.3 Paso 3: Agregado de argumentos y configuración.....	103
17.2.4 Paso 4: Generación del análisis (parsing) de argumentos.....	106
18. Captura y manejo de excepciones.....	107
18.1 Uso básico de la estructura try/except.....	108
18.2 try/except/finally y try/except/else.....	112
18.3 Lanzamiento de excepciones.....	113
18.4 Encadenamiento de excepciones.....	116
18.5 Tipos de excepciones más comunes.....	117
19. Generación de registros de sistema.....	118
19.1 Principales elementos del módulo logging.....	119
20. Módulos del sistema (os, sys y subprocess).....	123
20.1 El módulo OS.....	123
20.1.1 Variables de entorno: os.environ.....	125
20.2 Ejecución simplificada de comandos del sistema.....	126
20.3 Ejecución de comandos del sistema mediante Popen y shlex.split.....	127
20.3.1 Capturar la salida estándar y los errores.....	128
20.3.2 Emplear la salida de un comando como entrada de otro.....	129
20.4 Variables y funciones del módulo sys.....	131
21. Conexiones remotas (HTTP, FTP y SSH).....	133
21.1 Conexiones remotas vía HTTP y HTTPS.....	133
21.2 Conexiones remotas vía FTP.....	135
21.2.1 Solicitando la contraseña con getpass.....	137
21.3 Conexiones SSH con Paramiko.....	137
21.3.1 Requisitos previos.....	137
21.3.2 Uso de Paramiko.....	138

22. Bibliotecas para manejo avanzado de archivos en GNU/Linux.....	140
22.1 Compresión y descompresión de archivos con Itarfile y zipfile.....	140
22.1.1 La biblioteca tarfile.....	140
22.1.2 La biblioteca zipfile.....	141
22.2 Manejo de archivos temporales con la biblioteca tempfile.....	142
22.2.1 Lectoescritura de archivos temporales.....	142
22.3 Búsqueda de archivos con las bibliotecas glob y fnmatch.....	144
23. Paralelismo y concurrencia.....	145
23.1 Entender el paralelismo.....	145
23.2 Problemas con el paralelismo.....	147
23.3 Programación paralela en Python.....	147
23.3.1 Paralelismo basado en hilos.....	148
23.3.2 Paralelismo basado en procesos.....	153
23.3.2.1 Grupos de procesos mediante la clase Pool.....	156
24. Probabilidad y Estadística con Python.....	164
24.1 Funciones estadísticas básicas (len, sum, max, min).....	164
24.2 Probabilidad de sucesos simples y compuestos mutuamente excluyentes.....	164
24.2.1 Espacio muestral.....	164
24.2.2 Sucesos simples y compuestos.....	165
24.2.3 Asignación de probabilidades.....	165
24.2.4 Sucesos simples mutuamente excluyentes.....	166
24.2.5 Sucesos compuestos por sucesos simples mutuamente excluyentes.....	166
24.2.6 Funciones.....	168
24.3 Probabilidad condicional en Python.....	168
24.3.1 Funciones.....	169
24.3.2 Sucesos dependientes.....	170
24.3.2.1 Teoría de conjuntos en Python.....	172
24.3.3 Sucesos independientes.....	172
24.4 Teorema de Bayes en Python.....	173
24.4.1 Teorema de Bayes y probabilidad de causas.....	173
24.4.1.1 Datos: caso práctico.....	174

24.4.1.2	Análisis.....	174
24.4.1.3	Procedimiento.....	176
24.4.2	Funciones.....	181
24.4.3	Bibliografía complementaria.....	181
25.	Estadística descriptiva con Python.....	182
25.1	Estadística poblacional y muestral.....	182
25.1.1	Medidas descriptivas de tendencia central.....	182
25.1.2	Medidas descriptivas de dispersión.....	183
25.1.2.1	Cálculos de dispersión.....	183
25.2	Frecuencia estadística.....	185
25.2.1	Frecuencia absoluta.....	185
25.2.2	Frecuencia relativa.....	187
25.2.3	Frecuencias acumuladas.....	188
26.	Matemáticas nativas.....	189
26.1	El módulo math.....	190
26.1.1	Combinaciones, factoriales y permutaciones.....	191
26.1.2	Números racionales y divisor común.....	193
26.1.3	Logaritmos.....	194
26.1.4	Producto, potencia y raíz cuadrada.....	194
26.1.5	Razones trigonométricas.....	195
26.2	El problema de los números de coma flotante y el módulo decimal..	197
26.3	Manejo de racionales con fractions.....	200
26.4	Números aleatorios criptográficamente inseguros.....	203
26.5	El módulo statistic.....	206
27.	Python como CGI para aplicaciones Web.....	207
27.1	Entender la interfaz CGI.....	207
27.2	Entender el servidor HTTP de Apache.....	209
27.3	Montar un Virtual Host localmente.....	211
27.3.1	Instalación y configuración de Apache.....	212
27.3.2	Habilitación del módulo <i>cgi</i>	213
27.3.3	Definición de un nombre de <i>host</i> nuevo.....	214
27.3.4	Creación de la estructura de directorios.....	215
27.3.5	Creación del Virtual Host.....	215

27.3.6	Habilitación del nuevo Virtual Host.....	216
27.3.7	Reiniciar Apache.....	216
27.3.8	Probando la nueva Web.....	217
27.4	Separar el HTML del código Python.....	217
27.5	Envío de correo electrónico.....	219
27.6	Métodos GET y POST de HTTP.....	222
27.6.1	Recepción y análisis de solicitudes GET.....	223
27.6.2	Recepción y análisis de solicitudes POST.....	225
27.6.3	Carga de archivos con Python.....	225
27.6.3.1	Consideraciones sobre la seguridad.....	226
27.6.3.2	Servir archivos estáticos con Python.....	227
27.6.3.3	Obtener el tipo MIME de un archivo.....	231
27.6.3.4	Codificar un archivo en Base 64.....	232
28.	Consumo y producción de APIs con arquitecturas REST/JSON.....	233
28.1	El formato JSON.....	234
28.2	El protocolo HTTP.....	236
28.3	Consumo y producción de APIs.....	241
29.	Conexiones a bases de datos con MySQL® y MariaDB.....	245
29.1	Configuración de MariaDB.....	246
29.2	Trabajando con MariaDB y MySQL® desde Python.....	248
29.3	Seguridad: prevención de inyecciones SQL.....	254
29.4	Función para automatizar consultas SQL.....	256
30.	Generación y lectura de cookies HTTP.....	257
30.1	Creación de cookies HTTP con Python.....	258
30.1.1	Atributos adicionales de una cookie HTTP.....	258
30.2	Lectura de cookies HTTP desde Python.....	259
31.	Sistemas de control de acceso en entornos HTTPS.....	261
31.1	Trabajar con archivos de sesión.....	264
31.2	Consideraciones de seguridad.....	265
32.	Programación orientada a objetos con Python.....	267
32.1	Breve introducción a la programación orientada a objetos.....	267
32.2	Elementos y características de la programación orientada a objetos.....	268
32.2.1	Clases.....	268

32.2.2 Métodos y propiedades.....	269
32.2.3 Objetos.....	269
32.2.4 Polimorfismo.....	270
32.2.5 Encapsulado.....	270
32.2.6 Herencia.....	271
32.2.7 Composición.....	271
32.2.8 Visibilidad y Ocultación.....	273
32.3 Sobre el uso de <i>self</i> en Python.....	275
32.4 Artilugios de la programación orientada a objetos.....	276
32.4.1 El método constructor.....	276
32.4.2 Recorrido de propiedades.....	276
32.4.3 Patrón de diseño compuesto y agregación.....	277
33. Pruebas unitarias.....	278
33.1 Doctest.....	278
33.2 Unittest.....	280
33.2.1 Métodos Assert.....	280
33.2.2 Descubriendo pruebas.....	283

1. DIFERENCIA ENTRE PROGRAMAR Y CODIFICAR

En el ámbito de la ingeniería de software, un **problema** es un requerimiento concreto que necesita ser resuelto mediante un programa informático, mientras que su **análisis** consiste en entender qué se necesita hacer para determinar cómo hacerlo, mediante qué operaciones, con cuáles valores de entrada y para arrojar cuáles valores de salida. Los **algoritmos** constituyen el paso a paso de la solución del problema.

Como características principales de los algoritmos, puede decirse que:

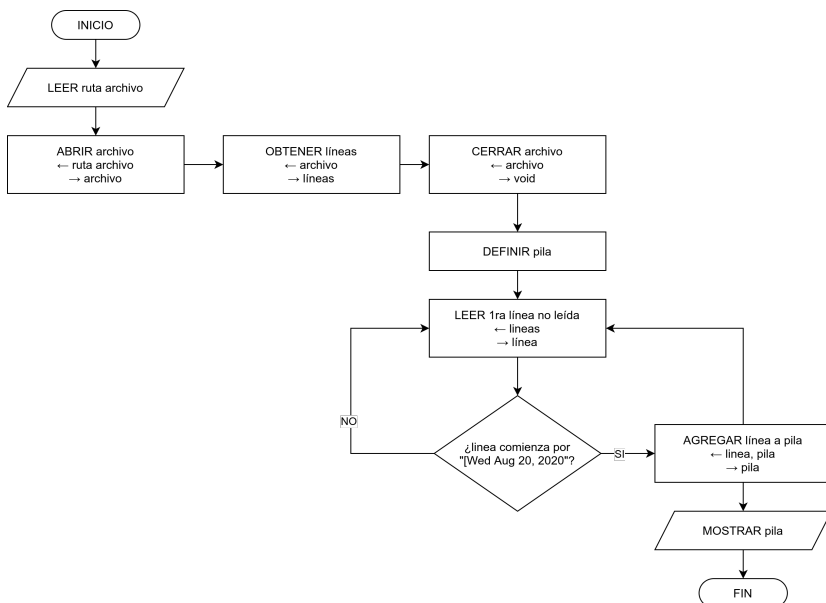
1. comienzan aceptando los datos de entrada que serán procesados;
2. las instrucciones definidas deben ser completas, precisas y concretas;
3. el tiempo total empleado por un algoritmo para llevar a cabo sus operaciones debe ser finito, así como la cantidad de instrucciones y el número de repeticiones cuando las hubiera;
4. finalmente, debe producir al menos un resultado (valor de salida).

Un factor clave de la programación es diseñar los algoritmos con una completa abstracción del lenguaje. Por ejemplo, si tras el análisis de un problema se determina que es necesario visualizar una pila con las entradas del registro de errores del archivo *errors.log* del servidor HTTP de Apache del día jueves 20 de agosto de 2020, considerando que cada entrada del registro necesaria es una línea que comienza por [Thu Aug

20, 2020, se podría resolver, independientemente del lenguaje a utilizar, con un algoritmo como el siguiente:

1. LEER ruta del archivo
2. ABRIR archivo
3. OBTENER líneas del archivo leído
4. CERRAR archivo
5. DEFINIR pila
6. SI la primera línea no leída comienza por "[Wed Aug 20, 2020"
AGREGAR línea a pila
7. Repetir el paso 6 por cada línea no leída hasta que no queden líneas sin leer
8. MOSTRAR la pila en pantalla

Para facilitar la comprensión de un algoritmo, puede representarse en un *diagrama de flujo* como el siguiente:



Estos diagramas emplean diversas figuras tales como:

- *paralelogramos*, para los valores de entrada y salida;
- *rectángulos*, para las operaciones;
- *rombos*, para los condicionales;
- *y rectángulos con bordes redondeados*, para indicar inicio y fin del algoritmo.

Posteriormente, los algoritmos son traducidos a código fuente, y aquellas operaciones no disponibles de forma nativa en el lenguaje, requerirán el diseño de un nuevo algoritmo.

El siguiente código es la traducción correspondiente al diagrama de flujo anterior:

```
def obtener_registros(ruta_archivo):  
    archivo = open(ruta_archivo, "r")  
    lineas = archivo.readlines()  
    archivo.close()  
    pila = []  
    for linea in lineas:  
        if linea.startswith("[Wed Aug 20, 2020"):  
            pila.append(linea)  
  
    print(pila)
```

El *código fuente* es el conjunto de órdenes escritas en un determinado lenguaje de programación, que componen un programa. Los *lenguajes de programación* son lenguajes informáticos diseñados para que un ordenador pueda llevar a cabo operaciones concretas. Los *lenguajes informáticos* son lenguajes artificiales creados exclusivamente para ser entendidos por los ordenadores. Los de programación son los que permiten la ejecución de órdenes pero los hay de otros tipos. Se manejan en dos niveles básicos: *bajo nivel*, que se refiere al código

máquina (código binario) y los de *alto nivel* que poseen una estructura humanamente legible e independiente del hardware.

Los hay *interpretados* (sus instrucciones son ejecutadas directamente sobre el sistema operativo por un intérprete) y *compilados* (aquellos que requieren ser traducidos a un lenguaje intermedio y empaquetados para su ejecución). Algunos funcionan en diversas plataformas y otros son exclusivos de sistemas concretos. Algunos aceptan solo un paradigma (técnica) de programación, mientras que otros, pueden aceptar más de uno.

Si bien al momento de traducir un algoritmo a código fuente la sintaxis empleada dependerá exclusivamente del lenguaje de programación elegido, todo lenguaje posee una serie de elementos comunes —a todos los lenguajes— que se encuentran disponibles. Solo la sintaxis del lenguaje es lo que cambia. Por ejemplo, mientras que una variable *foo* de tipo entero con valor 15, en Python se define mediante **`foo = 15`**, en PHP se define anteponiendo un signo dólar al nombre de la variable y un punto y coma tras el valor, tal que **`$foo = 15;`** y en C, requiere la declaración de tipo, tal que **`int foo = 15;`**.

El proceso de traducción de un algoritmo a código fuente en un lenguaje de programación determinado es lo que técnicamente se conoce como *codificación*. El proceso que abarca el análisis de un problema, el diseño de su solución (algoritmo) más su traducción a código fuente (codificación), es lo que se denomina *programación*. Así, el estudio de esta abarca tres grandes temas: lógica, análisis y diseño de algoritmos; y lenguajes de programación. «*Python Aplicado*» pertenece a esta tercera categoría.

2. PRIMER ACERCAMIENTO AL *SCRIPTING*

En programación, un *script* es un archivo de código fuente con instrucciones sencillas, que puede ser ejecutado a través de la línea de comandos. Se conoce como *scripting* a la **técnica de programación** empleada para crear este tipo de archivos.

Para que un archivo de código fuente sea considerado un script, debe cumplir las siguientes características:

1. Ser ejecutable.
2. Estar escrito en un lenguaje que pueda ser interpretado por el ordenador.
3. No depender de otros archivos.

1. Archivo ejecutable

Para que un archivo pueda ser ejecutado, hay que otorgarle permisos de ejecución. Desde la línea de comandos, se logra con el comando *chmod*:

```
chmod +x nombre-del-archivo
```

2. Lenguaje interpretado

Un *script* puede ser escrito en cualquier lenguaje interpretado, soportado por el Sistema Operativo, como Python, Perl, GNU Bash, PHP, Ruby, etc.

Para que el sistema operativo sepa qué intérprete utilizar para entender y ejecutar el *script*, se debe colocar la ruta de dicho intérprete en la primera línea del archivo, antecedida por los símbolos **#!**

Lo anterior se conoce como *hashbang* o *shebang*.

Dado que la ruta de un intérprete puede ser distinta en los diversos Sistemas Operativos o distribuciones, realizar la llamada al intérprete a través de **env**, es una práctica habitual:

```
#!/usr/bin/env python
```

Por este motivo, un *script* no requiere de una extensión para poder ser interpretado.

Algunos ejemplos:

<code>#!/usr/bin/env python</code>	Intérprete Python (versión por defecto)
<code>#!/usr/bin/env python2</code>	Fuerza a utilizar el intérprete de Python 2.x
<code>#!/usr/bin/env python3</code>	Fuerza a utilizar el intérprete de Python 3.x
<code>#!/usr/bin/env bash</code>	Intérprete de GNU Bash

3. Script vs Programa

Si un archivo de código fuente que es ejecutable y contiene la ruta del intérprete, depende de otros archivos, entonces se lo considera parte de un programa. Al no tener independencia, no cuenta como *script*.

2.1 CONVERTIR UN SCRIPT EN COMANDO DEL SISTEMA

Archivo: *hola-mundo.py*

```
#!/usr/bin/env python2  
print("Hola Mundo!")
```

Crear un archivo llamado «*hola-mundo.py*» (con el contenido anterior), asignarle permisos de ejecución y probar el *script* en la línea de comandos ejecutando la siguiente instrucción:

```
./hola-mundo.py
```

Ahora, como *root*, copiar el archivo en la carpeta */usr/bin* con el nombre «*hola-mundo*»:

```
cp hola-mundo.py /usr/bin/hola-mundo
```

El *script* está ahora disponible para ser ejecutado como comando.

```
usuario@equipo:~$ hola-mundo  
Hola Mundo!  
usuario@equipo:~$
```

3. ACERCA DE PYTHON

Dentro de los **lenguajes informáticos**, Python, pertenece al grupo de los **lenguajes de programación** y puede ser clasificado como un **lenguaje interpretado**, de alto nivel, **multiplataforma**, de **tipado dinámico** y **multiparadigma**.

Para la escritura de código, y a fin de alcanzar un mecanismo estándar en la forma de programar, **provee unas reglas de estilo** definidas a través de la *Python Enhancement Proposal N° 8 (PEP 8)*, cuyas especificaciones serán expuestas a lo largo de los primeros capítulos.

3.1 GLOSARIO

Lenguaje informático: es un idioma artificial utilizado por ordenadores, cuyo fin es transmitir información de algo a alguien. Los lenguajes informáticos, pueden clasificarse en: a) lenguajes de programación (Python, PHP, Pearl, C, etc.); b) lenguajes de especificación (UML); c) lenguajes de consulta (SQL); d) lenguajes de marcas (HTML, XML); e)

lenguajes de transformación (XSLT); f) protocolos de comunicaciones (HTTP, FTP); entre otros.

Lenguaje de programación: es un lenguaje informático, diseñado para expresar órdenes e instrucciones precisas, que deben ser llevadas a cabo por un ordenador. El mismo puede utilizarse para crear programas que controlen el comportamiento físico o lógico del ordenador. Está compuesto por una serie de símbolos, reglas sintácticas y semánticas que definen la estructura del lenguaje.

Lenguajes de alto nivel: son aquellos cuya característica principal, consiste en una estructura sintáctica y semántica legible, acorde a las capacidades cognitivas humanas. A diferencia de los lenguajes de bajo nivel (código máquina, lenguaje ensamblador), son independientes de la arquitectura del hardware, motivo por el cual, asumen mayor portabilidad.

Lenguajes interpretados: a diferencia de los compilados, no requieren de compiladores para ser ejecutados, sino de intérpretes. Un intérprete, actúa de manera similar a un compilador, con la salvedad de que ejecuta el programa directamente, sin necesidad de generar previamente un archivo ejecutable. Ejemplo de lenguajes de programación interpretados son Python, PHP, Ruby, Common Lisp, entre otros.

Tipado dinámico: un lenguaje de tipado dinámico es aquel cuyas variables, no requieren ser definidas asignando su tipo de datos, sino que este, se auto-asigna en tiempo de ejecución, según el valor declarado.

Multiplataforma: significa que puede ser interpretado en diversos Sistemas Operativos como GNU/Linux, OpenBSD, sistemas privativos, entre otros.

Multiparadigma: acepta diferentes paradigmas (técnicas) de programación, tales como la orientación a objetos, la programación imperativa y funcional.

Código fuente: es un conjunto de instrucciones y órdenes lógicas, compuestos de algoritmos que se encuentran escritos en un determinado lenguaje de programación, las cuales deben ser interpretadas o compiladas, para permitir la ejecución de un programa informático.

3.2 LA SHELL INTERACTIVA DE PYTHON

Toda vez que se desee probar la respuesta de Python frente a una instrucción particular, se puede recurrir a la shell interactiva del lenguaje, que en sistemas operativos basados en Linux o Unix, puede accederse mediante el comando **python**:

```
eugenia@bella:~$ python
Python 2.7.16 (default, Oct 10 2019, 22:02:15)
[GCC 8.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

En aquellos sistemas donde la versión por defecto sea anterior a la rama 3, se podrá acceder a la shell de Python 3, mediante el comando **python3** siempre que esta rama esté instalada:

```
eugenia@bella:~$ python3
Python 3.7.3 (default, Jan 22 2021, 20:04:44)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

El código que se ejecuta en la shell interactiva de Python es el mismo que se puede escribir en un archivo, solo que en la shell no persistirá.

No obstante, las funciones de búsqueda (**Ctrl+r**) y recorrido del historial con las teclas de dirección (flecha arriba, flecha abajo), se encuentran disponibles.

4. ELEMENTOS DEL LENGUAJE

Python, como todo lenguaje de programación, se compone de una serie de elementos característicos y estructuras. A continuación, se abarcarán los principales elementos.

4.1 VARIABLES

Una variable es un espacio para almacenar datos modificables de forma temporal en la memoria del ordenador.

En Python, una variable se define con la sintaxis:

```
nombre_de_la_variable = valor_de_la_variable
```

Cada variable, tiene un nombre y un valor. Este último, define el tipo de datos de la variable.

Existe un tipo de espacio de almacenamiento denominado **constante**. Una constante, se utiliza para definir valores fijos, que no requieran ser modificados. En Python, el concepto de constante es simbólico ya que todo espacio es variable.

PEP 8: variables

Utilizar nombres descriptivos y en minúsculas. Para nombres compuestos, separar las palabras por guiones bajos. Antes y después del signo =, debe haber uno (y solo un) espacio en blanco.

Nombre correcto:	mi_variable
Nombres incorrectos:	MiVariable, mivariable, VARIABLE
Asignación correcta:	variable = 12
Asignaciones incorrectas:	variable=12 variable = 12 variable = 12

PEP 8: constantes

Utilizar nombres descriptivos y en mayúsculas separando palabras por guiones bajos.

Ejemplo: **MI_CONSTANTE = 12**

4.2 ENTRADA Y SALIDA

Para **imprimir un valor en pantalla**, hasta la versión 2.7 de Python, se utiliza la palabra clave **print**:

```
mi_variable = 15
print mi_variable
```

Pero a partir de la versión 3.0 de Python, se utiliza la función **print()**:

```
mi_variable = 15
print(mi_variable)
```

Es importante destacar que la función **print()** también estaba disponible en Python 2.

Lo anterior, en ambos casos, imprime el valor de la variable *mi_variable* en pantalla.

Para **obtener datos solicitados al usuario**, hasta la versión 2.7 de Python, se utiliza la función `raw_input()`:

```
mi_variable = raw_input("Ingresar un valor: ")
```

A partir de la versión 3.0 de Python, se utiliza la función `input()`, también disponible en versiones anteriores, pero con un funcionamiento diferente:

```
mi_variable = input("Ingresar un valor: ")
```

4.3 TIPOS DE DATOS

Una variable puede contener valores de diversos tipos. Estos tipos se listan a continuación.

Cadena de texto (string):

```
mi_cadena = "Hola Mundo!"  
otra_cadena = 'Hola Mundo!'
```

```
mi_cadena_multilinea = """  
Esta es una cadena  
de varias líneas  
"""
```

Número entero:

```
edad = 35
```

Número real:

```
precio = 35.05
```

Booleano (verdadero / Falso):

```
verdadero = True  
falso = False
```

Existen además, otros tipos de datos más complejos, que se abarcarán más adelante.

4.4 CODIFICACIÓN DE CARACTERES (ENCODING)

En versiones anteriores a Python 3.0, la especificación de la codificación de caracteres a emplearse (cuando ésta no es ASCII), resulta obligatoria.

A partir de la versión 3.0, esta declaración es opcional.

```
# -*- coding: utf-8 -*-
```

utf-8 podría ser cualquier otra codificación de caracteres. Por defecto, versiones anteriores a Python 3, utilizan la codificación ASCII como predeterminada. En caso de emplear caracteres no ASCII en versiones anteriores a la 3.0, Python generará un error:

```
variable = "En el Ñágara encontré un Ñandú"  
SyntaxError: Non-ASCII character[...]
```

Indicando el *encoding* adecuado, el archivo se ejecutará con éxito:

```
# -*- coding: utf-8 -*-  
variable = "En el Ñágara encontré un Ñandú"
```

4.5 OPERADORES ARITMÉTICOS

Un operador aritmético es aquel que permite realizar operaciones aritméticas sobre las variables.

Símbolo	Significado	Ejemplo	Resultado
+	Suma	a = 10 + 5	a es 15
-	Resta	a = 12 - 7	a es 5
-	Negación	a = -5	a es -5

*	Multiplicación	a = 7 * 5	a es 35
**	Exponente	a = 2 ** 3	a es 8
/	División	a = 12.5 / 2	a es 6.25
//	División entera	a = 12.5 // 2	a es 6.0
%	Módulo	a = 27 % 4	a es 3

PEP 8: operadores

Siempre colocar un espacio en blanco, antes y después de un operador

Un ejemplo sencillo con variables y operadores aritméticos:

```
bruto = 175
tasa_interes = 12.5
interes = bruto * tasa_interes / 100
tasa_bonificacion = 5.0
bonificacion = bruto * tasa_bonificacion / 100
neto = (bruto - bonificacion) + interes
```

4.6 COMENTARIOS

En un archivo de código fuente, un comentario es un texto explicativo que no es procesado por el intérprete y solo tiene una utilidad humana (no informática). Los comentarios pueden ser de una o varias líneas:

```
# Esto es un comentario de una sola línea
mi_variable = 15

"""Y este es un comentario
de varias líneas"""
mi_variable = 15
mi_variable = 15 # Este es un comentario en línea
```

En los comentarios, pueden incluirse palabras que representen acciones:

```
# TODO esto es algo por hacer
# FIXME esto es algo que debe corregirse (un fallo)
# XXX esto es algo que debe corregirse (error crítico)
```

PEP 8: comentarios

Comentarios en la misma línea del código deben separarse con dos espacios en blanco. Luego del símbolo `#` debe ir un solo espacio en blanco.

Correcto: `a = 15 # Edad de María`

Incorrecto: `a = 15 # Edad de María`

5. MÓDULOS E IMPORTACIÓN

Módulo. En Python se considera módulo a cualquier archivo `.py`.

Paquete. En Python se considera paquete es una carpeta que contiene módulos, y un archivo `__init__.py` que puede estar (o no) vacío.

```
├── paquete
│   ├── __init__.py
│   ├── modulo1.py
│   ├── modulo2.py
│   └── modulo3.py
```

Espacio de nombres: La importación de módulos y paquetes se realiza a través del espacio de nombres, el cual estará determinado por la ruta del archivo importar (se omiten las extensiones `.py` y las barras diagonales son sustituidas por un punto), tal que si la ruta de un archivo es `foo/bar/baz.py` su espacio de nombre será `foo.bar.baz`

Ruta de inclusión: Python, en primer lugar buscará los archivos en las rutas de importación propias, y en segundo lugar, en el directorio desde el cual se esté ejecutando el script.

Importación total y parcial: Importar un módulo significa incluir el contenido de un archivo dentro de otro.

Se pueden importar módulos completos o solo elementos parciales como variables, funciones, etc.

Para importar un módulo completo se utiliza la palabra clave *import* mientras que para importar elementos se utiliza la dupla *from / import*:

```
import modulo
import paquete.modulo
import paquete.subpaquete.modulo

from modulo import variable
from modulo import variable, funcion
from modulo import *
```

El **asterisco** equivale a importar todos los elementos contenidos en un módulo aunque no es igual que importar el módulo completo. Su uso está **desaconsejado** en las **PEP 8**.

```
import modulo # para llamar a 'A' dentro de modulo:
modulo.A
from modulo import * # para llamar a 'A' dentro de modulo:
A
```

Los módulos a importar pueden ser archivos propios (en ese caso, se convierte el *script* en un programa) o **módulos de la biblioteca estándar** de Python.

La importación de módulos debe realizarse al comienzo del documento, en orden alfabético de paquetes y módulos.

Primero deben importarse los módulos propios de Python. Luego, los módulos de terceros y finalmente, los módulos propios de la aplicación.

Entre cada bloque de imports, debe dejarse una línea en blanco.

Uso de elementos importados

Tanto si se ha importado un módulo completo como elementos independientes, se accede a los elementos de dicho módulo a través del espacio de nombre importado:

```
import modulosypaquetes.paquete
modulosypaquetes.paquete.modulo.funcion()

import modulosypaquetes.paquete.modulo
modulosypaquetes.paquete.modulo.funcion()

from modulosypaquetes.paquete import modulo
modulo.funcion()
from modulosypaquetes.paquete.modulo import funcion
funcion()
```

Alias

Es posible crear alias (en tiempo de importación) para acceder a los espacios de nombre de forma abreviada:

```
import modulosypaquetes.paquete.modulo as m
m.funcion()
```

6. COLECCIONES

Python, posee además de los tipos ya vistos, tres tipos más complejos, que admiten una colección de datos. Estos tipos son:

- Tuplas
- Listas
- Diccionarios

Estos tres tipos, pueden almacenar colecciones de datos de diversos tipos y se diferencian por su sintaxis y por la forma en la cual los datos pueden ser manipulados.

6.1 TUPLAS

Una tupla es una **variable que permite almacenar varios datos inmutables** (no pueden ser modificados una vez creados), y estos datos pueden ser de tipos diferentes:

```
mi_tupla = ('cadena de texto', 15, 2.8, 'otro dato', 25)
```

Se puede acceder a cada uno de los datos mediante su índice correspondiente. El índice se corresponde con la posición del elemento en la colección, siendo 0 (cero), el índice del primer elemento:

```
mi_tupla[1] # Salida: 15
```

También se puede acceder a una porción de la tupla, indicando (opcionalmente) desde el índice de inicio hasta el índice de fin:

```
mi_tupla[1:4] # Devuelve: (15, 2.8, 'otro dato')  
mi_tupla[3:] # Devuelve: ('otro dato', 25)  
mi_tupla[:2] # Devuelve: ('cadena de texto', 15)
```

Otra forma de acceder a la tupla de forma inversa (de atrás hacia adelante), es colocando un índice negativo:

```
mi_tupla[-1] # Salida: 25  
mi_tupla[-2] # Salida: otro dato
```

6.2 LISTAS

Una lista es similar a una tupla en todos los aspectos. La diferencia radica en que los elementos de una lista sí pueden ser modificados:

```
mi_lista = ['cadena de texto', 15, 2.8, 'otro dato', 25]
```

A las listas se accede igual que a las tuplas, por su número de índice:

```
mi_lista[1]      # Salida: 15  
mi_lista[1:4]    # Devuelve: [15, 2.8, 'otro dato']  
mi_lista[-2]     # Salida: otro dato
```

Las lista no son inmutables: permiten modificar los datos una vez creados:

```
mi_lista[2] = 3.8 # el tercer elemento ahora es 3.8
```

Al poder ser modificadas, las listas, a diferencia de las tuplas, permiten agregar nuevos valores:

```
mi_lista.append('Nuevo Dato')
```

6.3 DICCIONARIOS

Los diccionarios, al igual que las tuplas y las listas, son colecciones. La diferencia con estos, es que mientras que los elementos de las listas y las tuplas se asocian a un número de índice (o posición), los valores de un diccionario se asocian a un nombre de clave:

```
mi_diccionario = {  
    'clave_1': valor_1,  
    'clave_2': valor_2,  
    'clave_7': valor_7  
}  
mi_diccionario['clave_2'] # Salida: valor_2
```

Un diccionario permite eliminar cualquier entrada:

```
del(mi_diccionario['clave_2'])
```

Al igual que las listas, el diccionario permite modificar los valores:

```
mi_diccionario['clave_1'] = 'Nuevo Valor'
```

Y también es posible agregar nuevos elementos, asignando valores a nuevas claves:

```
mi_diccionario['nueva_clave'] = 'nuevo elemento'
```

6.4 SERIES (ARRAYS)

Python incorpora un tipo de dato denominado *array* similar al tipo homónimo en el lenguaje C. Se trata de una serie formada por datos numéricos de un tipo específico.

Para definir un elemento de tipo *array*, es necesario recurrir a una biblioteca del lenguaje, denominada *array*:

```
from array import array
```

A partir de ese momento, pueden crearse series de elementos que se encuentren restringidos a uno de trece tipos disponibles:

Tipo	Descripción	Tamaño
b	Caracter con signo	1 byte

Tipo	Descripción	Tamaño
		Mín. -128, Máx. 127
B	Caracter sin signo	1 byte Mín: 0, Máx. 255
u	Caracter unicode	2 bytes
h	Caracter corto con signo	2 bytes Mín. -32768, Máx. 32767
H	Caracter corto sin signo	2 bytes Mín. 0, Máx. 65535
i	Entero con signo	2 bytes Mín. -2147483648 Máx. 2147483647
I	Entero sin signo	2 bytes Máx. 4294967295
l	Entero largo con signo	4 bytes Mín. -9223372036854775808 Máx. 9223372036854775807
L	Entero largo sin signo	4 bytes Máx. 18446744073709551615
q	Entero doblemente largo con signo	8 bytes <i>no disponible en todas las plataformas</i>
Q	Entero doblemente largo sin signo	8 bytes <i>no disponible en todas las plataformas</i>
f	Número con coma flotante	4 bytes
d	Doble	8 bytes

Algunos ejemplos con sus respectivas salidas:

```
>>> array('f', [0, 1, 2, 3, 4, 5]) # serie de tipo float
array('f', [0.0, 1.0, 2.0, 3.0, 4.0, 5.0])

>>> array('b', [-128, -5, 55, 105, 127])
array('b', [-128, -5, 55, 105, 127])

>>> array('B', [0, 55, 105, 127, 255])
array('B', [0, 55, 105, 127, 255])

>>> array('u', 'ñandú')
```

```
array('u', 'ñandú')
```

Algunos de los métodos disponibles en el objeto array, son los siguientes:

append	al igual que en las listas, agrega un elemento al final del array.
count	retorna la cantidad de veces que un elemento se repite en el array.
extend	agrega varios elementos al final del array.
index	devuelve la posición de la primera aparición de un elemento.
insert	agrega un nuevo elemento en la posición indicada.
pop	elimina el último elemento del array o el elemento de la posición indicada.
remove	elimina la primera aparición de un elemento.
reverse	invierte el orden de los elementos del array.
tolist	convierte el array en una lista.

Para hacer uso de estos métodos se emplea la sintaxis:

```
objeto.metodo(argumento)
```

Teniendo en cuenta siempre, que objeto es la variable de tipo array. Por ejemplo:

```
>>> serie = array('i', [-340, 12, 75, 8000, 12])
>>> serie.count(12)
2
```

Lo anterior significa que el número **12** aparece **2** veces en el array.

7. ESTRUCTURAS DE CONTROL DE FLUJO

Una estructura de control es un bloque de código fuente que permite agrupar instrucciones de forma controlada. A continuación se describirán dos tipos de estructuras de control:

1. **Estructuras de control condicionales:** controlan el flujo de los datos a partir de condiciones.
2. **Estructuras de control iterativas:** controlan el flujo de los datos, ejecutando una misma acción de forma repetida.

7.1 SANGRADO

En Python, las estructuras de control se delimitan sobre la base de bloques de código con sangrado fijo y proporcional (mismo sangrado para cada estructura).

No todos los lenguajes de programación, necesitan de este sangrado, aunque sí se estila implementarlo, a fin de otorgar mayor legibilidad al código fuente. **En el caso de Python, el sangrado es obligatorio**, ya que de él dependerá su estructura y la forma de controlar la información.

PEP 8: sangrado de código

Un sangrado de **4 (cuatro) espacios en blanco**, indicará que las instrucciones sangradas forman parte de una misma estructura de control.

Una estructura de control, entonces, se define de la siguiente forma:

```
inicio de la estructura de control:  
    expresiones
```

7.2 ESTRUCTURAS DE CONTROL DE FLUJO CONDICIONALES

Las estructuras de control condicionales son aquellas que permiten evaluar si una o más condiciones se cumplen, para decir qué acción ejecutar.

Las condiciones se evalúan como verdaderas o falsas. O la condición se cumple (la condición es verdadera), o la condición no se cumple (la condición es falsa).

En la vida diaria, toda persona actúa de acuerdo a la evaluación de condiciones, solo que no siempre se hace de forma explícita o evidente.

Por ejemplo:

Si el semáforo está en verde, cruzar la calle. **Sino**, esperar a que el semáforo se ponga en verde.

En ocasiones, se evalúa más de una condición para llegar a ejecutar la misma acción:

Si el semáforo está en verde **o** no hay vehículos circulando, cruzar la calle. **Sino**, esperar a que el semáforo se ponga en verde.

7.2.1 OPERADORES LÓGICOS

Para describir la evaluación a realizar sobre una condición, se utilizan operadores lógicos de comparación:

Símbolo	Significado	Ejemplo	Resultado
==	Igual que	5 == 7	Falso
!=	Distinto que	rojo != verde	Verdadero
<	Menor que	8 < 12	Verdadero
>	Mayor que	7 > 12	Falso
<=	Menor o igual que	12 <= 12	Verdadero
>=	Mayor o igual que	4 >= 5	Falso

Para evaluar más de una condición simultáneamente, se utilizan operadores lógicos:

Operador	Ejemplo	Resultado
and (y)	5 == 7 and 7 < 12	0 y 1 Falso
	9 < 12 and 12 > 7	1 y 1 Verdadero
	9 < 12 and 12 > 15	1 y 0 Falso
or (o)	12 == 12 or 15 < 7	1 o 0 Verdadero

	$7 > 5$ or $9 < 12$	1 o 1	Verdadero
xor*	$(4 == 4) \wedge (9 > 3)$	1 o 1	Falso
(o excluyente)	$(4 == 4) \wedge (9 < 3)$	1 o 0	Verdadero

(*) *xor* como tal, no se encuentra disponible en Python por lo que se debe utilizar, en su reemplazo, el símbolo $\wedge$.

Las estructuras de control de flujo condicionales, se definen mediante el uso de tres palabras claves reservadas, del lenguaje: **if** (si), **elif** (sino, si) y **else** (sino).

Algunos ejemplos:

Si semáforo está en verde, cruzar. Sino, esperar.

```
if semaforo == verde:
    cruzar()
else:
    esperar()
```

En el ejemplo anterior, *cruzar* y *esperar* son dos acciones (verbos). Los verbos o acciones, en programación, se denominan funciones y sintácticamente, se las diferencia de las variables por estar acompañadas de un par de paréntesis. Las funciones serán abarcadas más adelante.

Si gasto hasta \$100, pago con dinero en efectivo. Sino, si gasto más de \$100 pero menos de \$300, pago con tarjeta de débito. Sino, pago con tarjeta de crédito.

```
if gasto <= 100:
    pagar_en_efectivo()
elif gasto > 100 and gasto < 300:
    pagar_con_debito()
else:
    pagar_con_credito()
```

Si la compra es mayor a \$100, obtengo un descuento del 10%

```
descuento = 0

if compra > 100:
    descuento = compra * 0.10
```

7.3 ESTRUCTURAS DE CONTROL ITERATIVAS

Las estructuras de control iterativas (también llamadas cíclicas o bucles), permiten ejecutar un mismo código, de forma repetida, n cantidad de veces mientras se cumpla una condición.

Python dispone dos estructuras de control iterativas:

- El bucle **while**
- El bucle **for**

7.3.1 BUCLE WHILE

Este bucle se encarga de ejecutar una misma acción mientras que una determinada condición se cumpla:

Mientras que año sea menor o igual a 2022, imprimir la frase "Informes de <año>".

```
#!/usr/bin/env python2
# -*- coding: utf-8 -*-

year = 2011

while year <= 2022:
    print("Informes de", year)
    year += 1
```

La iteración anterior, generará la siguiente salida:

```
Informes de 2011
Informes de 2012
Informes de 2013
Informes de 2014
Informes de 2015
```

Informes de 2016
Informes de 2017
Informes de 2018
Informes de 2019
Informes de 2020
Informes de 2021
Informes de 2022

Observar la última línea:

year += 1

En cada iteración se incrementa en 1 (uno) el valor de la variable **year** que condiciona el bucle . Si no se incrementase esta variable, su valor siempre sería igual a **2011** y por lo tanto, el bucle se ejecutaría de forma infinita, ya que la condición **year <= 2022** siempre se estaría cumpliendo.

Notación BNF (Bakus-Naur Form)

Es un formato utilizado en programación e informática en general, para documentar expresiones lógicas. El uso de:

<nombre>

indica que donde aparece **<nombre>** deberá reemplazarse dicho modificador, por el valor de un elemento que represente un nombre. Por ejemplo:

<Apellido>, <Nombre>

Podría reemplazarse por:

Bahit, Eugenia

Romper un bucle

Cuando el valor que condiciona la iteración no puede incrementarse, puede romperse el bucle mediante la instrucción **break** precedida de un condicional:

```
while True:
    nombre = raw_input("Indique su nombre: ")
    if nombre:
        break
```

El bucle anterior, incluye un condicional anidado que verifica si la variable nombre es verdadera (solo será verdadera si el usuario aporta un dato cuando le es solicitado en pantalla). Si es verdadera, el bucle se rompe (*break*). Sino, seguirá ejecutándose hasta que el usuario ingrese un dato cuando le es solicitado.

7.3.2 BUCLE FOR

El bucle **for** de Python, es aquél que facilita la iteración sobre los elementos de una tupla o lista. El bucle for siempre se utilizará sobre una lista o tupla, de forma tal que en cada iteración, se puedan ejecutar las misma acciones sobre cada uno de los elementos de la lista o tupla.

Por cada elemento en `mi_lista`, imprimir el elemento

```
#!/usr/bin/env python3

mi_lista = ['Juan', 'Antonio', 'Pedro', 'Ana']

for elemento in mi_lista:
    print(elemento)
```

Por cada color en `mi_tupla`, imprimir color

```
#!/usr/bin/env python3

mi_tupla = ('rosa', 'verde', 'celeste', 'amarillo')

for color in mi_tupla:
    print(color)
```

En los ejemplos anteriores, *elemento* y *color*, son dos **variables declaradas en tiempo de ejecución** (es decir, se declaran dinámicamente durante el bucle y se sobrescriben en cada iteración), asumiendo como valor, el de cada elemento de la lista (o tupla) en cada iteración.

Otra forma de iterar con el bucle **for**, puede emular a *while*:

Por cada año en el rango 2011 a 2023, imprimir la frase "Informes de <año>".

```
for year in range(2011, 2023):  
    print("Informes de", year)
```

8. COLECCIONES POR COMPRENSIÓN

El concepto de comprensión en este ámbito puede resultar confuso. Para evitar ambigüedades se optará por describir su significado dentro del contexto al cual se refiere.

En este sentido, la generación de colecciones por comprensión, se refiere a la facultad de crear colecciones, por medio de la descripción del procedimiento que genera cada uno de los elementos que la conforman.

8.1 LISTAS Y CONJUNTOS POR COMPRENSIÓN

La generación por comprensión se basa en el método de tabulación para la descripción de conjuntos.

Si la generación de colecciones por comprensión se pudiese hacer describiendo el procedimiento en lenguaje natural, una lista de números naturales menores que 6, se generaría diciendo que:

```
lista = [número natural, para todos los números naturales  
menores que seis]
```

Pero dado que la lista anterior podría generarse sin un procedimiento complejo, describirla por comprensión perdería sentido ya que puede crearse mediante:

```
lista = range(1, 6)
```

Sin embargo, la lista anterior podría necesitar generarse a partir del siguiente enunciado:

$$A = \{x : \forall x \in X, x \in \mathbb{N} \wedge x < 6\}$$

Lo anterior, define un conjunto llamado A , formado por una colección de elementos x , que se toman de un conjunto X , cada vez que se cumple que x pertenece al conjunto de los números naturales $\mathbb{N}$, y además es menor que 6.

Aquí, los elementos del conjunto (o lista) X , son desconocidos, y los elementos que sean números naturales menores que 6, podrían ser incluso menos que 5 números, pues los elementos del conjunto X no se conocen.

De modo tradicional, para crear una lista con esos elementos, habría que recorrer el conjunto X , y constatar si:

- el elemento es un número natural (es decir, si es un entero mayor que cero).
- el elemento es menor que 6.

O, lo que es igual, si el elemento es un entero, mayor que cero y menor que seis:

```
for x in X:
    if isinstance(x, int) and x > 0 and x < 6:
        A.append(x)
```

Viendo el ejemplo anterior más de cerca, se obtendría lo siguiente:

```
>>> X = ['a', 'b', 1, 'c', 3, 6, 'h', 4]
>>> A = []
>>> for x in X:
...     if isinstance(x, int) and x > 0 and x < 6:
...         A.append(x)
...
>>> A
[1, 3, 4]
```

Crear la lista anterior por comprensión, implicaría describir el enunciado previo en código fuente. Es decir, tomar el enunciado «*número natural, para todos los números naturales menores que seis*» o su formalización

$A = \{x : \forall x \in X, x \in \mathbb{N} \wedge x < 6\}$, y traducir cualquiera de los dos, a código fuente. Algo como:

```
A = [<aquí el código>]
```

Si se traduce el enunciado formal, podría resultar algo como esto:

```
A = [x for x in X if isinstance(x, int) and x > 0 and x < 6]
```

Recuperando el código del primer ejemplo, el resultado sería el siguiente:

```
>>> X = ['a', 'b', 1, 'c', 3, 6, 'h', 4]
>>> A = [x for x in X if isinstance(x, int) and x > 0 and x < 6]
>>> A
[1, 3, 4]
```

Las colecciones por comprensión se crean formalizando el conjunto que se desea crear y traduciendo a código fuente dicha formalización.

De hecho, el enunciado formal de un conjunto, se puede traducir literalmente en un conjunto, reemplazando los corchetes por llaves:

```
>>> X = ['a', 'b', 1, 'c', 3, 6, 'h', 4]
>>> A = {x for x in X if isinstance(x, int) and x > 0 and x < 6}
>>> A
{1, 3, 4}
>>> type(A)
<class 'set'>
```

No obstante, muchas personas prescinden de formalizar un enunciado, y se arrojan directamente a crear listas por comprensión.

A continuación, se proponen algunos ejemplos más, a fin de lograr una mayor familiarización con el método de generación de listas por comprensión.

Ejemplo #1: A partir de una lista de palabras, generar una lista por comprensión con las palabras que comienzan por 'a'.

```
>>> palabras = [
    'beta', 'alga', 'gama', 'alma', 'zeta', 'arranque'
]
>>> lista = [p for p in palabras if p.startswith('a')]
>>> lista
['alga', 'alma', 'arranque']
```

La misma lista creada de forma tradicional:

```
>>> palabras = [
    'beta', 'alga', 'gama', 'alma', 'zeta', 'arranque'
]
>>> lista = []
>>> for p in palabras:
```

```
if p.startswith('a'):
    lista.append(p)
```

Ejemplo #2: A partir de una lista de números, generar una lista por comprensión solo con números enteros.

```
>>> numeros = [12.5, -6, 9, 12, -3.4, 0]
>>> lista = [n for n in numeros if isinstance(n, int)]
>>> lista
[-6, 9, 12, 0]
```

La misma lista creada de forma tradicional:

```
>>> numeros = [12.5, -6, 9, 12, -3.4, 0]
>>> lista = []
>>> for n in numeros:
    if isinstance(n, int):
        lista.append(p)
```

8.2 DICCIONARIOS POR COMPRENSIÓN

Además de listas y conjuntos, también pueden crearse diccionarios:

```
>>> datos = [
    ('Nombre', 'Juan'),
    ('Apellido', 'Perez'),
    ('Edad', 25)
]
>>> diccionario = {dato[0]: dato[1] for dato in datos}
>>> diccionario
{'nombre': 'Juan', 'Apellido': 'Perez', 'Edad': 25}
```

En los diccionarios por comprensión, tanto claves como valores se generan dinámicamente. En el ejemplo se ha empleado una lista cuyos elementos eran tuplas con pares de palabras. La primera palabra de cada tupla, ha sido utilizada como clave del diccionario, mientras que el segundo elementos de cada tupla, como valor.

8.3 SINTAXIS PARA COLECCIONES POR COMPRENSIÓN

La sintaxis para cada caso es la misma, y solo varían los elementos de la izquierda:

LISTA:

```
[<valor> for <valor> in <colección>[ <condición>]]
```

DICCIONARIO:

```
{<clave>: <valor> for <dato> in <colección>[ <condición>]}
```

Como puede observarse, la condición es un elemento opcional, y por tal motivo aparece entre corchetes.

Los elementos valor, colección, condición, clave y dato son valores variables, y por ello aparecen entre los signos menor que y mayor que.

La sintaxis pasada a ejemplo se vería como sigue:

LISTA (SIN CONDICIÓN):

```
[x for x in X]
```

LISTA (CON CONDICIÓN):

```
[x for x in X if x < 10]
```

DICCIONARIO (SIN CONDICIÓN):

```
{str(x): float(x) for x in X}
```

DICCIONARIO (CON CONDICIÓN):

```
{str(x): float(x) for x in X if x < 10}
```

9. FUNCIONES

Una función es una forma de agrupar expresiones y algoritmos de forma tal que estos queden contenidos dentro de «una cápsula», que solo pueda ejecutarse cuando el programador la invoque. Una vez que una función es definida, puede ser invocada tantas veces como sea necesario.

Funciones incorporadas

Una función puede ser provista por el lenguaje o definida por el usuario. A las funciones provistas por el lenguaje se las denomina «funciones incorporadas» y en inglés se conocen como «*Built-in functions*».

9.1 FUNCIONES DEFINIDAS POR EL USUARIO

En Python la definición de funciones se realiza mediante la instrucción **def** más un nombre de función descriptivo -para el cuál, aplican las mismas reglas que para el nombre de las variables- seguido de paréntesis de apertura y cierre.

Como toda estructura de control en Python, la definición de la función finaliza con dos puntos (:) y el algoritmo que la compone, irá indentado con 4 espacios en blanco:

```
def mi_funcion():  
    # aquí el algoritmo indentado
```

Una función no es ejecutada hasta tanto no sea invocada. Para invocar una función, simplemente se la llama por su nombre:

```
def mi_funcion():  
    print("Hola Mundo")
```

```
funcion()
```

Las funciones pueden **retornar datos**:

```
def funcion():  
    return "Hola Mundo"
```

Y el valor de retorno de una función puede **almacenarse** en una variable:

```
frase = funcion()
```

Imprimirse:

```
print(funcion())
```

O ignorarse:

```
funcion()
```

9.1.1 SOBRE LOS PARÁMETROS

Un parámetro es un valor que la función espera recibir cuando sea llamada (invocada). Una función puede esperar uno o más parámetros (que son definidos entre los paréntesis y separados por una coma) o ninguno.

```
def mi_funcion(param1, param2):  
    pass
```

La instrucción **pass** se utiliza para completar una estructura de control que no realiza ninguna acción.

Los parámetros que una función espera, serán utilizados por ésta, dentro de su algoritmo, a modo de **variables de ámbito local**. Es decir, que los parámetros serán variables locales que solo son accesibles dentro de la función:

```
def calcular_neto(bruto, alicuota):  
    iva = bruto * float(alicuota) / 100  
    neto = bruto + iva  
    return neto
```

Si se quisiera acceder a esas variables locales fuera del ámbito de la función, se obtendría un error:

```
def calcular_neto(bruto, alicuota):  
    iva = bruto * float(alicuota) / 100
```

```
neto = bruto + iva  
return neto
```

```
print(bruto)      # Retornará el error:  
                  # NameError: name 'bruto' is not defined
```

Al llamar a una función, **se le deben pasar sus argumentos en el mismo orden en el que los espera**. Para evitar pasarlos en el mismo orden, pueden utilizarse claves como argumentos (definidos más abajo).

9.1.2 PARÁMETROS POR OMISIÓN

Es posible asignar valores por defecto a los parámetros de las funciones. Esto significa, que la función podrá ser llamada con menos argumentos de los que ha definido:

```
def calcular_neto(bruto, alicuota=21):  
    iva = bruto * float(alicuota) / 100  
    neto = bruto + iva  
    return neto  
  
calcular_neto(100)      # Retorna 121.0  
calcular_neto(100, 10.5) # Retorna 110.5
```

PEP 8: Funciones

A la definición de una función la deben anteceder dos líneas en blanco.

Al asignar parámetros por omisión, no debe dejarse espacios en blanco ni antes ni después del signo =.

Los parámetros por omisión deben ser definidos a continuación de los parámetros obligatorios.

9.1.3 CLAVES COMO ARGUMENTOS

Las claves como argumentos (*keyword arguments*) son una característica de Python que no todos los lenguajes poseen. Python permite llamar a una función pasándole los argumentos esperados como pares de **claves=valor**:

```
def funcion(obligatorio1, opcional='valor por defecto',
opcional_dos=15):
    pass

funcion('valor obligatorio', opcional_dos=43)
```

9.1.4 PARÁMETROS ARBITRARIOS

Es posible que una función espere recibir un número arbitrario - desconocido- de argumentos. Estos argumentos, llegarán a la función en forma de tupla.

Para definir argumentos arbitrarios en una función, se antecede un asterisco (*) al nombre del parámetro :

```
def funcion(obligatorio, *arbitrarios):
    pass

funcion('fijo', 1, 2, 3, 4, 5)
funcion('fijo', 1, 2)
funcion('fijo', 1, 2, 3, 4, 5, 6, 7)
funcion('fijo')
```

Cuando una función espera recibir parámetros obligatorios y arbitrarios, **los arbitrarios siempre deben suceder a los obligatorios**.

Es posible también, obtener parámetros arbitrarios como pares de clave=valor. En estos casos, al nombre del parámetro deben precederlo dos asteriscos (**):

```
def funcion(obligatorio, **arbitrarios):  
    pass
```

```
funcion('fijo', a=1, b=2, c=3)
```

El recorrido de los parámetros arbitrarios es como el recorrido de cualquier tupla (para parámetros arbitrarios sin clave) o de cualquier diccionario (para los parámetros arbitrarios con clave):

```
def funcion(*arbitrarios):  
    for argumento in arbitrarios:  
        pass
```

```
def funcion(**arbitrarios):  
    for argumento in arbitrarios:  
        valor = arbitrarios[argumento]
```

9.1.5 DESEMPAQUETADO DE PARÁMETROS

Al invocar a una función se le pueden pasar los parámetros que espera dentro de una lista o de un diccionario (donde los nombres de las claves equivalen al nombre de los argumentos). A este procedimiento se lo conoce como desempaqueado de parámetros:

```
def funcion(unos, dos, tres):  
    pass
```

DESEMPAQUETADO DE LISTAS

```
parametros = [1, 2, 3]  
calcular(*parametros)
```

DESEMPAQUETADO DE DICCIONARIOS

```
parametros = dict(unos=1, dos=2, tres=3)  
calcular(**parametros)
```

```
parametros = {'unos': 1, 'dos': 2, 'tres': 3}  
calcular(**parametros)
```

9.1.6 LLAMADAS RECURSIVAS Y DE RETORNO

Una función puede llamar a otra función que retorne un valor. Esto se conoce como llamada de retorno:

```
def retornar(algo):  
    return str(algo)
```

```
def llamar():  
    algo = retornar()
```

La llamada interna a otra función puede almacenarse, retornarse o ignorarse.

```
def almacenar():  
    algo = retornar()
```

```
def volver_a_retornar():  
    return retornar()
```

```
def ignorar():  
    retornar()
```

Cuando la llamada que se hace es a la misma función, se conoce como **llamada recursiva**.

```
def get_nombre():  
    nombre = raw_input("Nombre: ")  
    if not nombre:  
        get_nombre()
```

9.2 SOBRE LA FINALIDAD DE LAS FUNCIONES

Una función puede tener cualquier tipo de algoritmo y cualquier cantidad de instrucciones. Sin embargo, **una buena práctica** indica que la finalidad de una función, debe ser **realizar una única acción**.

9.3 CONCEPTOS AVANZADOS SOBRE FUNCIONES

En esta sección se intentarán explicar cuatro conceptos sobre diferentes formas de definir e implementar una función. Estos conceptos no son estrictamente necesarios a la hora de programar, pero en Python suelen verse con frecuencia. Se exponen a título informativo.

9.3.1 LAMBDA

Las funciones lambda son funciones anónimas que permiten definir expresiones relativamente simples, generalmente en no más de una línea de código.

Una función como esta:

```
def sumar(sumandos=[]):  
    return sum(sumandos)
```

con lambda, podría definirse en una sola línea de código, ya que posee una única instrucción:

```
sumar = lambda sumandos: sum(sumandos)
```

Y se obtendría un resultado como el que sigue:

```
>>> sumar = lambda sumandos: sum(sumandos)  
>>> sumar([15, 30, 10])  
55
```

Su sintaxis puede resumirse de la siguiente forma:

```
variable = lambda parametro1, ..., parametron: expresión
```

Al igual que las funciones no anónimas, las funciones lambda admiten parámetros por omisión:

```
neto = lambda bruto, iva=21: bruto + (bruto * iva / 100)
```

9.3.2 CLAUSURAS (CLOSURES¹)

Las clausuras son funciones que dentro de ellas encierran a otra función y a su ámbito de aplicación, lo que permite el acceso a variables definidas dentro de la función interna, incluso fuera de su ámbito. Una clausura siempre retorna la función que define dentro. Obsérvese el siguiente ejemplo:

```
def closure():  
    def funcion_interna():  
        return 1  
  
    return funcion_interna
```

Al llamar a `closure()` lo que en realidad se obtiene es la función `funcion_interna`:

```
variable = closure() # Ahora variable es una función,  
                    # la función <funcion_interna>
```

Aquí *variable* ahora es una función (la función *funcion_intrna*) y puede ser invocada como una función:

```
print(variable()) # Imprimirá 1  
                 # Será como llamar a funcion_interna()
```

Esto es, a nivel práctico, la perspectiva desde el punto de vista sintáctico de una clausura, independiente de su finalidad.

¹ En español suelen referirse como “clausuras” aunque quizás “cierres” sería etimológicamente más apropiado. El término tiene su origen en la programación funcional y fue introducido por primera vez por Luca Cardelli en 1983.

Como se comentó previamente, se trata de funciones que definen otra función dentro y la retornan, y tienen la capacidad de reconocer y recordar el valor de variables y parámetros definidos dentro de la clausura ya que no solo encierran a la función sino también, su ámbito:

```
def closure(parametro):  
    def funcion():  
        return parametro + 1  
  
    return funcion  
  
variable = closure(parametro=1)  
print(variable()) # Imprime 2
```

Un ejemplo más funcional podría ser el siguiente:

```
def calcular_iva(alicuota):  
  
    def estimar_netto(importe_bruto):  
        return importe_bruto + (importe_bruto * alicuota / 100)  
  
    return estimar_netto  
  
# Productos gravados con el 21%  
get_netto_base_21 = calcular_iva(21)  
harina = get_netto_base_21(10)  
arroz = get_netto_base_21(8.75)  
azucar = get_netto_base_21(12.5)  
  
# Productos gravados con el 10.5%  
get_netto_base_105 = calcular_iva(10.5)  
tv = get_netto_base_105(12700)  
automovil = get_netto_base_105(73250)
```

El requerimiento anterior, podría haber sido implementado de dos formas sin el uso de clausuras:

1. Definiendo dos funciones independientes (get_netto_base_21 y get_netto_base_105) con el mismo algoritmo incurriendo en redundancia.

2. Definiendo una única función con 2 parámetros, la cual hubiese requerido replicar el valor de uno de los parámetros en las reiteradas llamadas, cuando el valor por defecto no fuese igual al valor necesario.

Lo anterior, podría servir de base para justificar el uso de una clausura:

- Evitar definir múltiples funciones con algoritmos redundantes.
- Evitar la sobrecarga de parámetros.

9.3.3 ENVOLTURAS (WRAPPERS) Y DECORADORES

Un *decorador* es una clausura que como parámetro recibe a una función (llamada función “decorada”) como único argumento:

```
def decorador(funcion_decorada):  
    def funcion():  
        pass  
  
    return funcion
```

Mientras que una *envoltura* (*wrapper*) es la función interna de una clausura que a la vez sea de tipo decorador:

```
def decorador(funcion_decorada):  
    def wrapper():  
        pass  
  
    return wrapper
```

La función decorada deberá ser invocada por la envoltura:

```
def decorador(funcion_decorada):  
    def wrapper():  
        return funcion_decorada()
```

```
return wrapper
```

El decorador no se invoca como una función de forma habitual, sino que lo hace con una sintaxis especial, sobre la definición de la función que se desea decorar:

```
@decorador
def funcion_decorada():
    pass
```

De esta forma, el nombre de la función decorada es pasado como parámetro de manera automática.

9.3.3.1 FUNCIONAMIENTO DE LAS ENVOLTURAS Y DECORADORES

Cuando una función es decorada, el decorador se acciona de forma automática en el momento que el *script* es ejecutado:

```
#!/usr/bin/env python3

def decorador(funcion):
    print('Soy el decorador()')

    def wrapper():
        print('Soy el wrapper()')
        return funcion()

    return wrapper

@decorador
def funcion_decorada():
    print('Soy la funcion_decorada()')
```

Al ejecutar este *script* sin haber invocado a ninguna función, se puede ver como el decorador ya ha actuado:

```
eugenia@host:~/scripts$ ./decoradores.py
Soy el decorador()
eugenia@host:~/scripts$
```

Cuando la función decorada es invocada, el decorador ya la habrá reemplazado por la envoltura, retornando a esta en lugar de la función original:

```
#!/usr/bin/env python3

def decorador(funcion):
    print('Soy el decorador()')

    def wrapper():
        print('Soy el wrapper()')
        # return funcion()

    return wrapper

@decorador
def funcion_decorada():
    print('Soy la funcion_decorada()')

funcion_decorada()
```

El resultado de la ejecución del *script* será el siguiente:

```
eugenia@host:~/scripts$ ./decoradores.py
Soy el decorador()
Soy el wrapper()
```

Como bien puede verse, al haber invocado a *funcion_decorada()*, no fue esta quien se ejecutó sino la función *wrapper()*.

Pero como se dijo al comienzo, la envoltura será quien tras su ejecución, invoque a la función decorada. Descomentando la línea *return funcion()*, el resultado sería el siguiente:

```
eugenia@host:~/scripts$ ./decoradores.py
Soy el decorador()
Soy el wrapper()
Soy la funcion_decorada()
```

El orden de ejecución en envolturas y decoradores se puede establecer como el siguiente:

1. decorador (automáticamente al ejecutar el *script* reemplazando la función decorada por la envoltura)
2. envoltura (al invocar a la función decorada)
3. función decorada (luego de ejecutarse la envoltura)

Puede decirse entonces que conceptualmente, un decorador es una clausura que se encarga de reemplazar a la función decorada por su función interna, a la cuál se denomina envoltura.

10. DOCSTRINGS (CADENAS DE DOCUMENTACIÓN)

La documentación del código fuente no se refiere al hecho de escribir líneas de comentarios en los archivos, sino a una práctica metódica mediante la cual se describen tres aspectos técnicos de las estructuras de control:

1. Acción u objetivo (qué hace).
2. Elementos que la conforman (desde variables y parámetros hasta módulos completos).
3. Resultados que pueden esperarse (qué retorna o produce).

Estos tres aspectos, por lo general, deberían ser suficientes para quien desee hacer uso de los mismos. No obstante, es posible complementarlos con ejemplos de uso.

Los elementos que conforman una documentación convencional, se describen en la siguiente tabla.

Elemento	Características	Ejemplo
Acción	Puede referirse tanto al procedimiento llevado a cabo por un método o función, como a la finalidad de una clase, módulo, o biblioteca.	En referencia a una función: <i>Calcular un número primo.</i> En referencia a una clase: <i>Base abstracta para objetos genéricos.</i> En referencia a una biblioteca: <i>Biblioteca para la generación y cómputo de números primos.</i>
Componentes	Se refiere a los datos que maneja la estructura. En el caso de funciones, se refiere a los parámetros que recibe. En el caso de las clases, puede hacer referencia a constantes de clase, e incluso a propiedades (dependiendo del contexto). En el caso de bibliotecas, puede referirse a variables y constantes disponibles. Si se trata de un módulo tipo script, también puede referirse a los argumentos que recibe.	Para este tipo de componentes suelen documentarse tres características: nombre de la variable, descripción (que bien puede referirse solo al rol que ocupa o bien al rol y/o al tipo de datos), y valor por defecto si lo hubiere: <i>alicuota -- índice para el cálculo del IVA (valor por defecto: 10.5)</i>
Retorno	Se refiere al valor de retorno tras invocar a una función o método de clase.	<i>Retorna el valor dado en el parámetro bruto multiplicado por la alícuota sobre 100.</i>

El código se documenta mediante el uso de cadenas de comentarios, que en inglés se denominan *docstrings*. En el caso de Python, su modo de escritura se encuentra definido en las *Python Enhancement Proposal* (PEP) 257².

En las PEP 257 se define una ***cadena de documentación*** (docstring) como una cadena de texto que se encuentra como primera

² <https://www.python.org/dev/peps/pep-0257/>

sentencia dentro de un módulo, función, clase o método, y que puede ser recuperada mediante el atributo especial `__doc__` de dicho elemento.

```
>>> def foo():  
...     """Hacer bar y retornar baz"""  
...     pass  
...  
>>> foo.__doc__  
'Hacer bar y retornar baz'
```

A la documentación de un componente también se puede acceder mediante la función `help()`:

```
>>> help(foo)
```

```
Help on function foo in module __main__:
```

```
foo()  
    Hacer bar y retornar baz  
(END)
```

Obsérvese que para **salir del modo ayuda** en la shell, debe pulsarse la tecla **q**.

La convención establecida en la PEP 257, establece que:

1. La cadena de documentación **debe ser lo primero que se especifique**. En un script, será la primera línea luego del hashbang. En una clase, método o función, la primera línea luego de la definición de la clase, método o función.

En un módulo:

```
#!/usr/bin/env python
"""Biblioteca para la generación y cómputo de números primos"""
```

En una clase:

```
class Primo(object):
    """Base para la generación de números primos"""

    def __init__(self, ...):
        ...
```

En una función (o método):

```
def obtener_primo_pseudoaleatorio():
    """Calcula un número primo menor que 100000 y lo retorna"""
    ...
```

2. Tanto en cadenas de documentación de una sola línea como de múltiples líneas se deben **emplear triples comillas**.

Cadenas de una sola línea:

```
def obtener_primo_pseudoaleatorio():
    """Calcula un número primo menor que 100000 y lo retorna"""
    ...
```

Cadenas de líneas múltiples:

```
def obtener_primo_pseudoaleatorio():
    """
    Calcula un número primo dentro del rango solicitado
    y lo retorna.
    """
    ...
```

3. En las cadenas de líneas múltiples se debe **especificar primero una descripción corta, separada de una línea en blanco** del resto de la documentación.

```
def obtener_primo_pseudoaleatorio():
    """
    Calcula un número primo y lo retorna.

    Calcula los números primos dentro de un rango solicitado
    y retorna uno de forma pseudoaleatoria.
    Cuando dentro del rango solicitado no sea posible hallar
    un número primo, devolverá None.

    """
```

4. Los argumentos y variables, así como cualquier otro elemento que se describa, deben **respetar el uso de mayúsculas y minúsculas** que poseen en el código.

```
def obtener_primo_pseudoaleatorio(limite=100000):
    """Calcula el último número primo menor que limite
    y lo retorna.

    limite -- entero que se empleará como límite máximo
              para calcular el número primo.
              (valor por defecto: 100000)

    """
    ...
```

Notar que en el último ejemplo, los argumentos de una función se describen con la siguiente **sintaxis**:

argumento -- descripción (valor por defecto)

11. MANIPULACIÓN DE CADENAS DE TEXTO

En Python, toda variable se considera un *objeto*. Sobre cada objeto, pueden realizarse diferentes tipos de acciones denominadas *métodos*. Los métodos son funciones pero que se desprenden de una variable. Por ello, se accede a estas funciones mediante la sintaxis:

```
variable.funcion()
```

En algunos casos, estos métodos (funciones de un objeto), aceptarán parámetros como cualquier otra función.

```
variable.funcion(parametro)
```

11.1 INYECCIÓN DE VARIABLES

Una variable se «inyecta» en una cadena de texto haciendo que su valor pase a formar parte de la cadena. Esto se hace mediante una operación de **formato**. Esto es necesario cuando la cadena a ser inyectada debe contener datos que son variables.

Para inyectar variables dentro de cadenas, las cadenas deben ser preparadas mediante el uso de modificadores. Un **modificador** puede ser un par de llaves vacías {} o un par de llaves con nombre {nombre}.

```
cadena = "Cadena preparada para recibir dos datos variables: {}  
y {}."
```

La misma cadena con modificadores con nombre:

```
cadena = "Cadena preparada para recibir dos datos variables:  
{dato1} y {dato2}."
```

Luego, se da formato a la cadena pasando una lista de variables que serán enlazadas a cada modificador:

```
cadena = "Cadena preparada para recibir dos datos variables: {}  
y {}."  
resultado = cadena.format(variable1, variable2)
```

Y si tiene modificadores con nombre:

```
cadena = "Cadena preparada para recibir dos datos variables:  
{dato1} y {dato2}."  
resultado = cadena.format(dato1=variable1, dato2=variable2)
```

La función **format()** es un método del objeto string. Los métodos son funciones. Los objetos, variables de un tipo de datos más complejo. Todo esto será abarcado en profundidad, en los cursos de *Scripting*.

11.2 MÉTODOS DE FORMATO

11.2.1 CONVERTIR A MAYÚSCULA LA PRIMERA LETRA

Método: `capitalize()`

Retorna: una copia de la cadena con la primera letra en mayúsculas

```
>>> cadena = "bienvenido a mi aplicación"
>>> resultado = cadena.capitalize()
>>> resultado
Bienvenido a mi aplicación
```

11.2.2 CONVERTIR UNA CADENA A MINÚSCULAS

Método: `lower()`

Retorna: una copia de la cadena en minúsculas

```
>>> cadena = "Hola Mundo"
>>> cadena.lower()
hola mundo
```

11.2.3 CONVERTIR UNA CADENA A MAYÚSCULAS

Método: `upper()`

Retorna: una copia de la cadena en mayúsculas

```
>>> cadena = "Hola Mundo"
>>> cadena.upper()
HOLA MUNDO
```

11.2.4 CONVERTIR MAYÚSCULAS A MINÚSCULAS Y VICEVERSA

Método: `swapcase()`

Retorna: una copia de la cadena convertidas las mayúsculas en minúsculas y viceversa

```
>>> cadena = "Hola Mundo"
>>> cadena.swapcase()
hOLA mUNDO
```

11.2.5 CONVERTIR UNA CADENA EN FORMATO TÍTULO

Método: `title()`

Retorna: una copia de la cadena convertida

```
>>> cadena = "hola mundo"
>>> cadena.title()
Hola Mundo
```

11.2.6 CENTRAR UN TEXTO

Método: `center(longitud[, "caracter de relleno"])`

Retorna: una copia de la cadena centrada

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.center(50, "=")
=====Bienvenido a mi aplicación=====
>>> cadena.center(50, " ")
                Bienvenido a mi aplicación
```

11.2.7 ALINEAR TEXTO A LA IZQUIERDA

Método: `ljust(longitud[, "caracter de relleno"])`

Retorna: una copia de la cadena alineada a la izquierda

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.ljust(50, "=")
Bienvenido a mi aplicación=====
```

11.2.8 ALINEAR TEXTO A LA DERECHA

Método: `rjust(longitud[, "caracter de relleno"])`

Retorna: una copia de la cadena alineada a la derecha

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.rjust(50, "=")
=====Bienvenido a mi aplicación
>>> cadena.rjust(50, " ")
                Bienvenido a mi aplicación
```

11.2.9 RELLENAR UN TEXTO ANTEPONIENDO CEROS

Método: `zfill(longitud)`

Retorna: una copia de la cadena rellena con ceros a la izquierda hasta alcanzar la longitud final indicada

```
>>> numero_factura = 1575
>>> str(numero_factura).zfill(12)
000000001575
```

11.3 MÉTODOS DE BÚSQUEDA

11.3.1 CONTAR CANTIDAD DE APARICIONES DE UN FRAGMENTO DE TEXTO

Método: `count("subcadena"[, posicion_inicio, posicion_fin])`

Retorna: un entero representando la cantidad de apariciones de *subcadena* dentro de *cadena*

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.count("a")
3
```

11.3.2 BUSCAR UN FRAGMENTO DE TEXTO DENTRO DE UNA CADENA

Método: `find("subcadena"[, posicion_inicio, posicion_fin])`

Retorna: un entero representando la posición donde inicia la *subcadena* dentro de cadena. Si no la encuentra, retorna -1

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.find("mi")
13
>>> cadena.find("mi", 0, 10)
-1
```

11.4 MÉTODOS DE VALIDACIÓN

11.4.1 SABER SI UNA CADENA COMIENZA POR UN TEXTO DETERMINADO

Método: `startswith("subcadena"[, posicion_inicio, posicion_fin])`

Retorna: True o False

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.startswith("Bienvenido")
True
>>> cadena.startswith("aplicación")
False
>>> cadena.startswith("aplicación", 16)
True
```

11.4.2 SABER SI UNA CADENA FINALIZA CON UN TEXTO DETERMINADO

Método: `endswith("subcadena"[, posicion_inicio, posicion_fin])`

Retorna: True o False

```
>>> cadena = "bienvenido a mi aplicación".capitalize()
>>> cadena.endswith("aplicación")
True
>>> cadena.endswith("Bienvenido")
```

```
False
>>> cadena.endswith("Bienvenido", 0, 10)
True
```

11.4.3 SABER SI UNA CADENA ES ALFANUMÉRICA

Método: `isalnum()`

Retorna: True o False

```
>>> cadena = "pepegrillo 75"
>>> cadena.isalnum()
False
>>> cadena = "pepegrillo"
>>> cadena.isalnum()
True
>>> cadena = "pepegrillo75"
>>> cadena.isalnum()
True
```

11.4.4 SABER SI UNA CADENA ES ALFABÉTICA

Método: `isalpha()`

Retorna: True o False

```
>>> cadena = "pepegrillo 75"
>>> cadena.isalpha()
False
>>> cadena = "pepegrillo"
>>> cadena.isalpha()
True
>>> cadena = "pepegrillo75"
>>> cadena.isalpha()
False
```

11.4.5 SABER SI UNA CADENA ES NUMÉRICA

Método: `isdigit()`

Retorna: True o False

```
>>> cadena = "pepegrillo 75"
>>> cadena.isdigit()
False
>>> cadena = "7584"
>>> cadena.isdigit()
True
>>> cadena = "75 84"
>>> cadena.isdigit()
False
>>> cadena = "75.84"
>>> cadena.isdigit()
False
```

11.4.6 SABER SI UNA CADENA CONTIENE SOLO MINÚSCULAS

Método: `islower()`

Retorna: True o False

```
>>> cadena = "pepe grillo"
>>> cadena.islower()
True
>>> cadena = "Pepe Grillo"
>>> cadena.islower()
False
>>> cadena = "Pepegrillo"
>>> cadena.islower()
False
>>> cadena = "pepegrillo75"
>>> cadena.islower()
True
```

11.4.7 SABER SI UNA CADENA CONTIENE SOLO MAYÚSCULAS

Método: `isupper()`

Retorna: True o False

```
>>> cadena = "PEPE GRILLO"
>>> cadena.isupper()
True
>>> cadena = "Pepe Grillo"
>>> cadena.isupper()
False
>>> cadena = "Pepegrillo"
```

```
>>> cadena.isupper()  
False  
>>> cadena = "PEPEGRILLO"  
>>> cadena.isupper()  
True
```

11.4.8 SABER SI UNA CADENA CONTIENE SOLO ESPACIOS EN BLANCO

Método: `isspace()`

Retorna: True o False

```
>>> cadena = "pepe grillo"  
>>> cadena.isspace()  
False  
>>> cadena = "      "  
>>> cadena.isspace()  
True
```

11.4.9 SABER SI UNA CADENA TIENE FORMATO TIPO TÍTULO

Método: `istitle()`

Retorna: True o False

```
>>> cadena = "Pepe Grillo"  
>>> cadena.istitle()  
True  
>>> cadena = "Pepe grillo"  
>>> cadena.istitle()  
False
```

11.5 MÉTODOS DE SUSTITUCIÓN

11.5.1 DAR FORMATO A UNA CADENA, SUSTITUYENDO TEXTO DINÁMICAMENTE

Método: `format(*args, **kwargs)`

Retorna: la cadena formateada

```
>>> cadena = "bienvenido a mi aplicación {0}"
>>> cadena.format("en Python")
bienvenido a mi aplicación en Python

>>> cadena = "Importe bruto: ${0} + IVA: ${1} = Importe
neto: {2}"
>>> cadena.format(100, 21, 121)
Importe bruto: $100 + IVA: $21 = Importe neto: 121

>>> cadena = "Importe bruto: ${bruto} + IVA: ${iva} =
Importe neto: {neto}"
>>> cadena.format(bruto=100, iva=21, neto=121)
Importe bruto: $100 + IVA: $21 = Importe neto: 121

>>> cadena.format(bruto=100, iva=100 * 21 / 100, neto=100 * 21 /
100 + 100)
Importe bruto: $100 + IVA: $21 = Importe neto: 121
```

11.5.2 REEMPLAZAR TEXTO EN UNA CADENA

Método: `replace("buscada", "reemplazo")`

Retorna: la cadena reemplazada

```
>>> buscada = "nombre apellido"
>>> reemplazo = "Juan Pérez"
>>> "Estimado Sr. nombre apellido:".replace(buscada,
reemplazo)
Estimado Sr. Juan Pérez:
```

11.5.3 ELIMINAR CARACTERES A LA IZQUIERDA Y DERECHA DE UNA CADENA

Método: `strip(["caracter"])`

Retorna: la cadena sustituida

```
>>> cadena = " www.eugeniabahit.co.uk "
>>> cadena.strip()
www.eugeniabahit.co.uk
>>> cadena.strip(' ')
www.eugeniabahit.co.uk
```

11.5.4 ELIMINAR CARACTERES A LA IZQUIERDA DE UNA CADENA

Método: `lstrip(["caracter"])`

Retorna: la cadena sustituida

```
>>> cadena = "www.eugeniabahit.co.uk"
>>> cadena.lstrip("w." )
eugeniabahit.co.uk

>>> cadena = "          www.eugeniabahit.co.uk"
>>> cadena.lstrip()
www.eugeniabahit.co.uk
```

11.5.5 ELIMINAR CARACTERES A LA DERECHA DE UNA CADENA

Método: `rstrip(["caracter"])`

Retorna: la cadena sustituida

```
>>> cadena = "www.eugeniabahit.co.uk      "
>>> cadena.rstrip( )
www.eugeniabahit.co.uk
```

11.6 MÉTODOS DE UNIÓN Y DIVISIÓN

11.6.1 UNIR UNA CADENA DE FORMA ITERATIVA

Método: `join(iterable)`

Retorna: la cadena unida con el iterable (la cadena es separada por cada uno de los elementos del iterable)

```
>>> rellenos = ("Nº 0000-0", "-0000 (ID: ", ")")
>>> numero = "275"
>>> numero_factura = numero.join(rellenos)
>>> numero_factura
Nº 0000-0275-0000 (ID: 275)
```

11.6.2 PARTIR UNA CADENA EN TRES PARTES, UTILIZANDO UN SEPARADOR

Método: `partition("separador")`

Retorna: una tupla de tres elementos donde el primero es el contenido de la cadena previo al separador, el segundo, el separador mismo y el tercero, el contenido de la cadena posterior al separador.

```
>>> url = "https://www.eugeniabahit.co.uk"
>>> tupla = url.partition("www.")
>>> tupla
('https://', 'www.', 'eugeniabahit.co.uk')

>>> protocolo, separador, dominio = tupla
>>>> "Protocolo: {0}\nDominio: {1}".format(protocolo,
dominio)
Protocolo: https://
Dominio: eugeniabahit.co.uk
```

11.6.3 PARTIR UNA CADENA EN VARIAS PARTES, UTILIZANDO UN SEPARADOR

Método: `split("separador")`

Retorna: una lista con todos elementos encontrados al dividir la cadena por un separador

```
>>> keywords = "python, guia, curso".split(", ")
>>> keywords
['python', 'guia', 'curso']
```

11.6.4 PARTIR UNA CADENA EN EN LÍNEAS

Método: `splitlines()`

Retorna: una lista donde cada elemento es una fracción de la cadena divida en líneas

```
>>> texto = """Linea 1
Linea 2
```

```
Linea 3
Linea 4
"""
>>> texto.splitlines()
['Linea 1', 'Linea 2', 'Linea 3', 'Linea 4']

>>> texto = "Linea 1\nLinea 2\nLinea 3"
>>> texto.splitlines()
['Linea 1', 'Linea 2', 'Linea 3']
```

12. MANIPULACIÓN DE LISTAS Y TUPLAS

En este capítulo, se verán los métodos que posee el objeto *lista*. Algunos de ellos, también se encuentran disponibles para las *tuplas*.

12.1 MÉTODOS DE AGREGADO

12.1.1 AGREGAR UN ELEMENTO AL FINAL DE LA LISTA

Método: `append("nuevo elemento")`

```
>>> nombres_masculinos = ["Alvaro", "Jacinto", "Miguel",
"Edgardo", "David"]
>>> nombres_masculinos.append("Jose")
>>> nombres_masculinos
['Alvaro', 'David', 'Edgardo', 'Jacinto', 'Jose', 'Ricky',
'Jose']
```

12.1.2 AGREGAR VARIOS ELEMENTOS AL FINAL DE LA LISTA

Método: `extend(otra_lista)`

```
>>> nombres_masculinos.extend(["Jose", "Gerardo"])
>>> nombres_masculinos
['Alvaro', 'David', 'Edgardo', 'Jacinto', 'Jose', 'Ricky',
'Jose', 'Jose', 'Gerardo']
```

12.1.3 AGREGAR UN ELEMENTO EN UNA POSICIÓN DETERMINADA

Método: `insert(posición, "nuevo elemento")`

```
>>> nombres_masculinos.insert(0, "Ricky")
>>> nombres_masculinos
['Ricky', 'Alvaro', 'David', 'Edgardo', 'Jacinto', 'Jose',
'Ricky', 'Jose', 'Jose', 'Gerardo']
```

12.2 MÉTODOS DE ELIMINACIÓN

12.2.1 ELIMINAR EL ÚLTIMO ELEMENTO DE LA LISTA

Método: `pop()`

Retorna: el elemento eliminado

```
>>> nombres_masculinos.pop()
'Gerardo'
>>> nombres_masculinos
['Ricky', 'Alvaro', 'David', 'Edgardo', 'Jacinto', 'Jose',
'Ricky', 'Jose', 'Jose']
```

12.2.2 ELIMINAR UN ELEMENTO POR SU ÍNDICE

Método: `pop(índice)`

Retorna: el elemento eliminado

```
>>> nombres_masculinos.pop(3)
'Edgardo'
>>> nombres_masculinos
['Ricky', 'Alvaro', 'David', 'Jacinto', 'Jose', 'Ricky', 'Jose',
'Jose']
```

12.2.3 ELIMINAR UN ELEMENTO POR SU VALOR

Método: `remove("valor")`

```
>>> nombres_masculinos.remove("Jose")
```

```
>>> nombres_masculinos  
['Ricky', 'Alvaro', 'David', 'Jacinto', 'Ricky', 'Jose', 'Jose']
```

12.3 MÉTODOS DE ORDEN

12.3.1 ORDENAR UNA LISTA EN REVERSA (INVERTIR ORDEN)

Método: `reverse()`

```
>>> nombres_masculinos.reverse()  
>>> nombres_masculinos  
['Jose', 'Jose', 'Ricky', 'Jacinto', 'David', 'Alvaro', 'Ricky']
```

12.3.2 ORDENAR UNA LISTA EN FORMA ASCENDENTE

Método: `sort()`

```
>>> nombres_masculinos.sort()  
>>> nombres_masculinos  
['Alvaro', 'David', 'Jacinto', 'Jose', 'Jose', 'Ricky', 'Ricky']
```

12.3.3 ORDENAR UNA LISTA EN FORMA DESCENDENTE

Método: `sort(reverse=True)`

```
>>> nombres_masculinos.sort(reverse=True)  
>>> nombres_masculinos  
['Ricky', 'Ricky', 'Jose', 'Jose', 'Jacinto', 'David', 'Alvaro']
```

12.4 MÉTODOS DE BÚSQUEDA

12.4.1 CONTAR CANTIDAD DE APARICIONES ELEMENTOS

Método: `count(elemento)`

```
>>> nombres_masculinos = ["Alvaro", "Miguel", "Edgardo",  
"David", "Miguel"]  
>>> nombres_masculinos.count("Miguel")  
2
```

```
>>> nombres_masculinos = ("Alvaro", "Miguel", "Edgardo",  
"David", "Miguel")  
>>> nombres_masculinos.count("Miguel")  
2
```

12.4.2 OBTENER NÚMERO DE ÍNDICE

Método: `index(elemento[, indice_inicio, indice_fin])`

```
>>> nombres_masculinos.index("Miguel")  
1  
  
>>> nombres_masculinos.index("Miguel", 2, 5)  
4
```

12.5 ANEXO SOBRE LISTAS Y TUPLAS

12.5.1 CONVERSIÓN DE TIPOS

En el conjunto de las funciones integradas de Python, es posible encontrar dos funciones que permiten convertir listas en tuplas, y viceversa. Estas funciones son *list* y *tuple*, para convertir tuplas a listas y listas a tuplas, respectivamente.

Uno de los usos más frecuentes es el de conversión de tuplas a listas, que requieran ser modificadas. Esto sucede a menudo con los resultados obtenidos a partir de una consulta a base de datos.

```
>>> tupla = (1, 2, 3, 4)  
>>> tupla  
(1, 2, 3, 4)  
  
>>> list(tupla)  
[1, 2, 3, 4]  
  
>>> lista = [1, 2, 3, 4]  
>>> lista  
[1, 2, 3, 4]  
  
>>> tuple(lista)  
(1, 2, 3, 4)
```

12.5.2 CONCATENACIÓN DE COLECCIONES

Se pueden concatenar (o unir) dos o más listas o dos o más tuplas, mediante el signo de adición +.

No puede unirse una lista a una tupla. Las colecciones a unir deben ser del mismo tipo.

```
>>> lista1 = [1, 2, 3, 4]
>>> lista2 = [3, 4, 5, 6, 7, 8]
>>> lista3 = lista1 + lista2
>>> lista3
[1, 2, 3, 4, 3, 4, 5, 6, 7, 8]

>>> tupla1 = (1, 2, 3, 4, 5)
>>> tupla2 = (4, 6, 8, 10)
>>> tupla3 = (3, 5, 7, 9)
>>> tupla4 = tupla1 + tupla2 + tupla3
>>> tupla4
(1, 2, 3, 4, 5, 4, 6, 8, 10, 3, 5, 7, 9)
```

12.5.3 VALOR MÁXIMO Y MÍNIMO

Se puede obtener el valor máximo y mínimo tanto de listas como de tuplas:

```
>>> max(tupla4)
10
>>> max(tupla1)
5
>>> min(tupla1)
1
>>> max(lista3)
8
>>> min(lista1)
1
```

12.5.4 CONTAR ELEMENTOS

La función `len()` sirve tanto para contar elementos de una lista o tupla, como caracteres de una cadena de texto:

```
>>> len(lista3)
10
>>> len(lista1)
4
```

13. MANIPULACIÓN DE DICCIONARIOS

13.1 MÉTODOS DE ELIMINACIÓN

13.1.1 VACIAR UN DICCIONARIO

Método: `clear()`

```
>>> diccionario = {"color": "violeta", "talle": "XS", "precio":
174.25}
>>> diccionario
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> diccionario.clear()
>>> diccionario
{}
```

13.2 MÉTODOS DE AGREGADO Y CREACIÓN

13.2.1 COPIAR UN DICCIONARIO

Método: `copy()`

```
>>> diccionario = {"color": "violeta", "talle": "XS", "precio":
174.25}
>>> camiseta = diccionario.copy()
>>> diccionario
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> camiseta
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> diccionario.clear()
>>> diccionario
{}
```

```
>>> camiseta
```

```
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> musculosa = camiseta
>>> camiseta
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> musculosa
{'color': 'violeta', 'precio': 174.25, 'talle': 'XS'}

>>> camiseta.clear()
>>> camiseta
{}

>>> musculosa
{}
```

13.2.2 CREAR UN NUEVO DICCIONARIO DESDE LAS CLAVES DE UNA SECUENCIA

Método: `dict.fromkeys(sequencia[, valor por defecto])`

```
>>> secuencia = ["color", "talle", "marca"]
>>> diccionario1 = dict.fromkeys(secuencia)
>>> diccionario1
{'color': None, 'marca': None, 'talle': None}

>>> diccionario2 = dict.fromkeys(secuencia, 'valor x defecto')
>>> diccionario2
{'color': 'valor x defecto', 'marca': 'valor x defecto',
'talle': 'valor x defecto'}
```

13.2.3 CONCATENAR DICCIONARIOS

Método: `update(diccionario)`

```
>>> diccionario1 = {"color": "verde", "precio": 45}
>>> diccionario2 = {"talle": "M", "marca": "Lacoste"}
>>> diccionario1.update(diccionario2)
>>> diccionario1
{'color': 'verde', 'precio': 45, 'marca': 'Lacoste', 'talle':
'M'}
```

13.2.4 ESTABLECER UNA CLAVE Y VALOR POR DEFECTO

Método: `setdefault("clave", [None|valor_por_defecto])`

Si la clave no existe, la crea con el valor por defecto. Siempre retorna el valor para la clave pasada como parámetro.

```
>>> camiseta = {"color": "rosa", "marca": "Zara"}
>>> clave = camiseta.setdefault("talle", "U")
>>> clave
'U'

>>> camiseta
{'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}

>>> camiseta2 = camiseta.copy()
>>> camiseta2
{'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}

>>> clave = camiseta2.setdefault("estampado")
>>> clave
>>> camiseta2
{'color': 'rosa', 'estampado': None, 'marca': 'Zara', 'talle': 'U'}

>>> clave = camiseta2.setdefault("marca", "Lacoste")
>>> clave
'Zara'

>>> camiseta2
{'color': 'rosa', 'estampado': None, 'marca': 'Zara', 'talle': 'U'}
```

13.3 MÉTODOS DE RETORNO

13.3.1 OBTENER EL VALOR DE UNA CLAVE

Método: `get(clave[, "valor x defecto si la clave no existe"])`

```
>>> camiseta.get("color")
'rosa'
```

```
>>> camiseta.get("stock")
>>> camiseta.get("stock", "sin stock")
'sin stock'
```

13.3.2 SABER SI UNA CLAVE EXISTE EN EL DICcionario

Método: 'clave' in diccionario

```
>>> existe = 'precio' in camiseta
>>> existe
False

>>> existe = 'color' in camiseta
>>> existe
True
```

13.3.3 OBTENER LAS CLAVES Y VALORES DE UN DICcionario

Método: items()

```
diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle':
'U'}
```

```
for clave, valor in diccionario.items():
    clave, valor
```

```
Salida:
('color', 'rosa')
('marca', 'Zara')
('talle', 'U')
```

En Python 2 existía iteritems():

```
>>> a = dict(a=1, b=2)
>>> a.iteritems()
```

En Python 3 ya no existe:

```
>>> a.iteritems()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'dict' object has no attribute 'iteritems'
```

Debe emplearse `items()` para generar código híbrido. No obstante, tener en cuenta que los objetos retornados se verán de forma diferente en ambas versiones.

Python 3:

```
>>> a.items()
dict_items([('a', 1), ('b', 9)])
```

Python 2:

```
>>> a.items()
[('a', 1), ('b', 9)]
```

Sin embargo, se itera igual en las dos:

```
for tupla in a.items():
    tupla

('a', 1)
('b', 9)
```

13.3.4 OBTENER LAS CLAVES DE UN DICCIONARIO

Método: `keys()`

```
diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}
for clave in diccionario.keys():
    clave
'marca'
'talle'
'color'
```

Obtener claves en una lista:

```
>>> diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}
>>> claves = list(diccionario.keys())
>>> claves
['color', 'marca', 'talle']
```

13.3.5 OBTENER LOS VALORES DE UN DICCIONARIO

Método: `values()`

```
diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}  
for clave in diccionario.values():  
    clave  
  
'rosa'  
'Zara'  
'U'
```

Obtener valores en una lista:

```
diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}  
claves = list(diccionario.values())  
  
Salida:  
['Zara', 'U', 'rosa']
```

13.3.6 OBTENER LA CANTIDAD DE ELEMENTOS DE UN DICCIONARIO

Para contar los elementos de un diccionario, al igual que con las listas y tuplas, se utiliza la función integrada `len()`

```
diccionario = {'color': 'rosa', 'marca': 'Zara', 'talle': 'U'}  
len(diccionario)  
# Salida: 3
```

14. MANEJO Y MANIPULACIÓN DE ARCHIVOS

Python permite trabajar en dos niveles diferentes con respecto al sistema de archivos y directorios.

Uno de ellos, es a través del módulo `os`, que facilita el trabajo con todo el sistema de archivos y directorios, a nivel del propios Sistema Operativo.

El segundo nivel, es el que permite trabajar con archivos manipulando su lectura y escritura desde la propia aplicación o *script*, tratando a cada archivo como un objeto.

14.1 MODOS DE APERTURA DE UN ARCHIVO

El **modo de apertura de un archivo**, está relacionado con el objetivo final que responde a la pregunta "*¿para qué se está abriendo este archivo?*". Las respuestas pueden ser varias: para leer, para escribir, o para leer y escribir.

Cada vez que se "abre" un archivo se está creando un ***puntero en memoria***.

Este puntero posicionará un ***cursor*** (o *punto de acceso*) en un lugar específico de la memoria (dicho de modo más simple, posicionará el cursor en un *byte* determinado del contenido del archivo).

Este cursor se moverá dentro del archivo, a medida que se lea o escriba en dicho archivo.

Cuando un archivo se abre en modo lectura, el cursor se posiciona en el *byte 0* del archivo (es decir, al comienzo del archivo). Una vez leído el archivo, el cursor pasa al *byte final* del archivo (equivalente a cantidad total de *bytes* del archivo). Lo mismo sucede cuando se abre en modo escritura. El cursor se moverá a medida que se va escribiendo.

Cuando se desea escribir al final de un archivo no nulo, se utiliza el modo *append* (agregar). De esta forma, el archivo se abre con el cursor al final del archivo.

El símbolo + como sufijo de un modo, agrega el modo contrario al de apertura una vez se ejecute la acción de apertura. Por ejemplo, el modo

r (read) con el sufijo + (r+), abre el archivo para lectura, y tras la lectura, vuelve el cursor al byte 0.

La siguiente tabla muestra los diferentes modos de apertura para un archivo abierto con la función *open*.

Indicador	Modo de apertura	Ubicación del puntero
r	Solo lectura	Al inicio del archivo
rb	Solo lectura en modo binario	Al inicio del archivo
r+	Lectura y escritura	Al inicio del archivo
rb+	Lectura y escritura en modo binario	Al inicio del archivo
w	Solo escritura. Sobrescribe el archivo si existe. Crea el archivo si no existe.	Al inicio del archivo
wb	Solo escritura en modo binario. Sobrescribe el archivo si existe. Crea el archivo si no existe.	Al inicio del archivo
w+	Escritura y lectura. Sobrescribe el archivo si existe. Crea el archivo si no existe.	Al inicio del archivo
wb+	Escritura y lectura en modo binario. Sobrescribe el archivo si existe. Crea el archivo si no existe.	Al inicio del archivo
a	Añadido (agregar contenido). Crea el archivo si éste no existe.	Si el archivo existe, al final de éste. Si el archivo no existe, al comienzo.
ab	Añadido en modo binario (agregar contenido). Crea el archivo si éste no existe.	Si el archivo existe, al final de éste. Si el archivo no existe, al comienzo.
a+	Añadido (agregar contenido) y lectura. Crea el archivo si éste no existe.	Si el archivo existe, al final de éste. Si el archivo no existe, al comienzo.
w+	Escritura y lectura. Sobrescribe el archivo si existe. Crea el archivo si no existe.	Al inicio del archivo

Indicador	Modo de apertura	Ubicación del puntero
ab+	Añadido (agregar contenido) y lectura en modo binario. Crea el archivo si éste no existe.	Si el archivo existe, al final de éste. Si el archivo no existe, al comienzo.

14.2 ALGUNOS MÉTODOS DEL OBJETO FILE

El objeto file, entre sus métodos dispone de los siguientes (no todos los métodos del objeto *file* se listan a continuación):

Método	Descripción
<code>read([bytes])</code>	Lee todo el contenido de un archivo. Si se le pasa la longitud de bytes, leerá solo el contenido hasta la longitud indicada.
<code>readlines()</code>	Lee todas las líneas de un archivo
<code>write(cadena)</code>	Escribe <i>cadena</i> dentro del archivo
<code>writelines(secuencia)</code>	Secuencia será cualquier iterable cuyos elementos serán escritos uno por línea

14.3 ACCESO A ARCHIVOS MEDIANTE LA ESTRUCTURA WITH

Con la estructura with y la función open(), puede abrirse un archivo en cualquier modo y trabajar con él, sin necesidad de cerrarlo o destruir el puntero, ya que de esto se encarga la estructura with.

Leer un archivo:

```
with open("archivo.txt", "r") as archivo:  
    contenido = archivo.read()
```

Escribir en un archivo:

```
contenido = ""
    Este será el contenido del nuevo archivo.
    El archivo tendrá varias líneas.
""

with open("archivo.txt", "r") as archivo:
    archivo.write(contenido)
```

15. MANEJO DE ARCHIVOS CSV

El formato **CSV** deriva su nombre del inglés «*comma separated values*» (valores separados por coma), definido en las RFC 4180³. Se trata de archivos de texto plano, destinados al almacenamiento masivo de datos. Es uno de los formatos más simples para efectuar análisis de datos. De hecho, muchos formatos de archivo no libres (o libres pero más complejos), suelen pasarse a formato CSV para aplicar ciencia de datos compleja con diversos lenguajes.

Un archivo CSV se encuentra formado por una cabecera que define nombres de columnas, y las filas siguientes, tienen los datos correspondientes a cada columna, separados por una coma. Sin embargo, muchos otros símbolos pueden utilizarse como separadores de celdas. Entre ellos, el tabulado y el punto y coma son igual de frecuentes que la coma.

15.1 ALGUNOS EJEMPLOS DE ARCHIVOS CSV

Datos meteorológicos (separados por ;)

```
ID;DATA;VV;DV;T;HR;PPT;RS;P
0;2016-03-01 00:00:00;;;9.9;73;;;
1;2016-03-01 00:30:00;;;9.0;67;;;
2;2016-03-01 01:00:00;;;8.3;64;;;
3;2016-03-01 01:30:00;;;8.0;61;;;
4;2016-03-01 02:00:00;;;7.4;62;;;
5;2016-03-01 02:30:00;;;8.3;47;;;
```

3 <https://tools.ietf.org/rfc/rfc4180.txt>

```
6;2016-03-01 03:00:00;;;7.7;50;;;
7;2016-03-01 03:30:00;;;9.0;39;;;
```

Puntajes obtenidos por jugadores de un torneo (separados por coma)

```
nombre,cantidad,anio
Maria,858,1930
Jose,665,1930
Rosa,591,1930
Juan Carlos,522,1930
Antonio,509,1930
Maria Esther,495,1930
Maria Luisa,470,1930
Juana,453,1930
Juan,436,1930
```

*Empresas registradas en la Inspección General de Justicia de Argentina
(separados por , y datos entrecomillados)*

```
"numero_correlativo","tipo_societario","descripcion_tipo_societa
rio","razon_social","dada_de_baja","codigo_baja","detalle_baja"
"10","10","SOCIEDAD COLECTIVA","A A VALLE Y
COMPANIA","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"11","10","SOCIEDAD COLECTIVA","A LUCERO Y H
CARATOLI","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"12","10","SOCIEDAD COLECTIVA","A PUIG E
HIJOS","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"13","10","SOCIEDAD COLECTIVA","A C I C
A","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"14","10","SOCIEDAD COLECTIVA","AÑON BEATRIZ S Y
CIA","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"15","10","SOCIEDAD COLECTIVA","ABA
DIESEL","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"16","10","SOCIEDAD COLECTIVA","ABADA L JOSE Y JORGE JOSE
ABADAL","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"17","10","SOCIEDAD COLECTIVA","ABADAL JOSE E
HIJO","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
"18","10","SOCIEDAD COLECTIVA","ABATE Y
MACIAS","S","42014","PERTENECE A REGISTRO ENTIDADES INACTIVAS"
```

Es posible también, encontrar datos almacenados en archivos de texto (TXT) con formatos muy similares al que se espera encontrar en un CSV.

A veces es posible desarrollar un script de formato para corregir estos archivos y así poder trabajar con un CSV.

Observaciones meteorológicas en TXT

FECHA	TMAX	TMIN	NOMBRE
07122017	28.0	19.0	AEROPARQUE AERO
07122017	26.8	12.4	AZUL AERO
07122017	29.6	7.8	BAHIA BLANCA AERO
07122017	22.7	6.7	BARILOCHE AERO
07122017	3.0	-8.5	BASE BELGRANO II
07122017	2.4	-0.2	BASE CARLINI (EX JUBANY)
07122017	3.9	-0.6	BASE ESPERANZA
07122017	0.7	-3.6	BASE MARAMBIO

15.2 TRABAJAR CON ARCHIVOS CSV DESDE PYTHON

Python provee de un módulo propio llamado `csv`, que facilita el parseo de los datos de archivos CSV, tanto para lectura como escritura.

Este módulo, se utiliza en combinación con la estructura `with` y la función `open`, para leer o generar el archivo, y el módulo `CSV` para su análisis (*parsing*).

15.2.1 LECTURA DE ARCHIVOS CSV

Contenido de archivo.csv

```
0;2016-03-01 00:00:00;;9.9;73;;;  
1;2016-03-01 00:30:00;;9.0;67;;;  
2;2016-03-01 01:00:00;;8.3;64;;;  
3;2016-03-01 01:30:00;;8.0;61;;;  
4;2016-03-01 02:00:00;;7.4;62;;;  
5;2016-03-01 02:30:00;;8.3;47;;;  
6;2016-03-01 03:00:00;;7.7;50;;;  
7;2016-03-01 03:30:00;;9.0;39;;;  
8;2016-03-01 04:00:00;;8.7;39;;;
```

```
from csv import reader
```

```
with open("archivo.csv", "r") as archivo:
    documento = reader(archivo, delimiter=';', quotechar='')
    for fila in documento:
        ' '.join(fila)
```

Salida:

```
'0 2016-03-01 00:00:00 9.9 73 '
'1 2016-03-01 00:30:00 9.0 67 '
'2 2016-03-01 01:00:00 8.3 64 '
'3 2016-03-01 01:30:00 8.0 61 '
'4 2016-03-01 02:00:00 7.4 62 '
'5 2016-03-01 02:30:00 8.3 47 '
'6 2016-03-01 03:00:00 7.7 50 '
'7 2016-03-01 03:30:00 9.0 39 '
'8 2016-03-01 04:00:00 8.7 39 '
```

Cuando el archivo CSV tiene una cabecera, es necesario saltar dicho encabezado:

Contenido de archivo.csv

```
ID;DATA;VV;DV;T;HR;PPT;RS;P
0;2016-03-01 00:00:00;;;9.9;73;;;
1;2016-03-01 00:30:00;;;9.0;67;;;
2;2016-03-01 01:00:00;;;8.3;64;;;
3;2016-03-01 01:30:00;;;8.0;61;;;
4;2016-03-01 02:00:00;;;7.4;62;;;
5;2016-03-01 02:30:00;;;8.3;47;;;
6;2016-03-01 03:00:00;;;7.7;50;;;
7;2016-03-01 03:30:00;;;9.0;39;;;
8;2016-03-01 04:00:00;;;8.7;39;;;
```

```
from csv import reader
```

```
with open("archivo.csv", "r") as archivo:
    documento = reader(archivo, delimiter=';', quotechar='')
    cabeceras = next(documento)
    for fila in documento:
        ' '.join(fila)
```

Salida:

```
'0 2016-03-01 00:00:00 9.9 73 '  
'1 2016-03-01 00:30:00 9.0 67 '  
'2 2016-03-01 01:00:00 8.3 64 '  
'3 2016-03-01 01:30:00 8.0 61 '  
'4 2016-03-01 02:00:00 7.4 62 '  
'5 2016-03-01 02:30:00 8.3 47 '  
'6 2016-03-01 03:00:00 7.7 50 '  
'7 2016-03-01 03:30:00 9.0 39 '  
'8 2016-03-01 04:00:00 8.7 39 '
```

Otra forma de leer archivos CSV con cabeceras, es utilizar el objeto *DictReader* en vez de *reader*, y así acceder solo al valor de las columnas deseadas, por su nombre:

```
from csv import DictReader  
  
with open("archivo.csv", "r") as archivo:  
    documento = DictReader(archivo, delimiter=';',  
quotechar='"')  
    for fila in documento:  
        fila['DATA']
```

Salida:

```
'2016-03-01 00:00:00'  
'2016-03-01 00:30:00'  
'2016-03-01 01:00:00'  
'2016-03-01 01:30:00'  
'2016-03-01 02:00:00'  
'2016-03-01 02:30:00'  
'2016-03-01 03:00:00'  
'2016-03-01 03:30:00'  
'2016-03-01 04:00:00'
```

15.2.2 ESCRITURA DE ARCHIVOS CSV

Escritura de un CSV sin cabecera:

```
from csv import writer  
  
with open("datos.csv", "w") as archivo:
```

```
doc = writer(archivo, delimiter=';', quotechar='')
doc.writerows(matriz)
```

En el ejemplo anterior, una matriz podría ser una lista de listas con igual cantidad de elementos. Por ejemplo:

```
matriz = [
    ['Juan', 373, 1970],
    ['Ana', 124, 1983],
    ['Pedro', 901, 1650],
    ['Rosa', 300, 2000],
    ['Juana', 75, 1975],
]
```

Lo anterior, generaría un archivo llamado *datos.csv* con el siguiente contenido:

```
eugenia@bella:~$ cat datos.csv
Juan;373;1970
Ana;124;1983
Pedro;901;1650
Rosa;300;2000
Juana;75;1975
```

Escritura de un CSV con cabecera:

En este caso, la matriz a ser escrita requerirá ser una lista de diccionarios cuyas claves coincidan con las cabeceras indicadas.

```
matriz = [
    dict(jugador='Juan', puntos=373, anio=1970),
    dict(jugador='Ana', puntos=124, anio=1983),
    dict(jugador='Pedro', puntos=901, anio=1650),
    dict(jugador='Rosa', puntos=300, anio=2000),
    dict(jugador='Juana', puntos=75, anio=1975),
]
```

```
from csv import DictWriter
```

```
cabeceras = ['jugador', 'puntos', 'anio']
```

```
with open("datos.csv", "w") as archivo:
    documento = DictWriter(
        archivo, delimiter=';',
        quotechar='"',
        fieldnames=cabeceras
    )
    documento.writeheader()
    documento.writerows(matriz)
```

16. EXPRESIONES REGULARES

Python provee de soporte nativo para búsquedas mediante expresiones regulares, de forma similar a Perl.

Una **expresión regular** es un patrón de caracteres de reconocimiento, que aplicado sobre una cadena de texto, permite encontrar fragmentos que coincidan con dicha expresión.

Para definir los patrones se utilizan **caracteres** de forma simbólica (es decir, que cada carácter posee un significado particular en el patrón). Por ejemplo, el patrón "^ho" significa «cadena que comienza por las letras ho», y "la\$", significa «cadena que finaliza por las letras la». Mientras que el acento circunflejo ^ simboliza los comienzos de cadenas, el signo dólar, simboliza los finales. Los caracteres simbólicos se listan a continuación.

Caracteres de posición			
^	Inicio de cadena	\$	Final de cadena
Cuantificadores			
?	Cero o uno	*	Cero o más
+	Uno o más	{n}	n veces
{n,}	n o más veces	{m,}	Entre 0 y n veces
{n,m}	Entre n y m veces		

Caracteres de posición			
Agrupamiento			
(...)	Grupo exacto	[...]	Caracteres opcionales y rangos
	Operador lógico «or» (A B)	-	Usado para expresar un rango [a-z]
Caracteres de formato			
\	Caracter de escape para expresar literales: \. (literal del carácter punto)	\d	Dígito ^{NOTA}
Caracteres de posición			
.	Cualquier carácter excepto el salto de línea	\n	Salto de línea
\s	Espacio en blanco ^{NOTA}	\w	Palabra ^{NOTA}

NOTA: En mayúsculas significa lo contrario. Por ejemplo, \S simboliza cualquier carácter que no sea un espacio en blanco.

16.1 BÚSQUEDA DE PATRONES

Para realizar búsquedas mediante expresiones regulares en Python, se utiliza el **módulo re**. La función **search** de este módulo, permite realizar una búsqueda mediante la sintaxis:

```
search(expresión, cadena)
```

Una búsqueda mediante la función **search**, en caso de encontrar al menos una coincidencia, retornará un objeto **SRE_Match**. Se accede a cada grupo de coincidencias mediante el método **group(índice)**.

```
from re import search
cadena = "ho la mundo"
ser = search("aism", cadena)
ser.group(0)
'a m'
```

En la administración de sistemas GNU/Linux, el uso del constructor **with** para la apertura de archivos, combinado con métodos de del objeto

string y expresiones regulares, se puede emplear en el análisis de registros (*logs*) del sistema.

Se toma como ejemplo el archivo de autenticación, `/var/log/auth.log`

Las siguientes líneas, representan un intento de autenticación fallida del usuario eugenia como root del sistema:

```
Nov 13 13:34:23 bella su[25375]: pam_unix(su:auth):  
authentication failure; logname= uid=1000 euid=0 tty=/dev/pts/0  
ruser=eugenia rhost= user=root  
Nov 13 13:34:25 bella su[25375]: pam_authenticate:  
Authentication failure  
Nov 13 13:34:25 bella su[25375]: FAILED su for root by eugenia
```

La última línea puede utilizarse como patrón, para por ejemplo, obtener una lista de autenticaciones fallidas similares:

```
from re import search

with open("/var/log/auth.log", "r") as f:
    log = f.read()

regex = "(.)+: FAILED su for root by [a-z]+\n"
ser = search(regex, log)

>>> ser.group(0)
'Nov 13 13:34:25 bella su[25375]: FAILED su for root by eugenia\n'
```

En la expresión anterior:

- (.)+** Indica cualquier carácter una o más veces.
Esto coincidirá con la fecha del registro, comando e ID del proceso: **Nov 13 13:34:25 bella su[25375]**
La cadena que sigue, es un literal.
- [a-z]+** Coincide con el nombre de usuario ya que indica cualquier letra entre la a y la z, repetidas una o más veces: **eugenia**

```
\n      El salto de línea coincidiría con el final del  
      registro
```

El mismo sistema puede emplearse para analizar registros de servicios, sistema, etc., entre ellos, el de Apache, el syslog y otros.

17. CREANDO MENÚS DE OPCIONES

17.1 CREACIÓN DE UN MENÚ DE OPCIONES BÁSICO

En el *scripting*, puede resultar útil, dar al usuario un menú de opciones y hacer que el script, actúe según la opción elegida por el usuario. A continuación, se muestra un truco para resolver esto de forma simple e ingeniosa.

- 1) Primero es necesario que todo el *script* esté organizado en funciones.
- 2) En segundo lugar, es necesario que todas las funciones tengan su documentación correspondiente, definiendo qué es exactamente lo que hace la función:

```
def leer_archivo():  
    """Leer archivo CSV"""  
    return "leer"  
  
def escribir_archivo():  
    """Escribir archivo CSV"""  
    return "escribir"  
  
def _sumar_numeros(lista):  
    """Sumar los números de una lista"""  
    return "privada"
```

- 3) A continuación, se define una lista con el nombre de todas las funciones que serán accesibles por el usuario, desde el menú:

```
funciones = ['leer_archivo', 'escribir_archivo']
```

El truco consistirá en automatizar tanto la generación del menú, como la llamada a la función.

Para **automatizar la generación del menú**, el truco consiste en valerse de:

- La lista del paso 3
- La función `locals()`
- El atributo `__doc__`

```
numero = 1 # se usará luego para acceder a la función
menu = "Elija una opción:\n"

for funcion in funciones:
    menu += "\t{.}. {}\n".format(
        numero, locals()[funcion].__doc__)
    numero += 1 # incrementa el número en cada iteración

print(menu)
opcion = int(input("Su opción: "))
```

Finalmente, para **acceder dinámicamente a la función** elegida por el usuario, el truco consistirá en emplear la opción elegida por el usuario, como índice para acceder al nombre de la función desde la lista, y recurrir nuevamente a *locals* para invocar a la función:

```
funcion = funciones[opcion - 1]
# se obtiene el nombre de la función

locals()[funcion]()
# se invoca a la función mediante locals()
```

17.2 CREACIÓN DE UN MENÚ DE OPCIONES CON ARGPARSE

En el *scripting*, puede resultar útil, dar al usuario un menú de opciones y hacer que el script, actúe según la opción elegida por el usuario. En el apartado anterior se muestra un truco para resolver un menú de forma simple e ingeniosa. Aquí se verá cómo analizar argumentos, pasado al script, por línea de comendos, mediante el módulo **argparse**.

17.2.1 PASO 1: IMPORTACIÓN DEL MÓDULO

Se debe importar la clase **ArgumentParser** del módulo **argparse**:

```
from argparse import ArgumentParser
```

17.2.2 PASO 2: CONSTRUCCIÓN DE UN OBJETO ARGUMENTPARSER

Se construye un objeto **ArgumentParser** a fin de establecer cuáles serán los argumentos que el programa recibirá.

Los parámetros aceptados por el método constructor del objeto **ArgumentParser** (función `__init__`) son todos opcionales. Entre otros, admite los siguientes parámetros:

- **prog**: el nombre del programa (por defecto, toma el nombre del ejecutable)
- **description**: descripción del programa que se mostrará en la ayuda
- **epilog**: texto que se mostrará al final de la ayuda

```
#!/usr/bin/env python
from argparse import ArgumentParser

argp = ArgumentParser(
    description='Descripción breve del programa',
    epilog='Copyright 2018 Autor bajo licencia GPL v3.0'
```

)

17.2.3 PASO 3: AGREGADO DE ARGUMENTOS Y CONFIGURACIÓN

Para agregar un argumento puede emplearse el método `add_argument`.

Existen dos tipos de argumentos que pueden declararse:

- Argumentos posicionales: por defecto, todos aquellos que sean declarados con un nombre en vez de emplear una bandera
- Opciones (banderas / flags): todos aquellos que empleen el prefijo de opción -

De esta forma, un argumento definido como 'foo' será posicional, mientras que si se lo define como '-f' o '--foo', será una opción:

```
argp.add_argument('foo')      # argumento posicional
argp.add_argument('--foo')    # opción foo
argp.add_argument('-f')       # opción f
```

`add_argumento` puede recibir, por lo tanto, solo un nombre de argumento posicional, o una lista de banderas de opción (flags). En el siguiente ejemplo, si se ejecutara el programa sin pasar ningún argumento, el fallo se produciría por la ausencia del argumento posicional «directorio», pero no, por la ausencia de las opción -f o --foo.

```
# Solo un nombre posicional
argp.add_argument('directorio')

# Una lista de banderas de opción
argp.add_argument('-f', '--foo')
```

Configuración de argumentos

El método *add_argument*, además del nombre del argumento posicional u opción, puede recibir de forma no obligatoria, algunos parámetros que establecen la forma en la que el nombre de argumento o bandera de opción, será tratado. Todos estos parámetros opcionales, se definen en la siguiente tabla.

Parámetro	Descripción	Valores posibles*	Valor por defecto
action	Acción a realizar con el parámetro	store : almacenar el valor append : agregarlo a una lista	store
nargs	Cantidad de valores admitidos	? : cero o uno * : cero o más + : uno o más valor entero	1
default	Valor por defecto para el argumento	cualquier valor es admisible	
type	Tipo de datos	cualquier tipo soportado por Python	None
choices	Lista de valores posibles	Una lista	None
required	Indica si el argumento es obligatorio	True o False	False
help	Texto de ayuda a mostrar para el argumento	Una string	None
metavar	El nombre del argumento que se empleará en la ayuda	Una string	el nombre del argumento o flag
dest	Nombre de la variable en la que será almacenado el argumento	Una string	El nombre del argumento o bandera

(*) Para una lista completa, remitirse a la documentación oficial o al artículo «Shell Scripting: Análisis de argumentos por línea de comandos» en la siguiente URL:

<http://fileserv.laeci.org/Art%c3%adculos%20t%c3%a9cnicos/Python/ArgParse.pdf>

```
argp.add_argument(  
    'vocal', # Argumento posicional  
    nargs='+', # Admite uno o más valores  
    choices=['a', 'e', 'o'], # Valores posibles  
    metavar='VOCAL', # Nombre de la variable a mostrar en la ayuda  
    help='Vocal abierta', # Texto a mostrar como ayuda  
)
```

Siguiendo el ejemplo anterior, si el programa se ejecutara sin argumentos:

```
usuario@host:~$ ./ejemplo.py
```

daría el siguiente error:

```
usage: ejemplo [-h] VOCAL [VOCAL ...]  
curl: ejemplo: the following arguments are required: VOCAL
```

Y si se ejecutase con la bandera de opción -h:

```
usuario@host:~$ ./ejemplo.py -h
```

arrojaría la siguiente ayuda:

```
usage: ejemplo [-h] VOCAL [VOCAL ...]
```

Descripción del programa

positional arguments:

VOCAL Vocal abierta

optional arguments:

-h, --help show this help message and exit

17.2.4 PASO 4: GENERACIÓN DEL ANÁLISIS (PARSING) DE ARGUMENTOS

```
argumentos = argp.parse_args()
```

El método **parse_args** es el encargado de generar un objeto cuyas propiedades serán los argumentos recibidos por línea de comandos. A cada argumento se accederá mediante la sintaxis:

```
objeto_generado.nombre_del_argumento
```

Por ejemplo:

```
argumentos.foo
```

Para comprender mejor el funcionamiento de *argparse*, se propone el siguiente ejemplo, el cual, se recomienda replicar a modo de ejercicio y ejecutarlo repetidas veces con modificaciones.

El siguiente ejemplo, se trata de un menú basado en el programa **curl**. Ejecutar **man curl** si se desea poner en contexto el ejemplo.

A modo de ejercicio, se recomienda intentar imprimir los valores obtenidos mediante **argumentos.parametro**.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from argparse import ArgumentParser

descripcion_del_programa = "{}, {}".format(
    "Herramienta para transferir datos desde o hacia un servidor",
    "utilizando uno de los protocolos compatibles"
)

argp = ArgumentParser(
    prog='curl',
    description=descripcion_del_programa,
)
```

```
argp.add_argument(
    '-H', '--header',      # Banderas
    action='append',        # Lista de valores
    nargs='+',             # Admite uno o más valores
    type=str,              # Convertir los valores a string
    metavar='LINE',        # Nombre de opción a mostrar en la ayuda
    help='Cabecera adicional a incluir en la solicitud HTTP a enviar'
)

argp.add_argument(
    '-d', '--data',
    action='append',
    nargs='+',
    type=str,
    help='envía los datos especificados en una solicitud post, al servidor
http'
)

argp.add_argument(
    'url',
    type=str,
    metavar='URL',
    help='URL a la cual realizar la solicitud.'
)

argumentos = argp.parse_args()
```

Observaciones:

Recordar que la especificación de la codificación de caracteres UTF-8, si bien en Python 3 no es necesaria ya que se interpreta por defecto, se utiliza a fin de hacerlo compatible con Python 2, y frente a ciertos tipos de objetos y bibliotecas.

18. CAPTURA Y MANEJO DE EXCEPCIONES

El código fuente de un programa puede fallar cuando se escribe mal la sintaxis, o se comete algún error de escritura. Este tipo de errores son detectados por el intérprete, antes de la ejecución del código.

Sin embargo, existen casos en los que el fallo no se produce por un error de quien ha escrito el código. En estos casos, y bajo determinadas circunstancias, el intérprete lanza un error, detectado en tiempo de

ejecución, ya que el mismo, como se ha dicho, depende de las circunstancias particulares en las que el código se está ejecutando. Esas circunstancias son diversas, pero algunos ejemplos frecuentes pueden ser:

- Una función que falla porque recibe unos parámetros no esperados.
- Una operación de cómputo falla porque el valor obtenido en tiempo de ejecución no es del tipo esperado.
- Una conexión no puede realizarse por factores externos.

Entre muchos otros ejemplos.

A este tipo de errores detectados en tiempo de ejecución, se los denomina **excepciones**, y pueden capturarse y manejarse a conveniencia, desde el propio código fuente.

El manejo de excepciones en Python, se puede llevar a cabo mediante la estructura **try / except**, y la declaración **raise**. Estas declaraciones se explican a continuación.

18.1 USO BÁSICO DE LA ESTRUCTURA TRY/EXCEPT

La estructura **try / except** permite capturar excepciones que de otro modo provocarían la finalización abrupta del script, cuando una excepción sea lanzada.

Cuando una instrucción tiene la posibilidad de fallar (normalmente, cuando depende de valores obtenidos al vuelo), puede colocarse el código dentro de la estructura **try**, y utilizarse **except** para ejecutar una

acción en caso de que el intento de ejecución de código del **try**, falle.
Su sintaxis podría interpretarse como la siguiente:

```
intentar:
    ejecutar esto
si falla:
    hacer esto otro
```

Pasado a lenguaje Python:

```
try:
    # instrucción que puede fallar
except:
    # instrucción a ejecutar en caso de que el try falle
```

También es posible capturar el tipo de excepción lanzada:

```
try:
    # instrucción que puede fallar
except TipoDeExcepción1:
    # instrucción a ejecutar en caso de que se produzca
    # una excepción de tipo TipoDeExcepcion1
except TipoDeExcepción2:
    # instrucción a ejecutar en caso de que se produzca
    # una excepción de tipo TipoDeExcepcion2
```

También es admisible capturar más de un tipo de excepción de forma simultánea:

```
try:
    # instrucción que puede fallar
except (TipoDeExcepción1, TipoDeExcepción2):
    # instrucción a ejecutar en caso de que se produzca
    # una excepción de tipo TipoDeExcepcion1 o
    # TipoDeExcepcion2
```

E incluso, puedo capturarse una descripción del error, aunque no se conozca el tipo de excepción:

```
try:
    # instrucción que puede fallar
```

```
except Exception as problema_encontrado:
    # instrucción a ejecutar en caso de que se produzca
    # una excepción desconocida
    print(problema_encontrado)
```

Los diferentes **tipos de excepciones**, pueden estudiarse en la documentación oficial de Python 2⁴ y de Python 3⁵. No obstante, debe tenerse en cuenta que los tipos de excepciones difieren en ambas ramas del lenguaje. Los tipos de excepciones más habituales se describen al final del capítulo.

Un caso habitual en el que una instrucción puede fallar, se dará cuando el código esté esperando un tipo de datos y reciba otro. Es el caso de la siguiente función:

```
def calcular_iva(importe_bruto, alicuota=21):
    iva = importe_bruto * alicuota / 100.0
    neto = importe_bruto + iva
    return neto
```

La función anterior espera que tanto importe_bruto como alicuota sean dos números, a fin de poder realizar una operación aritmética con ellos.

Cuando se reciben los datos esperados, la función se ejecuta con absoluta normalidad:

```
>>> calcular_iva(1500, 10.5)
1657.5
```

Sin embargo, cuando otro tipo de datos diferente al esperado es recibido, una excepción es lanzada:

```
>>> calcular_iva('1500', 10.5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in calcular_iva
```

4 <https://docs.python.org/2/library/exceptions.html>

5 <https://docs.python.org/3.7/library/exceptions.html>

TypeError: can't multiply sequence by non-int of type 'float'

Aquí, un error de tipo (*TypeError*) es lanzado, pues el primer parámetro ha sido enviado como cadena de texto.

Si bien podría resolverse haciendo una conversión de tipo (*type casting* en inglés):

```
>>> def calcular_iva(importe_bruto, alicuota=21):
...     iva = float(importe_bruto) * float(alicuota) / 100
...     neto = float(importe_bruto) + iva
...     return neto
...
>>> calcular_iva('1500', 10.5)
1657.5
```

otro error sería lanzado cuando el valor recibido no pudiese ser convertido:

```
>>> calcular_iva(None, 10.5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "<stdin>", line 2, in calcular_iva
TypeError: float() argument must be a string or a number, not
'NoneType'
```

Por lo tanto, un método eficaz para sanear la función, consistiría en capturar la excepción (siempre teniendo presente que el objetivo de todo programa al imponer restricciones debe ser optimizar el rendimiento, toda vez que hacerlo, no comprometa la seguridad de los datos):

```
def calcular_iva(importe_bruto, alicuota=21):
    try:
        importe_bruto = float(importe_bruto)
    except:
        importe_bruto = 0

    try:
        alicuota = float(alicuota)
```

```
except:
    alicuota = 21

iva = importe_bruto * alicuota / 100
neto = importe_bruto + iva
return neto
```

En este caso, cuando las variables no pudiesen ser convertidas, se llevarían a unos valores por defecto:

```
>>> calcular_iva(None, 10.5)
0.0
>>> calcular_iva('', 10.5)
0.0
>>> calcular_iva('1500', 10.5)
1657.5
>>> calcular_iva('1500', None)
1815.0
```

18.2 TRY/EXCEPT/FINALLY Y TRY/EXCEPT/ELSE

La estructura **try/except** puede contener una declaración **else** que solo se ejecuta cuando el bloque **try** haya sido exitoso, y una declaración **finally** que se ejecute siempre al finalizar la estructura, independientemente del resultado.

```
try:
    print('Ejecución exitosa')
except:
    print('Me ejecuto si hubo excepciones')
else:
    print('Si no hay excepciones, me ejecuto')
finally:
    print('Me ejecuto siempre, cuando todo lo anterior termina')
```

Lo anterior, siempre arrojará la siguiente salida:

```
Ejecución exitosa
Si no hay excepciones, me ejecuto
Me ejecuto siempre, cuando todo lo anterior termina
```

18.3 LANZAMIENTO DE EXCEPCIONES

ADVERTENCIA: el tema presentado a continuación puede ser considerado muy avanzado para aquellas personas que no cuenten con vasta experiencia en la programación de bibliotecas a bajo nivel, o destinadas a ser usadas por quienes programan. Si solo se cuenta con experiencia en el desarrollo de programas destinados al usuario final, o no se cuenta con experiencia previa en programación, se sugiere postergar la lectura y estudio de este tema para un estadio más avanzado en la carrera como profesional de la programación.

En algunas ocasiones, en las especificaciones de requerimientos del programa, se podría definir la necesidad de arrojar un error cuando una restricción específica no se cumpla. Esto es habitual en la programación de bibliotecas (es decir, en programas para ser usados por quienes programan) y no es una práctica habitual ni aconsejada, en el desarrollo de programas destinados al usuario final, donde el lanzamiento de errores reduce la seguridad del software.

Un ejemplo de la necesidad de lanzar errores, se da cuando se desarrolla una biblioteca, herramienta, o función, que estipula una serie de restricciones que no son respetadas por quien programa. Es el caso de una función que como parámetro espera un determinado tipo de datos y recibe otro.

IMPORTANTE: no debe confundirse con el caso tratado al comienzo del capítulo en los ejemplos de la función de cálculo de IVA. Cuando un programa se destina al usuario final, cuanta mayor autonomía logra el programa para resolver el requerimiento, mayor es el rendimiento del programa. Pero sucede lo opuesto cuando el

programa se destina como biblioteca para programar. En este caso, cuanto menor información brinde el programa, menor será su rendimiento, mientras que en el primer caso, mayor será su seguridad.

En el siguiente ejemplo, la función *componer* permitirá a quienes programan, asignar a una variable un valor de un tipo de datos concreto, emulando el funcionamiento de los lenguajes de tipado fuerte.

```
def componer(valor, tipo):
    """Verifica que un dato sea de un tipo esperado y lo
    retorna.

    Si el valor pasado es del tipo esperado, lo retorna.
    En caso contrario, arroja un error de tipo.

    valor -- la variable a verificar
    tipo -- el tipo de datos esperado.
           Puede ser uno de los tipos incorporados, o un tipo
           propio (como una clase definida por el usuario).
    """
    try:
        if not isinstance(valor, tipo):
            valor_str = str(valor)
            tipo_str = tipo.__name__
            raise TypeError('%s no es una variable de tipo %s' \
                             % (valor_str, tipo_str))
    except TypeError:
        raise
    else:
        return valor
```

En la función anterior, cuando **valor** no es del tipo esperado, se fuerza al programa a lanzar un error que de otro modo no sería lanzado. El error se lanza con la declaración:

```
raise TipoDeError('<descripción del error>')
```

Tras lanzar el error, se lo captura con **except**. Al capturarlo, se le pide al programa arrojar el error por pantalla mediante la declaración:

```
raise
```

Si no se ha lanzado ningún error (ese u otro), se retorna el valor analizado mediante la declaración **else**.

Cuando un error que no sea error de tipo sea producido, el intérprete se encargará de lanzarlo como es habitual, pues no es necesario forzar un error cuando realmente se produzca un error.

A continuación, se muestra el comportamiento de la función bajo diferentes escenarios.

El valor es del tipo esperado:

```
>>> componer(15, int)
15
```

Como puede verse, retorna el valor recibido, pues 15 efectivamente es de tipo entero.

El valor no es del tipo esperado:

```
>>> componer(15, str)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 6, in componer
TypeError: 15 no es una variable de tipo str
```

Cuando el valor no es del tipo esperado, fuerza el lanzamiento del error tal y como se ha programado.

```
>>> componer(15, 'string')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in componer
```

TypeError: isinstance() arg 2 must be a type or tuple of types

Cuando se produce una excepción diferente a la excepción forzada, es el lenguaje quien se encarga de lanzarla. Pues en el ejemplo, se ha cometido el error de pasar una cadena de texto como segundo parámetro, cuando se debería haber pasado un tipo.

18.4 ENCADENAMIENTO DE EXCEPCIONES

ADVERTENCIA: la lectura de este tema requiere haber alcanzado una comprensión en profundidad de los conceptos abarcados en el punto **18.3 Lanzamiento de excepciones**, explicado en página 113.

El encadenamiento de excepciones permite lanzar una excepción a causa del lanzamiento de otra. Por ejemplo, si se produce como en el caso anterior, una excepción de tipo, como consecuencia, se produce (además) una excepción de tiempo de ejecución. Por lo tanto, si se lanza (como en el caso anterior) una excepción de tipo, como consecuencia, se obtiene una segunda excepción de tiempo de ejecución.

Dicho encadenamiento se logra mediante la declaración **from**. Se modifica el ejemplo previo para demostrar su uso y aplicación.

```
def componer(valor, tipo):
    try:
        if not isinstance(valor, tipo):
            valor_str = str(valor)
            tipo_str = tipo.__name__
            raise TypeError('%s no es una variable de tipo %s' \
                            % (valor_str, tipo_str))
    except TypeError as type_error:
        raise RuntimeError('Se ha producido un error en\n'
                           'tiempo de ejecución') from type_error
    else:
        return valor
```

Probando la función con un valor que no es del tipo esperado, se produce el siguiente resultado:

```
>>> componer(15, str)
Traceback (most recent call last):
  File "<stdin>", line 7, in componer
TypeError: 15 no es una variable de tipo str
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 9, in componer
RuntimeError: Se ha producido un error en tiempo de ejecución
```

El procedimiento para lanzar excepciones encadenadas, consistirá entonces, en los siguientes pasos:

1. Forzar el lanzamiento de una excepción.
2. Capturar la excepción forzada.
3. Lanzar una nueva excepción a partir de la captura de la excepción forzada.

18.5 TIPOS DE EXCEPCIONES MÁS COMUNES

El listado completo de excepciones se encuentra documentado en <https://docs.python.org/3/library/exceptions.html#bltin-exceptions>. A continuación, se describen los tipos de excepciones más comunes en la siguiente tabla:

Tipo de Excepción	Descripción
ImportError	Lanzado cuando se intenta importar un elemento que por diversos motivos no puede ser importado (generalmente, por un error en el <i>namespace</i>).
NameError	Lanzado cuando se quiere acceder a un elemento no definido (como una variable, una función, o una clase).

Tipo de Excepción	Descripción
SyntaxError	Lanzado cuando se encuentra un error sintáctico en el código fuente.
IndentationError	Lanzado cuando existen errores en la tabulación, bien sea porque se ha sangrado mal el código, o bien porque se han mezclado tabulaciones con espacios en blanco.
TypeError	Lanzado cuando se intenta tratar un tipo de dato como un tipo diferente.
KeyboardInterrupt	Lanzado cuando se interrumpe la ejecución de un script mediante Ctrl+C.
OSError	Lanzado cuando una función obtiene un error, generalmente de E/S, desde el sistema operativo.
RuntimeError	Lanzado cuando se produce un error en tiempo de ejecución, no capturado por otra excepción.
IndexError	Lanzado cuando se intenta acceder al elemento de una colección, que se encuentra fuera de rango.
KeyError	Lanzado cuando se quiere acceder a un elemento de un diccionario, cuya clave no existe.

19. GENERACIÓN DE REGISTROS DE SISTEMA

Si se necesita que un programa o script del sistema guarde un registro, puede emplearse el módulo **logging**.

El módulo **logging** provee cinco niveles de registros, los cuales se describen en la siguiente tabla:

NIVEL		Utilizado generalmente cuando se desea...
DEBUG	10	monitorizar el funcionamiento de un programa permitiendo depurar un programa durante su ejecución normal, a fin de obtener la información deseada para efectuar un diagnóstico determinado.
INFO	20	registrar eventos afirmativos es decir, mantener un registro detallado, de ciertas acciones

NIVEL		Utilizado generalmente cuando se desea...
		ejecutadas en la aplicación, de forma satisfactoria.
WARNING	30	emitir una alerta sobre un evento determinado permitiendo grabar en el archivo de registros, información que, sin representar un error o momento crítico de fallo, podría ser indicativa de un posible fallo, error, o acción no deseada. Generalmente útil en advertencias de seguridad.
WARNING	30	emitir una alerta sobre un evento determinado permitiendo grabar en el archivo de registros, información que, sin representar un error o momento crítico de fallo, podría ser indicativa de un posible fallo, error, o acción no deseada. Generalmente útil en advertencias de seguridad.
ERROR	40	registrar un error cuando el programa no logra llevar a cabo una acción esperada
CRITICAL	50	registrar un error que frene la ejecución normal del programa. Suele emplearse cuando errores fatales son capturados, y la ejecución normal del programa se ve impedida.

El nivel por defecto es **WARNING**, por lo que si se desea grabar (o mostrar) registros de niveles inferiores como **INFO** o **DEBUG**, deberá modificarse el nivel de registro por defecto.

Los registros pueden mostrarse en pantalla o grabarse en un archivo, tal y como se hará en lo sucesivo.

19.1 PRINCIPALES ELEMENTOS DEL MÓDULO LOGGING

Constantes: representan los distintos niveles de registro. Estas son:

INFO, DEBUG, WARNING, ERROR, CRITICAL

Clase basicConfig: utilizada para inicializar un registro, configurar el nivel de registro por defecto, y opcionalmente, establecer la ruta del archivo de registro y el modo de escritura.

```
from logging import basicConfig, INFO
basicConfig(
    filename='/var/log/programa.log',
    filemode='a',
    level=INFO
)
```

Los parámetros compartidos en ambas ramas del lenguaje, para **basicConfig** con los siguientes:

- **filename:** ruta del archivo
- **filemode:** modo de apertura (comúnmente 'a' [append, valor por defecto] o 'w' [escritura])
- **format:** establece el formato en el que se generarán los registros
- **datefmt:** formato de fecha y hora que se utilizará en los registros
- **level:** nivel de registro (cualquiera de las 5 constantes)
- **stream** (esta opción no será abarcada en el curso)

Algunas de las variables admitidas como parte del valor del parámetro **format**, son las siguientes:

asctime	%(asctime)s
created	%(created)f
filename	%(filename)s
funcName	%(funcName)s
levelname	%(levelname)s
levelno	%(levelno)s
lineno	%(lineno)d
module	%(module)s
msecs	%(msecs)d
message	%(message)s
name	%(name)s
pathname	%(pathname)s
process	%(process)d
processName	%(processName)s
relativeCreated	%(relativeCreated)d
thread	%(thread)d
threadName	%(threadName)s

Para una descripción detallada, ver la sección «LogRecords Attributes» en la documentación oficial de Python⁶.

Ambas ramas conservan las mismas variables.

```
'[% (asctime)s] [% (levelname)s] [pid %(process)d] MYAPP
MyErrorLevel Alert: %(message)s'
```

El ejemplo anterior, producirá un registro similar al siguiente:

```
[2018-04-20 00:34:42,803] [WARNING] [pid 12318] MYAPP
MyErrorLevel Alert: Posible violación de seguridad
```

Para establecer el formato que tendrá la fecha, mediante el parámetro **datefmt** se pueden emplear las directivas descritas en la siguiente tabla:

DIRECTIVA	SIGNIFICADO	DIRECTIVA	SIGNIFICADO
%A	Nombre del día de la semana	%M	Minutos [00,59].
%b	Abreviatura del nombre del mes	%p	AM / PM.
%B	Nombre del mes completo	%S	Segundos [00,59].
%d	Número del día del mes [01,31]	%w	Número del día de la semana [0,6]
%H	Hora en formato de 24 horas [00,23]	%y	Año en formato YY [00,99]
%I	Hora en formato de 12 horas [00,12]	%Y	Año
%m	Número del mes [01,12].	%Z	Zona horaria

Tabla obtenida de la documentación oficial de Python:
<https://docs.python.org/3.6/library/time.html#time.strftime>

6 <https://docs.python.org/3/library/logging.html#logrecord-attributes>

Funciones de registro: utilizadas para mostrar o grabar los diferentes mensajes de registro. Estas son:

```
info(), debug(), warning(), error(), critical()
```

A estas funciones, se le debe pasar como parámetro, el mensaje que se desea almacenar en el registro:

```
funcion("mensaje a grabar")
```

También es posible emplear variables como parte del mensaje, utilizando modificadores formato en la cadena, y pasando las variables como argumentos:

```
funcion("Mensaje %s %i", variable_string, variable_entero)
```

Replicando el siguiente código y ejecutándolo repetidas veces con modificaciones, tanto de configuración, como de niveles de registro y mensajes, permitirá un mejor entendimiento de la biblioteca y sus posibilidades de implementación.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from logging import basicConfig, error, info, INFO
from sys import argv

basicConfig(
    filename='ejemplo_logging.log',
    filemode='a',
    level=INFO,
    format='[%asctime)s] [%levelname)s] [pid %(process)d] %(message)s',
    datefmt="%d/%m/%Y %H:%M"
)

try:
    with open(argv[1], "a") as f:
        f.write(argv[2])

    info("Agregado el texto %s al archivo %s", argv[2], argv[1])
except:
    error("Se produjo un error al escribir en el archivo %s", argv[1])
```

```
try:
    with open("/var/log/foo.log", "a") as f:
        f.write("Mensaje de prueba")
except Exception as problema:
    error(problema)
```

argv es una lista del módulo **system**, que almacena los argumentos pasados al script por línea de comandos, siendo la ruta del archivo o nombre del ejecutable, el primer elemento de la lista.

20. MÓDULOS DEL SISTEMA (OS, SYS Y SUBPROCESS)

Los módulos **os** y **subprocess** permiten manejar **funcionalidades del sistema operativo**, y **procesos del sistema**, respectivamente, mientras que el módulo **sys**, provee acceso a **variables del intérprete** del lenguaje.

A diferencia de **shutil**, un módulo de Python que permite manejar archivos a alto nivel, el módulo **os** provee funciones que operan a bajo nivel. Por este motivo, no se abarca el módulo **shutil**, sino, **os**.

20.1 EL MÓDULO OS

Este módulo permite operar a bajo nivel con funcionalidades del sistema operativo. Algunas de ellas, se listan en la tabla de la siguiente página.

Acción	Comando GNU/Linux	Método
ACCESO A ARCHIVOS Y DIRECTORIOS		
Obtener directorio actual	pwd	getcwd()

Acción	Comando GNU/Linux	Método
Cambiar el directorio de trabajo	<code>cd ruta</code>	<code>chdir(path)</code>
Mover el directorio de trabajo a la raíz	<code>cd</code>	<code>chroot()</code>
Modificar permisos	<code>chmod</code>	<code>chmod(path, permisos)</code>
Cambiar el propietario de un archivo o directorio	<code>chown</code>	<code>chown(path, permisos)</code>
Crear un directorio	<code>mkdir</code>	<code>mkdir(path[, modo])</code>
Crear directorios recursivamente	<code>mkdir -p</code>	<code>makedirs(path[, modo])</code>
ACCESO A ARCHIVOS Y DIRECTORIOS		
Eliminar un archivo	<code>rm</code>	<code>remove(path)</code>
Eliminar un directorio	<code>rmdir</code>	<code>rmdir(path)</code>
Renombrar un archivo	<code>mv</code>	<code>rename(actual, nuevo)</code>
Crear un enlace simbólico	<code>ln -s</code>	<code>symlink(origen, destino)</code>
Establecer máscara de creación de ficheros	<code>umask</code>	<code>umask(máscara)</code>
Obtener listado de archivos y directorios	<code>ls -a</code>	<code>listdir(path)</code>
Obtener el estado de un fichero	<code>stat</code>	<code>stat(path)</code>
EVALUACIÓN DE ARCHIVOS Y DIRECTORIOS (Módulo <code>os.path</code>)		
Obtener ruta absoluta	-	<code>path.abspath(path)</code>
Obtener directorio base	-	<code>path.basename(path)</code>
Saber si un directorio existe	-	<code>path.exists(path)</code>
Conocer último acceso a un directorio	-	<code>path.getatime(path)</code>
Conocer tamaño del directorio	-	<code>path.getsize(path)</code>
Saber si una ruta es:	-	
<i>absoluta</i>	-	<code>path.isabs(path)</code>
<i>un archivo</i>	-	<code>path.isfile(path)</code>
<i>un directorio</i>	-	<code>path.isdir(path)</code>

Acción	Comando GNU/Linux	Método
<i>un enlace simbólico</i>	-	<code>path.islink(path)</code>
<i>un punto de montaje</i>	-	<code>path.ismount(path)</code>
FUNCIONALIDADES DEL SISTEMA OPERATIVO		
Obtener el valor de una variable de entorno	<code>\$VARIABLE</code>	<code>getenv(variable)</code>
FUNCIONALIDADES DEL SISTEMA OPERATIVO		
Obtener los datos del sistema operativo	<code>uname -a</code>	<code>uname()</code>
Obtener UID	<code>id -u</code>	<code>getuid()</code>
Obtener ID del proceso	<code>pgrep</code>	<code>getpid()</code>
Crear variable de entorno (del sistema)	<code>export \$VARIABLE</code>	<code>putenv(variable, valor)</code>
Forzar la escritura del caché al disco	<code>sync</code>	<code>sync()</code>
Matar un proceso	<code>kill</code>	<code>kill(pid, señal)</code>

Para una definición completa y detallada, referirse a la documentación oficial del lenguaje: [https://docs.python.org/\[2|3\]/library/os.html](https://docs.python.org/[2|3]/library/os.html)

20.1.1 VARIABLES DE ENTORNO: OS.ENVIRON

environ es un diccionario del módulo **os** que provee variables de entorno. Esto significa que las variables disponibles en **environ** (como claves del diccionario), varían de acuerdo al entorno en el que se esté ejecutando el programa o script. Por ejemplo, no serán las mismas variables si se ejecuta un script en la *shell*, que si se llama a **environ** desde una aplicación Web. Mientras que en el primer caso, las variables disponibles serán las de la *shell* de GNU Bash, en el segundo (y suponiendo que el servidor sea Apache), las de Apache.

Para ver las variables disponibles en la *shell*, ejecutar en ambas versiones del lenguaje:

```
from os import environ

for variable, valor in environ.items():
    variable, valor
```

20.2 EJECUCIÓN SIMPLIFICADA DE COMANDOS DEL SISTEMA

Si se requiere ejecutar comandos del sistema sin necesidad de hacer un manejo de la entrada y salida (E/S) estándar, o de los errores, la opción simplificada es la función **system** del módulo **os**, la cual efectúa una llamada directa a la función *system* de C. Esto significa que el comando (cadena de texto a ser ejecutada) que sea pasado a la función *system*, será lanzado directamente sobre el sistema operativo.

```
from os import system

command = "curl http://una-api.tld/delete -d 'id=15'"
system(command)
```

Esta función resulta útil y óptima —en cuanto a consumo de recursos— la mayoría de las veces. Sin embargo, el uso de **Popen** y **shlex.split** (como se verá adelante) ofrecen opciones de manejo para la E/S estándar y los errores, que en algunos casos podría resultar más viable que **system**.

Cuando con **system** se requiera obtener la salida estándar o los errores de los comandos ejecutados, será necesario implementar la redirección de salida y/o error del propio sistema:

```
command = "curl http://una-api.tld/delete -d 'id=15' > salida"
system(command)
```

En este caso, la única forma de acceder a la salida almacenada será leerla como a cualquier archivo:

```
with open('salida', 'r') as f:
    salida = f.read()
```

20.3 EJECUCIÓN DE COMANDOS DEL SISTEMA MEDIANTE POPEN Y SHLEX.SPLIT

A través de la clase **Popen** del módulo **subprocess**, es posible ejecutar comandos directamente sobre el sistema operativo, y manipular tanto la E/S estándar como los errores. La función **split** del módulo **shlex**, puede emplearse como complemento de **Popen**, para el *parsing* de cadenas de texto como lista de comandos y argumentos.

La clase **Popen** (*Process open – proceso abierto*), abre un nuevo proceso en el sistema, permitiendo emplear tuberías para manejar la E/S estándar y los errores:

```
from subprocess import Popen, PIPE
proceso = Popen(<comando/argumentos>, stdout=PIPE, stderr=PIPE)
```

La captura y manejo de la E/S estándar y los errores, se trata más adelante.

El primer argumento pasado a Popen, debe ser una lista. Dicha lista, deberá contener cada uno de los comandos, listas de opciones (banderas) y cada uno de los argumentos, como un elemento. Esto significa que para ejecutar el comando:

```
ls -la /home/usuario/Documentos
```

los elementos serán 3 ya que existe:

- 1 argumento (ls)
- 1 lista de opciones (-la)
- 1 argumento

Total: 3 elementos

```
lista = ['ls', '-la', '/home/usuario/Documentos']  
proceso = Popen(lista)
```

Sin embargo, la instrucción completa podría escribirse en una cadena de texto, y emplear la función `split` del módulo `shlex`, para generar la lista necesaria para `Popen`:

```
from shlex import split  
from subprocess import Popen  
  
comando = 'ls -la /home/usuario/Documentos'  
proceso = Popen(split(comando))
```

20.3.1 CAPTURAR LA SALIDA ESTÁNDAR Y LOS ERRORES

Para capturar la salida estándar y los errores, puede emplearse una tubería:

```
from shlex import split  
from subprocess import Popen, PIPE  
  
comando = 'ls -la /home/usuario/Documentos'  
proceso = Popen(split(comando), stdout=PIPE, stderr=PIPE)  
salida = proceso.stdout.read()  
errores = proceso.stderr.read()  
  
if not errores:  
    acción a realizar si no hubo errores  
else:  
    acción a realizar si hubo errores
```

El siguiente es un ejemplo del código anterior, ejecutado en la *shell* de Python y con errores:

```
>>> from shlex import split
>>> from subprocess import Popen, PIPE
>>>
>>> comando = 'ls -la /home/usuario/Documentos'
>>> proceso = Popen(split(comando), stdout=PIPE, stderr=PIPE)
>>> salida = proceso.stdout.read()
>>> errores = proceso.stderr.read()
>>> errores
"ls: no se puede acceder a '/home/usuario/Documentos': No existe
el fichero o el directorio\n"
>>> salida
''
```

Y a continuación, el mismo ejemplo pero con una ruta accesible:

```
>>> comando = 'ls -la /home/eugenia/borrador/python'
>>> proceso = Popen(split(comando), stdout=PIPE, stderr=PIPE)
>>> salida = proceso.stdout.read()
>>> errores = proceso.stderr.read()
>>> errores
''
>>> salida
'total 20\ndrwxr-xr-x  3 eugenia eugenia  4096 nov 16 21:17 .\
ndrwxr-xr-x 43 eugenia eugenia 12288 nov 16 20:03 ..\ndrwxr-xr-x
5 eugenia eugenia  4096 nov 16 20:06 a\n'
```

20.3.2 EMPLEAR LA SALIDA DE UN COMANDO COMO ENTRADA DE OTRO

En la línea de comandos, se emplea la salida de un comando como entrada de otro, al utilizar el símbolo `|` (pipe). El siguiente es ejemplo de ello:

```
ls -la /home/eugenia/borrador/python | grep '20:06'
```

En el ejemplo anterior, se utiliza la salida del comando *ls* como entrada del comando *grep*.

Al emplear Popen, la salida de un comando se encuentra disponible en:

`proceso_creado.stdout`

Esta salida, puede utilizarse como valor del argumento stdin del segundo proceso creado con Popen:

```
from shlex import split
from subprocess import Popen, PIPE

# Comandos necesarios
ls_command = "ls -la /home/eugenia/borrador/python"
grep_command = "grep '20:06'"

# Procesos
ls_process = Popen(
    split(ls_command), stdout=PIPE, stderr=PIPE)

grep_process = Popen(
    split(grep_command),
    stdin=ls_process.stdout,
    # Salida del proceso anterior como entrada
    stdout=PIPE,
    stderr=PIPE
)
```

A continuación, la ejecución del *script* anterior, y la salida del *stdout*, en la *shell* de Python:

```
>>> from shlex import split
>>> from subprocess import Popen, PIPE
>>> ls_command = "ls -la /home/eugenia/borrador/python"
>>> grep_command = "grep '20:06'"
>>> ls_process = Popen(split(ls_command), stdout=PIPE,
stderr=PIPE)
>>> grep_process = Popen(
...     split(grep_command),
...     stdin=ls_process.stdout,
...     stdout=PIPE,
...     stderr=PIPE
... )
>>>
>>> grep_process.stdout.read()
'drwxr-xr-x 5 eugenia eugenia 4096 nov 16 20:06 a\n'
```

20.4 VARIABLES Y FUNCIONES DEL MÓDULO SYS

Algunas de las variables disponibles en este módulo, se describen en la siguiente tabla.

Variable	Descripción
sys.argv	Retorna una lista con todos los argumentos pasados por línea de comandos, incluyendo como primero elemento, el del archivo ejecutado. Al ejecutar: <code>python modulo.py arg1 arg2</code> <code>sys.argv</code> retornará una lista: <code>['modulo.py', 'arg1', 'arg2']</code>
sys.executable	Retorna el path absoluto del binario ejecutable del intérprete de Python
sys.path	Retorna una lista con las rutas empleadas por el intérprete para buscar los archivos
sys.platform	Retorna la plataforma sobre la cuál se está ejecutando el intérprete
sys.version	Retorna el número de versión de Python con información adicional

Entre las funciones del módulo, `exit()` se emplea para finalizar un programa o *script* de forma abrupta:

```
from shlex import split
from subprocess import Popen, PIPE

ls = "ls -la /home/no-existe"
grep = "grep '20:06'"

ps_ls = Popen(split(ls), stdout=PIPE, stderr=PIPE)

if ps_ls.stderr.read():
    exit("Terminación abrupta tras error en comando ls")

ps_grep = Popen(
    split(grep),
    stdin=ps_ls.stdout,
```

```
        stdout=PIPE,  
        stderr=PIPE  
    )
```

El resultado de la ejecución anterior en una shell:

```
>>> from shlex import split  
>>> from subprocess import Popen, PIPE  
>>>  
>>> ls = "ls -la /home/no-existe"  
>>> grep = "grep '20:06'"  
>>>  
>>> ps_ls = Popen(split(ls), stdout=PIPE, stderr=PIPE)  
>>>  
>>> if ps_ls.stderr.read():  
...     exit("Terminación abrupta tras error en ls")  
...  
Terminación abrupta tras error en ls  
eugenia@bella:~/borrador/python$
```

Se debe notar que:

1. El mensaje pasado a la función `exit()` es opcional.
2. La función `exit()` puede recibir un entero representativo del motivo de salida (cero es el valor por defecto, e indica una salida normal)
3. La función `exit()` del módulo `sys` tiene un propósito similar a la constante incorporada `exit`, sin embargo, ambos elementos responden de manera no exactamente idénticas:

```
eugenia@bella:~/borrador/python$ python  
...  
Type "help", "copyright", "credits" or  
"license" for more information.  
>>> print exit # constante incorporada  
Use exit() or Ctrl-D (i.e. EOF) to exit  
>>> import sys  
>>> print sys.exit  
<built-in function exit>  
>>> print sys.exit()  
eugenia@bella:~/borrador/python$
```

21. CONEXIONES REMOTAS (HTTP, FTP Y SSH)

Python provee dos bibliotecas, `http` y `ftplib`, para efectuar conexiones mediante los protocolos HTTP/HTTPS y FTP, respectivamente. Sin embargo, para realizar conexiones mediante el protocolo SSH, se empleará la biblioteca **Paramiko**⁷, creada por Robey Pointer⁸.

21.1 CONEXIONES REMOTAS VÍA HTTP Y HTTPS

Pueden efectuarse con el módulo `client` de la biblioteca `http` de Python.

Para crear la **conexión** se utilizan las clases `HTTPConnection` y `HTTPSConnection`:

```
from http.client import HTTPConnection
http = HTTPConnection('host.com', port=80, timeout=10)
```

El número de puerto y el tiempo de espera, son dos parámetros opcionales, y son admitidos, junto al parámetro posicional **host**, por ambas clases.

Las **solicitudes** se realizan mediante el método `request` que requiere de dos parámetros posicionales:

1. El método HTTP
2. El recurso HTTP

```
http.request("GET", "/foo/bar")
```

⁷ <http://www.paramiko.org>

⁸ <https://robey.lag.net/>

Adicionalmente, admite otros parámetros como **headers** (un diccionario con campos de cabecera) y **body** (una cadena de texto), útiles sobre todo, para peticiones que requieren el envío de información, como por ejemplo, **envío de datos por POST**:

```
parametros = "nombre=Juan&apellido=Perez"
cabeceras = {
    "Content-Type": "application/x-www-form-urlencoded"
}
http.request(
    "POST", "/foo/bar",
    headers=cabeceras, body=parametros
)
```

La **respuesta** recibida, se obtiene mediante el método **getresponse**, que retorna un objeto **HTTPResponse**, el cual, entre sus propiedades, posee **status** (el código de respuesta HTTP) y **reason** (la descripción de la respuesta), y entre sus métodos, **read**, que retorna el cuerpo de la respuesta:

```
respuesta = http.getresponse()
codigo = respuesta.status
descripcion = respuesta.reason
body = respuesta.read()
```

El **cierre** de una conexión HTTP, se efectúa mediante el método **close**:

```
http.close()
```

El siguiente ejemplo, realiza una petición POST a un *host* local que como respuesta, imprime el mensaje «Gracias <nombres>!»:

```
>>> from http.client import HTTPConnection
>>> http = HTTPConnection('juanproyecto.local', port=80,
    timeout=30)
>>> parametros = "nombre=Juan&apellido=Perez"
>>> cabeceras = {"Content-Type": "application/x-www-form-
    urlencoded"}
```

```
>>> http.request("POST", "/foo/bar", headers=cabeceras,  
body=parametros)  
>>> respuesta = http.getresponse()  
>>> codigo = respuesta.status  
>>> descripcion = respuesta.reason  
>>> body = respuesta.read()  
>>> body  
'Gracias Juan'  
>>> codigo  
200  
>>> descripcion  
'OK'
```

21.2 CONEXIONES REMOTAS VÍA FTP

La biblioteca **ftplib** permite conexiones a través del protocolo FTP.

Para **crear una instancia FTP**, dispone de las clases **FTP** y **FTP_TLS**, la segunda, con soporte del protocolo **TLS** (evolución de SSL). Si bien estas clases, como parámetros opcionales, el host, el usuario y la clave (entre otros), a fin de obtener un mejor control sobre las operaciones, estos datos serán enviados mediante los métodos **connect** y **login**, que serán abarcados adelante.

```
from ftplib import FTP  
ftp = FTP()
```

Para **abrir la conexión** se emplea el método **connect**, que como parámetros admite, entre otros, el host y el puerto:

```
ftp.connect('algunhost.com', 21)
```

De ser necesario establecer el **modo pasivo**, se dispone del método **set_pasv**:

```
ftp.set_pasv(True)
```

La **autenticación** se realiza mediante el método **login**, quien recibe por parámetros, usuario y contraseña, respectivamente:

```
ftp.login('algunusuario', 'clave')
```

Para **cerrar** una conexión puede utilizarse el método **quit**. Esto cierra la conexión de ambos lados siempre que el servidor lo soporte y no retorne un error. En caso de que así sea, se llamará al método **close**, el cual cierra la conexión unilateralmente.

```
ftp.quit()
```

Otros **métodos** disponibles se citan a continuación:

Acción	Método
Directorios	
Listar directorios	<code>dir()</code> <code>dir('ruta/a/listar')</code>
Crear un directorio	<code>mkd('ruta/a/nuevo-dir')</code>
Moverse a un directorio	<code>cwd('ruta/a/algun-dir')</code>
Eliminar un directorio	<code>rmd('ruta/a/dir-a-borrar')</code>
Obtener directorio actual	<code>pwd()</code>
Archivos	
Recuperar un archivo remoto	<code>retrbinary('RETR origen', open('/ruta/destino', 'w').write)</code>
Enviar un archivo local	<code>storbinary('STOR destino/remoto.txt', open('/origen/local.txt', 'r'))</code>
Eliminar un archivo	<code>delete('archivo/a/eliminar')</code>
Renombrar (mover) un archivo	<code>rename('origen', 'destino')</code>

Cuando los modos «w» y «r» aparecen en negritas, significa que para archivos binarios debe agregarse «b» al modo.

21.2.1 SOLICITANDO LA CONTRASEÑA CON GETPASS

La biblioteca **getpass** permite solicitar mediante un input, una contraseña al estilo GNU/Linux, para evitar tener que trabajar con la contraseña en crudo en el código fuente:

```
from getpass import getpass
clave = getpass('Ingresar clave: ')
```

La función **getpass** puede utilizarse en forma conjunta con el método login, y así se evita escribir la clave en crudo dentro del código fuente:

```
from ftplib import FTP
from getpass import getpass

ftp = FTP()
ftp.connect('algunhost.com', 21)
ftp.login('algunusuario', getpass('Clave FTP: '))
```

21.3 CONEXIONES SSH CON PARAMIKO

La biblioteca **paramiko** debe instalarse de forma adicional, ya que no forma parte de las bibliotecas de Python, ni está mantenida por Python. Se trata de una biblioteca de terceros, que puede instalarse a través de PyPI (el gestor de paquetes de Python). En Debian 9, también es posible instalarla desde apt (tanto para Python 2 como 3). Sin embargo, se instalará mediante PyPI, ya que es la opción recomendada por el fabricante⁹.

21.3.1 REQUISITOS PREVIOS

Para poder instalar un paquete desde **PyPI**, se necesita la herramienta **pip** de Python. En Debian 9, el gestor de paquetes de Python se instala mediante *apt*:

9 <http://www.paramiko.org/installing.html>

```
apt install python-pip # para Python 2, y
apt install python3-pip # para Python 3
```

Una vez instalado el gestor de paquetes de Python, las instalaciones para Python 2 y para Python 3, se manejarán de forma independiente, por lo cual, habrá que instalar **Paramiko** en ambas versiones. Esto se hará como **root**:

```
pip install paramiko # para Python 2
pip3 install paramiko # Para Python 3
```

21.3.2 USO DE PARAMIKO

Una conexión SSH se inicializa con la **creación de un objeto SSHClient**:

```
from paramiko import SSHClient, AutoAddPolicy

ssh = SSHClient()
```

Al igual que con la biblioteca FTP, tanto la conexión como la autenticación, se realizarán de forma separada (y no al construir el objeto), a fin de tener un mayor control sobre las mismas.

Para la **autenticación mediante llave** pública (en vez de uso de contraseñas), se empleará **set_missing_host_key_policy**, a fin de localizar las llaves y facilitar el intercambio de las mismas:

```
ssh.set_missing_host_key_policy(AutoAddPolicy())
```

Normalmente, el uso de este método no debería ser necesario, y bastaría con emplear **load_system_host_keys**:

```
ssh.load_system_host_keys()
```

Sin embargo, utilizar `set_missing...` resuelve el problema anticipadamente, evitando algoritmos complejos para captura y tratamiento de errores.

La **conexión** al servidor se hará mediante el método **connect**, quien recibe como parámetros, entre otros, el host o IP del servidor, el puerto de conexión y el nombre de usuario:

```
ssh.connect('123.45.67.89', 22, 'usuario')
```

Cuando se requiera **autenticación por contraseña**, como cuarto parámetro del método **connect**, puede pasarse la clave:

```
ssh.connect('123.45.67.89', 22, 'usuario', 'clave')
```

La **ejecución de comandos** en el servidor, se realiza mediante el método **exec_command** a quien se le debe pasar una cadena con la instrucción que se desea ejecutar. Este método retorna tres objetos, de E/S estándar y errores:

```
entrada, salida, error = ssh.exec_command('ls -la')
```

Los objetos de salida y error, pueden ser leídos mediante el método **read**:

```
salida.read()  
error.read()
```

Mientras que la entrada, puede ser escrita mediante **write**:

```
entrada.write('entrada que espera el comando\n')  
salida.read()
```

Finalmente, para **cerrar** la conexión, se utiliza el método **close**:

```
ssh.close()
```

22. BIBLIOTECAS PARA MANEJO AVANZADO DE ARCHIVOS EN GNU/LINUX

22.1 COMPRESIÓN Y DESCOMPRESIÓN DE ARCHIVOS CON LTARFILE Y ZIPFILE

La biblioteca `tarfile` puede utilizarse para leer, comprimir y descomprimir archivos `.tar`, `.tar.gz`, `tar.bz2` y `tar.xz`, mientras que la biblioteca `zipfile`, se utiliza para los archivos `.zip`

22.1.1 LA BIBLIOTECA TARFILE

Bien sea para leer un archivo comprimido, o bien para comprimir o descomprimir, un objeto `TarFile`, se crea mediante la función `open` del módulo.

A diferencia de una apertura estándar, los modos lectura y escritura, se acompañan del formato deseado, mediante la sintaxis **<modo>:<formato>**, donde **<modo>** puede ser **r** (lectura) o **w** (escritura), y **<formato>**, **gz** (*gzip*), **bz2** (*bzip2*) o, solo en Python 3, **xz** (*lzma*).

Modo de apertura	Comando tar
[r w]:gz	tar -[c x]z
[r w]:bz2	tar -[c x]j
[r w]:xz	tar -[c x]J

Modos de apertura y equivalencias con el comando tar. El formato lzma (xz) solo está disponible a partir de la rama 3 del lenguaje.

Descomprimir archivos

```
from tarfile import open as tar_open
```

```
with tar_open("origen.tar.bz2", "r:bz2") as tar:
    tar.extractall('carpeta/destino')
```

Comprimir archivos

```
from tarfile import open as tar_open

with tar_open("carpeta/destino.tar.gz", "w:gz") as tar:
    tar.add('foo.txt')
    tar.add('bar.txt')
    tar.add('baz.txt')
```

Observaciones generales sobre el código

Se utiliza un alias, para que el método **open** de la biblioteca **tarfile**, no sobrescriba la función incorporada **open**. Se emplea la estructura **with**, para no utilizar el método **close**.

Observaciones de seguridad

No deben descomprimirse archivos sin verificar el nombre de los mismos. Un nombre de archivo, podría contener una **/** o **..**, que provocarían que los archivos se almacenasen en un directorio no esperado.

Observaciones de compatibilidad entre versiones

El formato *lzma* solo está disponible en la rama 3 del lenguaje. Para que un *script* o herramienta sea compatible con ambas versiones, la única opción es utilizar los formatos *gzip* o *bzip2*.

22.1.2 LA BIBLIOTECA ZIPFILE

La extracción y compresión de archivos *zip* se realiza de la siguiente forma:

```
from zipfile import ZipFile
```

```
# Escritura de archivos zip
with ZipFile('carpeta/destino.zip', 'w') as z:
    z.write('foo.txt')
    z.write('bar.txt')
    z.write('baz.txt')

# Lectura de archivos zip
with ZipFile('carpeta/origen.zip', 'r') as z:
    z.extractall('carpeta/destino', pwd='clave')
    # el parámetro pwd (contraseña) es opcional
```

Para los archivos *zip*, aplican las mismas observaciones de seguridad que para los archivos *tar*.

22.2 MANEJO DE ARCHIVOS TEMPORALES CON LA BIBLIOTECA TEMPFILE

Cuando sea necesario que un *script*, guarde temporalmente archivos, no es una buena práctica que el mismo *script* los guarde y luego los elimine, ni que intente escribir directamente en el directorio `/tmp`. Para este caso, se debe emplear la biblioteca **tempfile**.

22.2.1 LECTOESCRITURA DE ARCHIVOS TEMPORALES

Cuando se crean objetos de archivos temporales mediante la clase **TemporaryFile** del módulo **tempfile**, los mismos se crean y destruyen en tiempo de ejecución. La destrucción se lleva a cabo al cerrar el archivo. Esto implica que si se trabaja con la estructura **with**, al finalizar dicha estructura, el archivo se habrá eliminado.

```
from tempfile import TemporaryFile

with TemporaryFile() as tmp:
    # aquí el archivo existe

# Aquí el archivo ya no existe
```

Todo archivo temporal escrito, para ser escrito, requiere que el contenido se pase como un **objeto tipo bytes** (y no una cadena). Este

requerimiento es exigencia de Python 3, sin embargo, en Python 2 está perfectamente soportado. Para que una cadena sea convertida a bytes, basta con especificar su tipo:

```
from tempfile import TemporaryFile

with TemporaryFile() as tmp:
    tmp.write(b"Cadena de texto que será pasada a bytes")
```

Finalmente, se debe tener en cuenta que una vez escrito, el cursor estará al final del archivo, por lo que si se lo quiere leer, retornará una cadena nula. Por lo tanto, habrá que mover el cursor al byte 0 a fin de poder leerlo:

```
from tempfile import TemporaryFile

with TemporaryFile() as tmp:
    tmp.write(b"Cadena de texto que será pasada a bytes")
    # ... acciones intermedias
    tmp.seek(0) # Se mueve el cursor al byte 0
    contenido = tmp.read()
```

Observaciones importantes: es necesario aclarar que los archivos temporales creados con **TemporaryFile**, no son archivos persistentes en memoria, sino en disco. De hecho, se almacenan en el directorio temporal del sistema, independientemente de la plataforma. Es posible conocer este directorio invocando a la función **gettempdir()**:

```
from tempfile import TemporaryFile, gettempdir

with TemporaryFile() as tmp:
    tmp.write(b"Cadena de texto")
    tmp_dir = gettempdir()
```

22.3 BÚSQUEDA DE ARCHIVOS CON LAS BIBLIOTECAS GLOB Y FNMATCH

Estas bibliotecas permiten buscar archivos que coincidan con un patrón, con el mismo estilo empleado en sistemas **nix*. Mientras que el módulo **glob** busca archivos que coincidan con un patrón, **fnmatch** verifica si un patrón coincide con el nombre de un archivo.

Símbolos interpretados

Símbolo	Significado
*	Cualquier coincidencia
?	Coincidencia con un único carácter
[secuencia]	Coincidencia con cualquier carácter de la secuencia
[!secuencia]	Coincidencia con cualquier carácter, excepto los de la secuencia

Uso de glob

```
>>> from glob import glob
>>> glob('*.txt')
['foo.txt', 'baz.txt', 'bar.txt']
>>> glob('*[!of].txt')
['baz.txt', 'bar.txt']
```

Uso de fnmatch con os.listdir

```
>>> from os import listdir
>>> from fnmatch import fnmatch
>>> for archivo in listdir('.'):
...     archivo, fnmatch(archivo, '*[!0-9].txt')
...
('3.r', False)
('foo.gif', False)
('.bar', False)
('carpeta', False)
('foo.txt', True)
('baz.txt', True)
('origen.tar.xz', False)
```

```
('foo', False)
('2.r', False)
('bar.txt', True)
('baz', False)
('origen.tar.bz2', False)
('a.r', False)
('1.r', False)
('origen.tar.gz', False)
```

23. PARALELISMO Y CONCURRENCIA

Paralelismo y concurrencia son dos conceptos abstractos que hacen referencia —en la práctica aplicada—, a la multiprogramación (o programación en paralelo) y a la administración de la ejecución de múltiples procesos.

Paralelismo: conjunto de técnicas empleadas para procesar información de forma simultánea, con el objetivo de reducir la cantidad de tiempo que consume la ejecución de un programa.

Concurrencia: forma de gestionar y coordinar la ejecución de múltiples procesos.

Proceso: programa en ejecución.

23.1 ENTENDER EL PARALELISMO

Para entender el paralelismo es necesario comprender cómo trabaja un procesador. Cuando los programas se ejecutan se crean procesos. Si hay más de un programa en ejecución, hay por lo menos, un proceso por cada uno de ellos. Cada procesador, en un mismo instante de tiempo,

puede ejecutar uno —y solo un— proceso, por lo que cada uno de ellos se van ejecutando secuencial y alternadamente, y es el procesador quien decide qué procesos y en qué momento, darles prioridad, siendo entonces, quien administra los recursos.

Cuando un programa se ejecuta y un proceso es creado, este puede permanecer en uno de tres estados posibles:

- En *ejecución*, si es que el proceso está siendo ejecutado en el preciso instante de tiempo en el que se consulta su estado.
- *Listo*, si ha sido creado pero se encuentra a la espera de ser ejecutado.
- *Bloqueado*, si ha sido creado, ejecutado, y se encuentra a la espera de una entrada de datos.

Multiprogramación: cuando varios procesos se ejecutan simultáneamente en una misma CPU, y en realidad la CPU está alternando rápidamente entre un proceso y otro, a no ser que realmente, el ordenador cuente con más de una CPU.

En el modelo de procesamiento más simple, un programa ejecutará una única tarea en simultáneo, por lo que solo requerirá de un único proceso. Sin embargo, es habitual que en un programa se requieran múltiples tareas siendo ejecutadas simultáneamente. Crear nuevos procesos para dichas tareas no es viable, pues todos los procesos compartirían el mismo espacio de memoria, ya que cada programa posee el suyo. Para resolverlo, se generan unos procesos más pequeños dentro del proceso principal, llamados *hilos*.

Al crear hilos en paralelo, un mismo programa puede ejecutar varias tareas de forma casi simultánea, por lo que se reduce la cantidad de tiempo que el programa insume en completar las mismas. De esta forma, en sistemas con múltiples CPU donde la ejecución en paralelo es posible, la velocidad de ejecución de los programas se incrementa considerablemente.

La capacidad de soportar múltiples hilos en un mismo proceso se conoce como *multithreading*.

23.2 PROBLEMAS CON EL PARALELISMO

En la programación paralela, cuando dos procesos trabajan de forma simultánea compartiendo un mismo conjunto de datos, si intentan escribir sobre estos, el resultado dependerá de cuál de los dos procesos sea más rápido y logre escribir primero. Esta situación se conoce como *condición de carrera*.

Para evitar las condiciones de carrera, se deben implementar mecanismos que eviten que dos o más procesos, lean y/o escriban simultáneamente sobre el mismo conjunto de datos. Esto se conoce como *exclusión mutua*.

23.3 PROGRAMACIÓN PARALELA EN PYTHON

La biblioteca nativa de Python, a través de sus módulos *threading* y *multiprocessing*, provee soporte para multiprogramación basada en hilos y en procesos respectivamente.

23.3.1 PARALELISMO BASADO EN HILOS

El script (o programa) que se haya creado, será el proceso en ejecución que se dividirá en hilos. Se crearán tantos hilos como se desee, y cada hilo deberá ser inicializado luego de su creación.

Si se hace un repaso de los **estados de un proceso**, pueden trasladarse a los hilos: creado (listo), en ejecución y bloqueado. Para aplicar cada cambio de estado en Python, se recurre al módulo **threading**, mediante el cual, se crean nuevos hilos con la clase **Thread**, se ejecutan con la llamada al método **start**, y se bloquean con el método **join**.

A continuación, se crean cinco hilos que ejecutarán una función (la cual debió haber sido definida previamente) llamada *MiFunción*:

```
from threading import Thread

for i in range(0, 5):
    hilo = Thread(target=MiFuncion)
    hilo.start()
    hilo.join()
```

Dado que no tendría sentido bloquear un proceso (hilo en este caso) que ya haya finalizado, previamente al bloqueo se puede comprobar que el hilo siga en ejecución, mediante el método **is_alive**:

```
for i in range(0, 5):
    hilo = Thread(target=MiFuncion)
    hilo.start()
    if hilo.is_alive():
        hilo.join()
```

La ejecución del código anterior, no debe interpretarse secuencialmente, pues se trata de un módulo que provee soporte para programación paralela. Esto significa que **is_alive** no será llamado inmediatamente después de que **start** termine de ejecutar el hilo, sino después de que

start sea llamado (mientras tanto, el hilo podría seguir en ejecución). La mejor forma de comprobar esto, sería que el destino de la llamada (*MiFunción*) se tome su tiempo. Para simular este tiempo, se puede utilizar la función **sleep** del módulo **time**, que simplemente esperará (sin hacer nada) el tiempo que se le indique. Se simulará *MiFunción* con una función llamada **leer** que simplemente espere 10 segundos (notar que esta espera solo se utiliza con fines académicos, para simular una función que tarda 10 segundos en completar la tarea encomendada):

```
from threading import Thread
from time import sleep

def leer():
    sleep(10)

for i in range(0, 5):
    hilo = Thread(target=leer)
    hilo.start()
    if hilo.is_alive():
        print("El hilo se está ejecutando y join será llamado")
    hilo.join()
```

Si se ejecuta el código anterior, se verá que la primera salida en pantalla, ocurre inmediatamente al invocar el script.

El bloqueo del hilo se emplea —como se comentó en el apartado previo— para evitar las condiciones de carrera. Sin embargo, cuando el acceso a los datos sea de solo lectura y ningún otro hilo o proceso los esté modificando, el bloqueo podría evitarse.

Para entender cómo funciona el bloqueo en la práctica, se agregará una salida en pantalla a la función **leer** y se eliminará el bloqueo. Cuando se ejecute el script se verá que la salida en pantalla de los cinco hilos se producirá inmediatamente después de invocar al script. Por otra parte,

se guardarán los hilos en una pila, de forma tal que luego de ver la salida en pantalla (cuando parezca que el script ha finalizado), se pueda ejecutar un llamado a **is_alive** sobre cualquiera de los cinco hilos. Se recomienda ejecutar el código en la shell de Python para una depuración más eficiente:

```
from threading import Thread
from time import sleep

def leer():
    print("Ejecutando...")
    sleep(10)

hilos = []
for i in range(0, 5):
    hilos.append(Thread(target=leer))
    hilos[i].start()
```

Si se ejecuta el código directamente en la shell, se podrá invocar a **is_alive** sobre los diferentes hilos y ver su salida:

```
>>> from threading import Thread
>>> from time import sleep
>>>
>>> def leer():
...     print("Ejecutando...")
...     sleep(10)
...
>>>
>>> hilos = []
>>> for i in range(0, 5):
...     hilos.append(Thread(target=leer))
...     hilos[i].start()
...
Ejecutando...
Ejecutando...
Ejecutando...
Ejecutando...
Ejecutando...
>>> hilos[3].is_alive()
True
>>> hilos[1].is_alive()
```

```
True
>>> hilos[4].is_alive()
True
>>> hilos[1].is_alive()
False
>>> hilos[3].is_alive()
False
```

De esta forma, la función leer, podría emplearse para la lectura simultánea de registros de una base de datos. Por ejemplo, en vez de leer 300 registros en un único proceso, leerlos de a grupos de a 100:

```
from threading import Thread
from time import sleep

def leer(registros=(0, 100)):
    print("Leyendo registros {n} a {m}".format(
        n=registros[0], m=registros[1]))
    sleep(10)

hilos = []
for i in range(0, 3):
    n = i * 100
    m = n + 100 - 1
    hilos.append(Thread(target=leer, args=((n, m), )))
    hilos[i].start()
```

Como se observa en la siguiente salida, los registros se estarían leyendo en paralelo:

```
>>> from threading import Thread
>>> from time import sleep
>>>
>>>
>>> def leer(registros=(0, 100)):
...     print("Leyendo registros {n} a {m}".format(
...         n=registros[0], m=registros[1]))
...     sleep(10)
...
>>>
>>> hilos = []
>>> for i in range(0, 3):
...     n = i * 100
```

```
...     m = n + 100 - 1
...     hilos.append(Thread(target=leer, args=((n, m), )))
...     hilos[i].start()
...
Leyendo registros 0 a 99
Leyendo registros 100 a 199
Leyendo registros 200 a 299
>>> hilos[0].is_alive()
True
```

Los parámetros (todos opcionales) de la clase **Thread** son:

target	Destino a ser ejecutado por el nuevo hilo.
args	Una tupla con los argumentos requeridos por la función destino.
kwargs	Un diccionario (en vez de la tupla anterior).
name	Un nombre para identificar al hilo.
daemon	Si es True, ejecuta el hilo como demonio de sistema.

Otros datos que pueden resultar útiles a la hora de programar con hilos, son:

- *ID del proceso* que se obtiene mediante **getpid()** del módulo **os**.
- La *ID del hilo*, que se obtiene mediante el atributo **ident** del objeto **Thread**.
- La *ID nativa del hilo* provista por el sistema operativo, que desde la versión 3.8 de Python se obtiene con el atributo **native_id** del objeto **Thread**.

Para más atributos, métodos, y documentarse sobre otros objetos del módulo **threading**, por favor, visítase la documentación oficial en <https://docs.python.org/3/library/threading.html>.

23.3.2 PARALELISMO BASADO EN PROCESOS

Si bien la implementación del multiprocesamiento en Python no es tan diferente de la programación multihilos, el paralelismo basado en procesos tienen un mayor grado de complejidad. Dado el gran consumo de recursos que hace —en comparación a los hilos—, su diseño no puede depender de una decisión de programación, si no de un exhaustivo trabajo de ingeniería.

No obstante, la programación basada en procesos se presenta en esta sección a simple título informativo, y se sugiere no implementar la programación paralela basada en procesos, sin el correspondiente estudio en profundidad del tema, y el análisis particular y minucioso del problema que se intenta resolver.

En su forma más simple, el módulo **multiprocessing** provee la clase **Process** que se implementa de la misma forma que **Thread**, con la diferencia que en vez de crear hilos en un mismo proceso, crea múltiples procesos:

```
from multiprocessing import Process
from time import sleep

def leer():
    sleep(3)

for i in range(0, 5):
    proceso = Process(target=leer)
    proceso.start()
    if proceso.is_alive():
        print("El proceso se está ejecutando y join será llamado")
        proceso.join()
```

Notar que a diferencia de los hilos, cuando se trabaje con procesos, siempre es recomendable invocar al método `join` para **evitar procesos zombie remanentes**¹⁰.

Una de las diferencias de **Process** con **Thread**, es la posibilidad de **finalizar un proceso**. Los procesos pueden finalizarse de dos formas:

1. *Cerrar el proceso* una vez que el proceso finaliza, con el método **close**. Este método lanzará un error si el proceso intenta cerrarse estando en ejecución.
2. *Terminar un proceso* cuando aún se esté ejecutando, enviando una señal **SIGTERM** mediante el método **terminate** (el cual podría dejar hilos huérfanos), o una señal **SIGKILL** mediante el método **kill**.

Estos métodos pueden utilizarse en forma conjunta con **is_alive**:

```
if proceso.is_alive():  
    proceso.kill()
```

Notar que el código anterior podría ser necesario en casos donde el proceso continúe en ejecución pero sin responder.

Para conocer el motivo por el cual un proceso ya no está en ejecución, se puede utilizar el atributo **exitcode**:

```
>>> def leer():  
...     print("Ejecutando...")  
...     sleep(30)  
...  
>>>  
>>> proceso = Process(target=leer)
```

10 <https://docs.python.org/3/library/multiprocessing.html#programming-guidelines>

```
>>> proceso.start()
>>> Ejecutando...

>>> proceso.kill()
>>> proceso.exitcode
-9
```

Por otra parte, al momento de crear un proceso, también puede definirse el método que será empleado para su creación, el cual dependerá de la plataforma en la que se ejecute. Los **métodos de creación de procesos** y características se describen en la siguiente tabla:

Método	Plataformas soportadas	Descripción
<i>spawn</i>	Unix, Windows, MacOS	El proceso se inicia en un nuevo intérprete. Es el método por defecto en Windows y MacOS, y es mucho más lento que los otros dos métodos.
<i>fork</i>	Unix	Divide el intérprete de Python (lo bifurca) para el nuevo proceso. El proceso hijo es una copia exacta del proceso padre y dispone de exactamente los mismos recursos. Es el método por defecto en sistemas Unix.
<i>forkserver</i>		Para crear un nuevo proceso, el proceso principal solicita la bifurcación a un proceso de servidor de un único hilo, creado al inicio del programa. El proceso hilo solo hereda los recursos necesarios. No funciona en plataformas Unix que no admitan el paso de descriptores de archivos mediante tuberías.

Para definir el método de creación del proceso, al inicio del programa se debe recurrir a la función **set_start_method** del módulo **multiprocessing**, quien recibe como parámetro una cadena de texto con uno de los tres valores descritos en la tabla anterior.

```
>>> from multiprocessing import set_start_method
>>> set_start_method('forkserver')
```

23.3.2.1 GRUPOS DE PROCESOS MEDIANTE LA CLASE POOL

Además de la clase *Process*, el módulo *multiprocessing* dispone de una clase llamada **Pool** que administra los recursos disponibles para un grupo de procesos determinado.

Esta clase recibe un conjunto de procesos que son ejecutados en la medida que se disponga de recursos libres para llevar a cabo las tareas. Esto supone la posibilidad de ir liberando dichos recursos conforme las tareas se vayan completando.

Puede compararse **Pool** a una cola de procesos, pero sin un orden de salida, en la que, como se ha dicho, los procesos se irán ejecutando solo si hay disponibilidad de recursos para hacerlo. Por ello, no se llama *cola* si no que posee una denominación (*piscina*, en español) más acercada a la de un *contenedor de procesos*.

En lo sucesivo se emplean los términos *grupo de procesos*, *conjunto de procesos*, y *contenedor de procesos*, indistintamente, para referirse en español a los objetos de tipo **Pool**.

La forma de operar, consiste en crear primero un conjunto de procesos vacío. Por defecto, Python creará un contenedor con un proceso por cada CPU disponible¹¹ en el equipo. Este valor es verificado por la función **cpu_count** del módulo **os**:

```
>>> from os import cpu_count
>>> cpu_count()
4
```

Para constatar los recursos que se vayan ocupando y liberando, durante todo el procedimiento se irá recurriendo al comando **ps**, manteniendo

¹¹ En este contexto, *disponible* se refiere a *cantidad total* y no a *recursos libres*.

dos terminales abiertas: una para ejecutar el código en una shell interactiva, y otra, para el comando **ps**. Dado que a los fines requeridos interesan los valores correspondientes a ID de proceso (*pid*), estado del proceso (*stat*), y comando (*cmd*), se dará formato a la salida de **ps** solo con dichos campos. Adicionalmente, se agregará también la terminal (*tty*), con el objetivo de localizar la salida deseada rápidamente.

Solo con la shell interactiva de Python se verá una salida similar a la siguiente:

```
eugenia@bella:~$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      922988 S+      \_ grep python3
pts/2      922985 S+      \_ python3
```

Donde:

pts/2 hace referencia a la TTY2,

922985 es la ID del proceso de la shell interactiva de Python 3,

S+ es el estado del proceso (durmiendo en segundo plano), y

python3 el comando ejecutado (o binario en ejecución).

Si se crea un objeto Pool en la TTY2, enseguida se podrán constatar los nuevos procesos en la TTY1:

```
# TTY2
from multiprocessing import Pool
p = Pool()

# TTY1
$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      923015 S+      \_ grep python3
pts/2      922985 Sl+      \_ python3
pts/2      923007 S+      \_ python3
pts/2      923008 S+      \_ python3
pts/2      923009 S+      \_ python3
pts/2      923010 S+      \_ python3
```

ENTENDER LOS ESTADOS DE UN PROCESO ARROJADOS POR PS

Estados	
Estado	Descripción
S	(Sleep) durmiendo, interrumpible.
D	Durmiendo, ininterrumpible.
R	(Running) en ejecución.
T	Parado por una señal de interrupción.
t	Parado por un depurador.
Z	Zombie. Un proceso caducado.
Símbolos adicionales	
Símbolo	Descripción
+	Proceso en segundo plano
l	Proceso multihilo (proceso clonado).
<	Proceso de alta prioridad.
N	Proceso de baja prioridad.
L	Proceso con páginas bloqueadas en la memoria.

Para más información se puede visitar las páginas del manual, mediante la ejecución del comando:

```
man ps
```

Por defecto, se han creado cuatro procesos (coincidentes con el valor arrojado por `os.cpu_count()`), todos dependientes del PID 922985. Estos se encuentran inicializados y deberán ser finalizados para liberar recursos (pues mientras la TTY2 permanezca ejecutando la shell interactiva, los procesos del grupo seguirán activos).

Para finalizarlos, se debe recurrir al método `terminate()` pues `close()` solo impide que se agreguen nuevas tareas, razón por la cual los procesos quedan en estado zombie:

```
# TTY2
p.close()

# TTY1
$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      923258 S+      \_ grep python3
pts/2      922985 S+      \_ python3
pts/2      923007 Z+      \_ [python3] <defunct>
pts/2      923008 Z+      \_ [python3] <defunct>
pts/2      923009 Z+      \_ [python3] <defunct>
pts/2      923010 Z+      \_ [python3] <defunct>
```

En cambio, invocando a `terminate()` los procesos son destruidos (incluso aunque no hayan concluido su tarea):

```
# TTY2
p.terminate()

# TTY1
eugenia@bella:~$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      923318 S+      \_ grep python3
pts/2      922985 S+      \_ python3
eugenia@bella:~$
```

Para crear una cantidad de procesos diferente, se debe instanciar `Pool` con el argumento `processes`, asignándole un número natural como valor:

```
# TTY2
p = Pool(processes=6)

# TTY1
$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      923499 S+      \_ grep python3
pts/2      922985 Sl+     \_ python3
pts/2      923482 S+      \_ python3 <-- 1
pts/2      923483 S+      \_ python3 <-- 2
pts/2      923484 S+      \_ python3 <-- 3
pts/2      923485 S+      \_ python3 <-- 4
```

```
pts/2      923486 S+      \_ python3 <-- 5
pts/2      923487 S+      \_ python3 <-- 6
```

A un objeto de tipo **Pool** se le pueden ir agregando procesos que realicen diversas tareas de forma asíncrona. Para ello se puede emplear el método **apply_async**. En el siguiente ejemplo, se recurre a la función leer empleada en la página 154:

```
# TTY2
salida = p.apply_async(leer)
```

La función **leer** demorará por lo menos 30 segundos en finalizar. El resultado de la ejecución se puede recuperar con el método **get**, al que además se le puede establecer un tiempo de espera máximo mediante el parámetro **timeout**.

salida.get() no arrojará ninguna salida a menos que el proceso haya finalizado su tarea. Si se invoca **get** con un límite de tiempo y el proceso no logra finalizar, se obtendrá un error de tiempo de espera agotado:

```
>>> salida = p.apply_async(leer)
>>> salida.get(timeout=10)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.7/multiprocessing/pool.py", line 653,
in get
    raise TimeoutError
multiprocessing.context.TimeoutError
```

Recordar que **leer**, invoca a **sleep** con 30 segundos, y por lo tanto, un tiempo de espera máximo (*timeout*) de 10, resultará insuficiente (a no ser que se invoque a **get** cuando hayan pasado al menos 20 segundos desde el inicio de su ejecución).

Mientras tanto, la TTY1 seguirá arrojando la misma salida para **ps**:

```
$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      924190 S+      \_ grep python3
pts/2      922985 Sl+     \_ python3
pts/2      923986 S+      \_ python3
pts/2      923987 S+      \_ python3
pts/2      923988 S+      \_ python3
pts/2      923989 S+      \_ python3
pts/2      923990 S+      \_ python3
pts/2      923991 S+      \_ python3
```

Modificando la función `leer`, se podrá obtener una mejor perspectiva de cómo actúa el paralelismo mediante los grupos de procesos. Para este ejemplo se finalizan los procesos anteriores mediante la salida de la shell y se crea un nuevo escenario.

TTY2

```
>>> from multiprocessing import Pool
>>> from time import sleep
>>>
>>>
>>> def leer(tarea, tiempo):
...     print('Ejecutar {} en {}'.format(tarea, tiempo))
...     sleep(tiempo)
...     print("Tarea {} finalizada".format(tarea))
...
>>>
>>> p = Pool(2)
>>> a = p.apply_async(leer, (1, 10))
>>> b = p.apply_async(leer, (2, 3))
>>>
```

Ejecutar 1 en 10"

```
>>> b.get()
```

Ejecutar 2 en 3"

Tarea 2 finalizada

```
>>> a.get()
```

Tarea 1 finalizada

```
>>>
```

TTY1

```
eugenia@bella:~$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      924937 S+      \_ grep python3
pts/2      924921 Sl+     \_ python3
pts/2      924931 S+      \_ python3
pts/2      924932 S+      \_ python3
```

Un ejemplo similar podría verse con mayor claridad si se invocase al método **map** para implementar paralelismo de datos. Se restablece el escenario, esta vez empleando el siguiente código:

```
from multiprocessing import Pool
from time import sleep

def leer(datos):
    tarea, tiempo = datos
    print('Ejecutar {} en {}'.format(tarea, tiempo))
    sleep(tiempo)
    print("Tarea {} finalizada".format(tarea))

p = Pool(2)

argumentos = (
    (1, 10),
    (2, 5),
    (3, 6),
    (4, 9),
    (5, 8)
)
resultado = p.map(leer, argumentos)
```

La salida se verá como la siguiente:

```
>>> resultado = p.map(leer, argumentos)
Ejecutar 1 en 10"
Ejecutar 2 en 5"
Tarea 2 finalizada
Ejecutar 3 en 6"
Tarea 1 finalizada
Ejecutar 4 en 9"
Tarea 3 finalizada
Ejecutar 5 en 8"
Tarea 5 finalizada
Tarea 4 finalizada
```

Como se comentó al comienzo, ninguno de los procesos finalizará por completo hasta que el proceso que los ha creado finalice, o se llame a **terminate()**. Una forma de liberar los recursos sin recurrir a **terminate()** es emplear **Pool** dentro de la estructura **with**. Se

finalizan los procesos previos mediante `p.terminate()`, y se procede a crear un nuevo grupo de procesos con la estructura `with`, no sin antes verificar los ID de cada proceso mediante `ps`:

```
# TTY1
eugenia@bella:~$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      926494 S+    \_ grep python3
pts/2      925397 Sl+    \_ python3
pts/2      926476 S+    \_ python3
pts/2      926477 S+    \_ python3
```

```
# TTY2
with Pool(2) as p:
    argumentos = (
        (1, 10),
        (2, 5),
        (3, 6),
        (4, 9),
        (5, 8)
    )
    resultado = p.map(leer, argumentos)
```

Salida:

```
Ejecutar 1 en 10"
Ejecutar 2 en 5"
Tarea 2 finalizada
Ejecutar 3 en 6"
Tarea 1 finalizada
Ejecutar 4 en 9"
Tarea 3 finalizada
Ejecutar 5 en 8"
Tarea 5 finalizada
Tarea 4 finalizada
```

```
# TTY1 (nuevamente, al finalizar las tareas anteriores)
eugenia@bella:~$ ps -otty -opid -ostat -ocmd -af | grep python3
pts/1      926503 S+    \_ grep python3
pts/2      925397 S+    \_ python3
```

Como se puede observar, los recursos se han liberado tras finalizar las tareas.

Para un estudio completo de la clase **Pool**, se sugiere visitar la documentación oficial:

<https://docs.python.org/3/library/multiprocessing.html#module-multiprocessing.pool>

24. PROBABILIDAD Y ESTADÍSTICA CON PYTHON

24.1 FUNCIONES ESTADÍSTICAS BÁSICAS (LEN, SUM, MAX, MIN)

Sobre listas y tuplas pueden efectuarse operaciones estadísticas simples empleando funciones incorporadas del lenguaje:

Contar elementos	<code>len(coleccion)</code>
Sumar elementos	<code>sum(coleccion)</code>
Obtener número mayor	<code>max(coleccion)</code>
Obtener número menor	<code>min(coleccion)</code>

En lo sucesivo se abarcarán operaciones más complejas empleadas para la ciencia de datos.

24.2 PROBABILIDAD DE SUCESOS SIMPLES Y COMPUESTOS

MUTUAMENTE EXCLUYENTES

24.2.1 ESPACIO MUESTRAL

Un **espacio muestral** es un conjunto de sucesos posibles, como los que podrían resultar al lanzar un dado:

$$E = \{ 1, 2, 3, 4, 5, 6 \}$$

```
espacio_muestral = [1, 2, 3, 4, 5, 6]
```

Se refiere como **punto muestral** a cada elemento en un espacio muestral. La cantidad de puntos muestrales se denota por n tal que para el espacio muestral $E = \{1, 2, 3, 4, 5, 6\}$, $n = 6$

```
n = len(espacio_muestral)
```

24.2.2 SUCESOS SIMPLES Y COMPUESTOS

Un **suceso**, es un conjunto de resultados dentro de un espacio muestral. Por ejemplo:

- el lanzamiento de un dado es un suceso
- la probabilidad de que en dicho lanzamiento salga el número 5, es un **suceso simple** y es **excluyente**: si sale 5, no puede simultáneamente salir ningún otro número.
- la probabilidad de que en el lanzamiento salga un número impar, es el **suceso compuesto** $B = \{1, 3, 5\}$ que dependerá a su vez de los sucesos simples excluyentes $B_1 = \{1\}$, $B_2 = \{3\}$ y $B_3 = \{5\}$.

24.2.3 ASIGNACIÓN DE PROBABILIDADES

La asignación de probabilidades es aquella que provee modelos matemáticos para calcular las posibilidades de que sucesos específicos ocurran o no.

La probabilidad de un suceso se denota por $P(\text{suceso})$.

Los sucesos pueden ser:

- simples o compuestos
- mutuamente excluyentes o independientes

24.2.4 SUCESOS SIMPLES MUTUAMENTE EXCLUYENTES

Si se considera un espacio muestral A , cada uno de los puntos muestrales k , quedará denotado por A_k y la probabilidad de éstos, designada como $P(A_k)$, quedará determinada por:

$$P(A_k) = \frac{1}{n}$$

probabilidad = 1.0 / n

*En **Python**, se requiere que al menos un elemento de la ecuación sea un número real si lo que se requiere como resultado es un número real.*

La probabilidad de cada punto muestral, como sucesos excluyentes entre sí, es la misma para cada suceso.

$$P(6) = P(5) = P(4) = P(3) = P(2) = P(1) = \frac{1}{n} = \frac{1}{6}$$

24.2.5 SUCESOS COMPUESTOS POR SUCESOS SIMPLES MUTUAMENTE EXCLUYENTES

Cuando los sucesos simples que conforma al suceso compuesto A son mutuamente excluyente, la probabilidad del suceso compuesto estará dada por la suma de las probabilidades de cada suceso simple

$P(A_k)$, tal que:

$$P(A) = P(A_1) + P(A_2) + \dots + P(A_k)$$

Por ejemplo, para estimar la probabilidad de que en un único lanzamiento de dado, salga un número par, se obtiene el suceso $A = \{2, 4, 6\}$ dado por la suma de las probabilidades de cada uno de sus sucesos simples $P(2) + P(4) + P(6)$ del espacio muestral $E = \{1, 2, 3, 4, 5, 6\}$ tal que:

$$P(A) = P(2) + P(4) + P(6)$$

$$P(A) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6}$$

$$P(A) = \frac{1}{2}$$

En el primer resultado $\frac{3}{6}$ (en el segundo paso, antes de hallar el máximo común divisor [MCD] y reducir la fracción a $\frac{1}{2}$), el numerador es equivalente a la cantidad de sucesos simples dentro del suceso compuesto «números pares» y se denota por h . El denominador, 6 , es n , el total de todos los sucesos del espacio muestral. De esta forma, la probabilidad de un suceso A compuesto por sucesos mutuamente excluyentes queda dada por el cociente de h y n tal que:

$$P(A) = \frac{h}{n}$$

```
numeros_pares = [i for i in espacio_muestral if i % 2 is 0]
h = len(numeros_pares)
```

```
probabilidad = float(h) / n
```

Un suceso compuesto se puede denotar por la unión de sus sucesos simples (símbolo $\cup$, leído como "o"), tal que:

$$P(A_1 \cup A_2 \cup \dots A_k) = P(A_1) + P(A_2) + \dots P(A_k)$$

Por ejemplo, para el caso del suceso «números pares», se obtiene que:

$$P(2 \cup 4 \cup 6) = P(2) + P(4) + P(6)$$

$$P(2 \cup 4 \cup 6) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6}$$

$$P(2 \cup 4 \cup 6) = \frac{1}{2}$$

Tal que $P(2 \cup 4 \cup 6)$ es un suceso y $P(2)$, $P(4)$ y $P(6)$ son las probabilidades de los tres sucesos que lo componen. En un nuevo contexto, $P(2 \cup 4 \cup 6)$ puede ser tratado como un suceso A .

24.2.6 FUNCIONES

```
# Probabilidad de sucesos simples mutuamente excluyentes
pssme = lambda e: 1.0 / len(e)
```

```
# Probabilidad de sucesos compuestos mutuamente excluyentes
def pscme(e, sc):
    n = len(e)
```

```
    return len(sc) / float(n)
```

24.3 PROBABILIDAD CONDICIONAL EN PYTHON

c. Probabilidad de B:

$$B = \{2, 4, 6\}$$

$$P(B) = \frac{h}{n} = \frac{3}{6} = \frac{1}{2}$$

d. Probabilidad de la intersección:

$$P(A \cap B) = P(A) P(B)$$

$$P(A \cap B) = \frac{1}{2} \cdot \frac{1}{2} P(A \cap B) = \frac{1}{4}$$

```
e = espacio_muestral = [1, 2, 3, 4, 5, 6]
n = len(e) # total de la muestra
```

```
# probabilidad de A
a = [i for i in e if i % 2 is not 0]
pa = len(a) / float(n)
```

```
# probabilidad de B
b = [i for i in e if i % 2 is 0]
pb = len(b) / float(n)
```

```
# probabilidad de la intersección de sucesos
```

```
pi = pa * pb
```

24.3.1 FUNCIONES

```
# Probabilidad condicional: sucesos dependientes
```

```
def pscd(e, a, b):
    i = list(set(a).intersection(b))
    pi = pscme(e, i)
    pa = pscme(e, a)
    return pi / pa
```

```
# Probabilidad condicional: sucesos independientes
```

```
def psci(e, a, b):
    pa = pscme(e, a)
    pb = pscme(e, b)
```

```
    return pa * pb
```

24.3.2 SUCESOS DEPENDIENTES

Se refiere a la probabilidad de que dos sucesos ocurran simultáneamente siendo que el segundo suceso depende de la ocurrencia del primero.

La probabilidad de que ocurra B si ocurre A , se denota por $P(A|B)$ y se lee como "la probabilidad de B dado A ", tal que:

$$P(B|A) = \frac{P(A \cap B)}{P(A)}$$

Donde $P(A \cap B)$ es la probabilidad de la intersección de los sucesos de A y B — definida como: $P(A \cap B) \equiv P(A) P(A|B)$ —, tal que la intersección es un nuevo suceso compuesto por sucesos simples. En el siguiente ejemplo, equivaldría a $\{1,3\}$ (porque 1 y 3 están tanto en A como en B).

Ejemplo: ¿qué probabilidad existe de que al lanzar un dado resulte un número impar menor que 4?

El lanzamiento del dado es un suceso en sí mismo. Se desea averiguar la probabilidad de $B = \{1,2,3\}$ (número menor que 4) dado que $A = \{1,3,5\}$ (número impar) ocurriese en el espacio muestral $E = \{1,2,3,4,5,6\}$.

```
espacio_muestral = [1, 2, 3, 4, 5, 6]
a = [i for i in espacio_muestral if i % 2 is not 0]
```

```
b = [i for i in espacio_muestral if i < 4]
```

Para calcular la probabilidad de una intersección, primero se obtiene la intersección:

$$A \cap B = \{1, 3\}$$

```
intersec = [i for i in a if i in b]
```

Y luego, se calcula la probabilidad del nuevo suceso compuesto $\{1, 3\}$:

$$P(A \cap B) = P(1) + P(3) = \frac{1}{6} + \frac{1}{6} = \frac{2}{6} = \frac{1}{3}$$

o, lo que es igual:

$$P(A \cap B) = \frac{h}{n} = \frac{2}{6} = \frac{1}{3}$$

Es necesario además, obtener la probabilidad de A , teniendo en cuenta que es también un suceso compuesto:

$$P(A) = \frac{h}{n} = \frac{3}{6} = \frac{1}{2}$$

Finalmente, se obtiene que:

$$P(B|A) = P(A \cap B)$$

$$P(B|A) = \frac{1/3}{1/2}$$

$$P(B|A) = \frac{2}{3} = 0.\bar{6}$$

```
e = espacio_muestral = [1, 2, 3, 4, 5, 6]
```

```
a = [i for i in e if i % 2 is not 0] # números impares
```

```
b = [i for i in e if i < 4] # números menores que 4
```

```
intersec = [i for i in a if i in b] # intersección de A y B
```

```
n = len(e) # total de la muestra
```

```
ha = len(a) # total de sucesos simples en A
```

```
# total de sucesos simples en la intersección
```

```
hintersec = len(intersec)

# probabilidad de la intersección
probabilidad_intersec = float(hintersec) / n

# probabilidad de 'a'
probabilidad_a = float(ha) / n

# probabilidad condicional
probabilidad_b_dado_a = probabilidad_intersec /
probabilidad_a
```

24.3.2.1 TEORÍA DE CONJUNTOS EN PYTHON

Al obtener la intersección de dos sucesos compuestos se ha empleado un método manual al decir: devolver 'i' por cada 'i' en la lista 'a' si está en la lista 'b'.

No obstante, dado que cada suceso compuesto es un conjunto y que Python provee un tipo de datos llamado **set** (conjunto), es posible obtener la intersección manipulando los sucesos compuestos como conjuntos de Python. Con **set** se puede convertir cualquier iterable a conjunto y realizar operaciones de conjuntos como unión e intersección cuando sea necesario.

```
intersec = list(set(a).intersection(b))
```

Aquí el conjunto obtenido se convierte en lista a fin a fin de guardar coherencia con el resto del código y que el elemento resultante soporte las operaciones y tratamiento habituales de una lista. Ante la duda de si corresponde usar listas o conjuntos, se debe aplicar el principio de simplicidad e implementar la solución más simple.

24.3.3 SUCESOS INDEPENDIENTES

A diferencia del caso anterior, aquí la probabilidad de que ocurra **B** no está afectada por la ocurrencia de **A**. Por ejemplo, la probabilidad

de lanzar un dado y obtener un número par (suceso B) no está afectada por el hecho de que en un lanzamiento previo se obtuviese un número impar (suceso A). La probabilidad de B es independiente de A y está dada por el producto de la probabilidad de ambos sucesos:

$$P(A \cap B) = P(A) P(B)$$

Aquí la intersección es la probabilidad de que confluyan ambos sucesos.

Calculada la probabilidad de ambos sucesos independientes, se multiplican obteniendo:

a. Espacio muestral (para ambos sucesos):

$$E = \{ 1, 2, 3, 4, 5, 6 \}$$

b. Probabilidad de A:

$$A = \{ 1, 3, 5 \}$$

$$P(A) = \frac{h}{n} = \frac{3}{6} = \frac{1}{2}$$

24.4 TEOREMA DE BAYES EN PYTHON

24.4.1 TEOREMA DE BAYES Y PROBABILIDAD DE CAUSAS

Dada una serie de sucesos A_k cuya suma total es un espacio muestral E y un suceso B cualquiera, el Teorema de Bayes permite conocer la probabilidad de que cada suceso A_k de E , sea la causa de B . Por este motivo, también se lo conoce como **probabilidad de causas**.

24.4.1.1 DATOS: CASO PRÁCTICO

Dada una ciudad de **50 000 habitantes**, con la siguiente distribución:

Niñas	Niños	Mujeres	Hombres
11000	9000	16000	14000

Y, un reporte de **9 000 casos de gripe**, distribuidos de la siguiente forma:

Niñas	Niños	Mujeres	Hombres
2000	1500	3000	2500

Se pretende obtener la probabilidad de que la causa de contraer gripe sea el hecho de pertenecer a un determinado sector demográfico (por ejemplo, al sector demográfico conformado por niños o niñas).

24.4.1.2 ANÁLISIS

De lo expuesto *ut supra* se obtiene que:

- La ciudad (total absoluto de habitantes) es el espacio muestral E .
- La cantidad de niñas, niños, mujeres y hombres es cada uno de los sucesos A_k del espacio muestral E .
- Como valor de n se toma la suma del espacio muestral $\sum A_k$, tal que $n = 50000$.
- El valor de h para los sucesos A_k es cada uno de los valores dados en la tabla de distribución de habitantes.
- Tener gripe es el suceso B .

- La tabla de distribución de casos de gripe, se corresponde a las intersecciones del suceso B con cada suceso A_k , es decir a cada $A_k \cap B$

Según el cálculo de probabilidad que se aplique, se podrá obtener:

- La probabilidad de ser niña, niño, mujer u hombre en la ciudad, por medio de $P(A_k)$. Se considera una **probabilidad a priori**.
- La probabilidad de ser niña, niño, mujer u hombre y tener gripe, que se obtiene con $P(A_k|B)$ y se considera una **probabilidad condicional**.
- La probabilidad de que cualquier habitante, independientemente del sector al que pertenezca tenga gripe,

se obtiene con $P(B) = \sum_{k=1}^n P(A_k) P(B|A_k)$ y se la

considera una **probabilidad total**.

- La probabilidad de que alguien con gripe sea niña, niño, mujer u hombre se obtiene con el **Teorema de Bayes**. A esta probabilidad se la considera una **probabilidad a posteriori**, permitiendo dar respuesta a preguntas cómo ¿cuál es la probabilidad de que un nuevo caso de gripe sea de en un niño o niña?

Una forma eficaz y ordenada de obtener una probabilidad a *posteriori* con el Teorema de Bayes, es obtener primero las tres probabilidades previas: a priori, condicional y total.

Conversión recursiva de elementos de una lista

En lo sucesivo, se utilizará `map(float, <lista>)` en el código fuente, para convertir los elementos de una lista en números reales, toda vez que hacerlo no sobrecargue el código.

24.4.1.3 PROCEDIMIENTO

1. Cálculo de probabilidad a priori

Retorna: probabilidad de que un habitante pertenezca a un sector demográfico específico.

Fórmula: $P(A_k) = \frac{h}{n}$

Datos necesarios:

h_k = datos de la tabla de distribución de habitantes

n = siempre es la cantidad total del espacio muestral (50 000) .

Resultados:

$$P(A_1) = \frac{11000}{50000} = 0.22 \text{ probabilidad de ser niña}$$

$$P(A_2) = \frac{9000}{50000} = 0.18 \text{ probabilidad de ser niño}$$

$$P(A_3) = \frac{16000}{50000} = 0.32 \text{ probabilidad de ser mujer}$$

$$P(A_4) = \frac{14000}{50000} = 0.28 \text{ probabilidad de ser hombre}$$

Código Python:

```
habitantes = map(float, [11000, 9000, 16000, 14000])  
n = sum(habitantes)
```

```
pa = [h / n for h in habitantes]
```

2. Probabilidad condicional

Retorna: probabilidad de tener gripe perteneciendo a un sector demográfico específico.

Certeza: A_k (el sector demográfico)

Objetivo: B (la probabilidad de tener gripe)

$$\text{Fórmula: } P(B|A_k) = \frac{P(A_k \cap B)}{P(A_k)}$$

Datos necesarios:

$$P(A_k \cap B) = \frac{h = B_k}{n = 50000}$$

h = intersecciones (datos de la tabla de distribución de casos de gripe)

Resultados:

$$P(B|A_1) = \frac{\frac{2000}{50000}}{0.22} = 0.\overline{18} \text{ probabilidad de tener gripe siendo niña}$$

$$P(B|A_2) = \frac{\frac{1500}{50000}}{0.18} = 0.1\overline{6} \text{ probabilidad de tener gripe siendo niño}$$

$$P(B|A_3) = \frac{\frac{3000}{50000}}{0.32} = 0.19 \text{ probabilidad de tener gripe siendo mujer}$$

$$P(B|A_4) = \frac{\frac{2500}{50000}}{0.28} = 0.18 \text{ probabilidad de tener gripe siendo hombre}$$

Código Python:

```
afectados = map(float, [2000, 1500, 3000, 2500])  
pi = [k / n for k in afectados]
```

```
pba = [pi[i] / pa[i] for i in range(len(pi))]
```

3. Probabilidad total

Retorna: probabilidad de que cualquiera de los habitantes, independientemente del sector demográfico al que pertenezcan, pueda tener gripe.

$$\text{Fórmula: } P(B) = \sum_{k=1}^n P(A_k) P(B|A_k)$$

Datos necesarios:

- probabilidad a priori
- probabilidad condicional.

Resultados:

$$P(B) = P(A_1) P(B|A_1) + P(A_2) P(B|A_2) + P(A_3) P(B|A_3) + P(A_4) P(B|A_4)$$

$$P(B) = 0.22 \cdot 0.18 + 0.18 \cdot 0.1\bar{6} + 0.32 \cdot 0.19 + 0.28 \cdot 0.18$$

$$P(B) = 0.04 + 0.03 + 0.06 + 0.05$$

$$P(B) = 0.18$$

Código Python:

```
productos = [pa[i] * pba[i] for i in range(len(pa))]  
pb = sum(productos)
```

Observaciones:

(a) notar que en la salida anterior existirá una diferencia de .01 con respecto a la solución manual. Esto es debido al redondeo efectuado en la solución manual. Dicha diferencia puede ser erradicada empleando 3 decimales en los valores de la probabilidad condicional (en lugar de dos) en la solución manual.

(b) la probabilidad de NO tener gripe estará dada por $1 - P(B)$ tal que $1 - 0.18 = 0.82$ pero no será necesario utilizarla para este ejemplo con el Teorema de Bayes.

4. Probabilidad a posteriori

Retorna: probabilidad de pertenecer a un sector demográfico específico y tener gripe.

Certeza: B (tener gripe)

Objetivo: A_k (la probabilidad de pertenecer a un sector demográfico concreto)

$$\text{Fórmula: } P(A_k|B) = \frac{P(A_k) P(B|A_k)}{\sum_{k=1}^n P(A_k) P(B|A_k)}$$

Datos necesarios:

$P(A_k) P(B|A_k)$ = el producto obtenido en cada uno de los términos de la probabilidad total

$$\sum_{k=1}^n P(A_k) P(B|A_k) = \text{la probabilidad total}$$

Resultados:

$$P(A_1|B) = \frac{0.04}{0.18} = 0.\overline{22} \text{ probabilidad de ser niña teniendo gripe}$$

$$P(A_2|B) = \frac{0.03}{0.18} = 0.1\overline{6} \text{ probabilidad de ser niño teniendo gripe}$$

$$P(A_3|B) = \frac{0.06}{0.18} = 0.\overline{33} \text{ probabilidad de ser mujer teniendo gripe}$$

$$P(A_4|B) = \frac{0.05}{0.18} = 0.2\overline{7} \text{ probabilidad de ser hombre teniendo gripe}$$

Código Python:

```
pab = [p / pb for p in productos]
```

24.4.2 FUNCIONES

```
# Teorema de Bayes
def bayes(e, b):
    n = float(sum(e))
    pa = [h / n for h in e]
    pi = [k / n for k in b]
    pba = [pi[i] / pa[i] for i in range(len(pi))]
    prods = [pa[i] * pba[i] for i in range(len(pa))]
    ptb = sum(prods)
    pab = [p / pb for p in prods]

    return pab
```

24.4.3 BIBLIOGRAFÍA COMPLEMENTARIA

[0] Probabilidad y Estadística, Murray Spiegel. McGraw-Hill, México 1988. ISBN: 968-451-102-7

25. ESTADÍSTICA DESCRIPTIVA CON PYTHON

25.1 ESTADÍSTICA POBLACIONAL Y MUESTRAL

La estadística poblacional y muestral suele ser parte de la bioestadística. En ella, **población** se refiere al Universo de los elementos que van a ser investigados, es decir, aquellos elementos que serán objeto de una investigación científica. Mientras que una **muestra** se refiere sólo a la parte de esos sujetos, que se tomará en cuenta para la investigación.

AVISO IMPORTANTE. Para creación de scripts retrocompatibles con versiones anteriores a Python 3.4, toda operación de cómputo relativa a medidas descriptivas de tendencia central y de dispersión, podrá hacerse como se explica en lo sucesivo. A partir de la versión 3.4 de Python, se encuentra disponible también, el módulo **statistic**, descrito en la página 206 del capítulo 26 sobre Matemáticas nativas.

25.1.1 MEDIDAS DESCRIPTIVAS DE TENDENCIA CENTRAL

En una muestra, la **media** es la medida descriptiva de tendencia central, de dicha muestra. Matemáticamente, se corresponde con el promedio de una variable determinada, y se obtiene mediante el cociente de la suma de las variables y el total de la muestra:

$$\bar{x} = \frac{\sum x_i}{n}$$

```
media = sum(muestra) / float(len(muestra))
```

Una **variable** es cualquier magnitud que vaya a ser medida, generalmente de forma cuantitativa.

Por ejemplo:

- **Población**, podría ser el total de los miembros de un club de ajedrez de 1500 socios.
- La **muestra**, podrían ser una porción significativa de esos miembros (500).
- La **variable** a cuantificar, podría ser las edades de los sujetos de la muestra.
- La **media** sería el promedio de edad de la muestra.

25.1.2 MEDIDAS DESCRIPTIVAS DE DISPERSIÓN

Dado que una muestra es sólo una parte de la población, en cualquier estadística cuantificable, habrá ciertos desvíos y variaciones, a los que se denomina **medidas descriptivas de dispersión**. Estas medidas, pueden ser la **varianza** y el **desvío estándar**, las cuales determinarán el grado de variabilidad y dispersión de los datos, respectivamente.

25.1.2.1 CÁLCULOS DE DISPERSIÓN

La varianza y el desvío estándar se miden tanto sobre la población, como sobre la muestra.

Para calcular la varianza se requiere calcular previamente la media. Y para calcular el desvío estándar se requiere calcular previamente la varianza, ya que el desvío estándar es la raíz cuadrada de la varianza.

La **varianza** se calcula en base a la *suma de los cuadrados de la diferencia entre cada elemento de la muestra y la media*.

Cuando se trata de la varianza poblacional, se divide por n :

$$s^2 = \frac{\sum_{i=0}^n (x_i - \bar{x})^2}{n}$$

y cuando se trata de varianza muestral, se divide por $n - 1$.

$$s^2 = \frac{\sum_{i=0}^n (x_i - \bar{x})^2}{n-1}$$

En ecuaciones tan complejas, la forma de reducir la ecuación a código fuente, sigue siendo siempre la misma que se utiliza para resolver la ecuación manualmente: comenzar resolviendo desde dentro hacia afuera, es decir que en este caso, primero se resuelven las restas:

```
diferencias = [xi - media for xi in muestras]
```

Luego los cuadrados:

```
potencias = [x ** 2 for x in diferencias]
```

Luego se suma:

```
sumatoria = sum(potencias)
```

Y finalmente se divide:

```
varianza_muestral = sumatoria / (n - 1)  
varianza_poblacional = sumatoria / n
```

Una vez obtenida la varianza, para calcular la **desviación estándar**, se realiza la raíz cuadrada de la varianza: $\sigma = \sqrt{s^2}$

Tener en cuenta que para obtener una raíz cuadrada, habrá que importar la función *sqrt* del módulo *math*:

```
from math import sqrt

desvio_muestral = sqrt(varianza_muestral)
desvio_poblacional = sqrt(varianza_poblacional)
```

Conclusión

La estadística poblacional y muestral es una de las bases de la inferencia en bioestadística. Permite, por un lado, extraer consecuencias sobre la población en base a lo que se observa, y por el otro, establecer una ponderación de riesgos.

25.2 FRECUENCIA ESTADÍSTICA

En estadística la frecuencia está relacionada con la cantidad de veces que un suceso determinado aparece en una muestra. La distribución de frecuencias o tablas de frecuencias son las que se utilizan para visualizar numéricamente las características de un conjunto de datos. Los diferentes cálculos de frecuencia son de los más habituales en estadística, ya que sus resultados facilitan la toma de decisiones. La frecuencia puede ser absoluta o relativa y a su vez acumulada.

25.2.1 FRECUENCIA ABSOLUTA

Representa la cantidad de veces que un valor determinado se repite en una muestra. Por ejemplo, en una muestra de N estudiantes, donde la variable a medir es la edad, ¿cuántos de estos estudiantes tienen x años?

La siguiente es una muestra de 10 estudiantes, en la que la variable de la muestra es la edad. Por lo tanto, la muestra se compone de las edades de cada uno de los 10 estudiantes.

```
muestra = [23, 23, 34, 18, 20, 18, 18, 21, 23, 18]
```

La frecuencia absoluta f_i , será la cantidad de veces que cada una de las edades, se repite en la muestra. Para obtener las frecuencias absolutas, primero se obtienen los valores únicos (sin repeticiones) y luego, se cuenta cuántas veces se repite cada uno de esos valores en la muestra en la muestra.

```
absolutos = []
frecuencias = []
for n in muestra:
    if not n in absolutos:
        absolutos.append(n)
        fi = muestra.count(n)
        frecuencias.append(fi)
```

Dado que para crear una tabla de distribución de frecuencias se deben ordenar los datos de de forma correlativa, una buena práctica es ordenar primero la muestra antes de hacer cualquier cálculo de frecuencia:

```
muestra.sort()
# Salida: [18, 18, 18, 18, 20, 21, 23, 23, 23, 34]
```

De esta forma se obtienen ya organizados los valores absolutos y las frecuencias absolutas, por lo que su visualización será más sencilla:

```
>>> absolutos
[18, 20, 21, 23, 34]

>>> frecuencias
[4, 1, 1, 3, 1]
```

Edad	Frecuencia absoluta
18	4
20	1
21	1
23	3
34	1

La suma de las frecuencias absolutas va a ser igual al total de elementos de la muestra:

$$N = \sum_{i=0}^n f_i$$

```
N = sum(frecuencias)
```

Es decir que $N = n$:

```
n = len(muestra)
```

25.2.2 FRECUENCIA RELATIVA

La frecuencia relativa h_i establece la proporción o porcentaje que una frecuencia absoluta representa para la muestra. Por ejemplo, si de un total de 10 estudiantes, solo 1 tiene 20 años, ¿qué proporción representa 1 para un total de 10? Ese 10% es la frecuencia relativa de 1, y se obtiene dividiendo la frecuencia absoluta por el total de elementos:

$$h_i = \frac{f_i}{N}$$

```
relativas = [float(fi) / N for fi in frecuencias]
```

El resultado será:

```
[0.4, 0.1, 0.1, 0.3, 0.1]
```

Edad	Frecuencia absoluta	Frecuencia relativa
18	4	0.4
20	1	0.1
21	1	0.1
23	3	0.3
34	1	0.1

Por lo tanto la suma de todas las frecuencias relativas siempre será 1:

```
suma_relativas = round(sum(relativas))
```

Se debe tener en cuenta que en un ordenador, los números decimales sólo son representaciones logradas a partir de números binarios. Por lo tanto, no siempre son exactas.

Una muestra de ello, podría ser realizar la suma anterior sin aplicar la función round:

```
>>> sum([0.4, 0.1, 0.1, 0.3, 0.1]...)
0.9999999999999999
```

25.2.3 FRECUENCIAS ACUMULADAS

La **frecuencia acumulada** F_i o *frecuencia absoluta acumulada*, se obtiene sumando todas las frecuencias absolutas menores o iguales a la frecuencia absoluta considerada.

```
acumuladas = [sum(frecuencias[:i+1]) for i, fi in enumerate(frecuencias)]
```

Alternativamente, puede obtenerse sin desperdicio de valores, mediante:

```
acumuladas = [sum(frecuencias[:i+1]) for i in range(len(frecuencias))]
```

La **frecuencia relativa acumulada** H_i , es aquella que resulta del cociente de su frecuencia absoluta acumulada y el total de datos:

```
relativas_acumuladas = [float(f) / N for f in acumuladas]
```

La tabla de distribución de frecuencias podría verse como sigue:

Edad	f_i	h_i	F_i	H_i	p_i
18	4	0.4	4	0.4	40%
20	1	0.1	5	0.5	10%
21	1	0.1	6	0.6	10%
23	3	0.3	9	0.9	30%
34	1	0.1	10	1.0	10%
SUMA	N=10	1.0	-----		100%

El valor de la última columna p_i , es el **porcentaje**, el cual se obtiene multiplicando el valor relativo por 100: $p_i=h_i \cdot 100$

```
porcentajes = [hi * 100 for hi in relativas]
```

26. MATEMÁTICAS NATIVAS

La biblioteca estándar de Python ofrece siete módulos para dar soporte a operaciones numéricas, estadísticas, y matemáticas en general. Estos módulos se describen en la tabla de la siguiente página.

Módulo	Función										
<i>numbers</i>	Se trata de un módulo que define pseudoclases abstractas para tipos numéricos, pero que no están destinados a ser instanciados, sino heredados ¹² . La jerarquía de tipos definidos es la siguiente:										
	<table><tr><th>Clase</th><th>Propiedades</th></tr><tr><td>Complex</td><td>real, imag</td></tr><tr><td>Real</td><td>- -</td></tr><tr><td>Rational</td><td>numerator, denominator</td></tr><tr><td>Integral</td><td>- -</td></tr></table>	Clase	Propiedades	Complex	real, imag	Real	- -	Rational	numerator, denominator	Integral	- -
	Clase	Propiedades									
	Complex	real, imag									
	Real	- -									
	Rational	numerator, denominator									
Integral	- -										
<i>math</i>	El módulo <i>math</i> provee un acceso directo a funciones matemáticas generales definidas en el estándar del lenguaje C. Algunas de sus funciones se describen a continuación. La documentación completa se encuentra disponible en https://docs.python.org/es/3/library/math.html										
<i>cmath</i>	Similar a <i>math</i> pero con soporte para números complejos.										
<i>decimal</i>	Es una alternativa al tipo <i>float</i> que ofrece mayor precisión en el cómputo aritmético y redondeo de números de coma flotante.										
<i>fractions</i>	Módulo para realizar operaciones de cómputo con números racionales.										
<i>random</i>	Módulo para generación de números pseudoaleatorios criptográficamente inseguros.										
<i>statistic</i>	Funciones estadísticas.										

26.1 EL MÓDULO MATH

Algunas de las funciones destacadas de este módulo son las funciones *comb* (combinatoria), *factorial*, *perm* (permutaciones), *gcd* (máximo común divisor), *prod* (producto), *log* (logaritmo), *pow* (potencia), *sqrt* (raíz cuadrada), *sin*, *cos*, y *tan* (seno, coseno, y tangente, respectivamente). También retorna algunas constantes como *pi* y *e* (número de Euler).

12 Para una mejor comprensión sobre los términos instancia y herencia, se recomienda leer el capítulo 32 sobre Programación orientada a objetos con Python.

26.1.1 COMBINACIONES, FACTORIALES Y PERMUTACIONES

La teoría **combinatoria** formula las bases necesarias para poder realizar cuentas en situaciones complejas que no se resuelven con una simple suma. Una *combinación* es cualquier selección de parte o de todos los objetos simultáneamente, a partir de un número de objetos dados.

Por ejemplo, ¿cuántas posibles combinaciones de dos letras existen a partir de las letras de la palabra *hola*? Se pueden combinar: *ho*, *hl*, *ha*, *ol*, *oa*, y *la*. Es decir, que para un total de 4 letras, tomadas de a 2 letras por vez, existe un total de 6 posibles combinaciones.

El número de combinaciones posibles para n objetos diferentes, seleccionando r objetos por vez, para $r \leq n$ está determinado por:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

Donde:

$n!$ se denomina **n factorial** o factorial de n y se obtiene multiplicando todos los enteros positivos de n hasta el 1. Así, para $n=7$, $n!=7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$ (en la práctica, se prescinde de la multiplicación por 1).

Dado que entre 0 y 1 el único entero positivo es 1, el factorial de 0 es 1, tal que $0!=1$.

nC_r Se lee como combinaciones de n tomadas de r en r . También se denota con el binomio $\binom{n}{r}$.

Así, retomando el ejemplo de combinaciones de letras de la palabra *hola* tomadas de a 2 por vez, las 6 combinaciones posibles se obtendrían a partir de:

$${}^4C_2 = \frac{4!}{2!(4-2)!} = \frac{4 \times 3 \times 2 \times 1}{2 \times 1(4-2)!} = \frac{24}{2 \times 1 \times 2 \times 1} = \frac{24}{4} = 6$$

La función `math.comb(n, r)` disponible desde **Python 3.8**, retorna el número de combinaciones posibles para n elementos tomados de r elementos por vez, sin necesidad de tener que recurrir al factorial:

```
>>> from math import comb
>>> comb(4, 2)
6
```

Sin embargo, en versiones anteriores, sí será necesario recurrir al cálculo factorial, que podrá hacerse mediante la función `math.factorial(x)` donde x es el número a factorizar.

```
>>> from math import factorial
>>> n = 4
>>> r = 2
>>> factorial(n)
24
>>> c = factorial(n) / (factorial(r) * factorial(n-r))
>>> c
6.0
```

En una combinación, el orden de los elementos es indistinto. Cuando el orden —y no la combinación— es importante, se aplican las leyes de la **permutación**. Una permutación es la disposición en un orden determinado de los objetos de un conjunto finito, y la cantidad de permutaciones de n objetos distintos tomados de r en r está determinada por:

$${}^nP_r = \frac{n!}{(n-r)!}$$

Al igual que `comb`, la función `math.perm(n, r)` se encuentra disponible a partir de **Python 3.8** y en versiones previas, habrá que recurrir al cálculo factorial.

```
>>> from math import perm
>>> perm(4, 2)
12
```

26.1.2 NÚMEROS RACIONALES Y DIVISOR COMÚN

Todo número que pueda ser expresado como el cociente de dos enteros (es decir, de una fracción) es un número racional y se expresa en la forma:

$$\frac{p}{q}$$

Para $q \neq 0$. En una fracción existen dos componentes: **numerador** y **denominador**:

$$\begin{array}{l} 3 \leftarrow \text{numerador} \\ \hline 4 \leftarrow \text{denominador} \end{array}$$

Y pueden simplificarse las fracciones, hallando un *número común divisor* por el cual dividiendo el numerador y el denominador, se obtenga un numerador y un denominador más pequeños. Así el número 5 sería el común divisor para $\frac{25}{15}$, pues $25/5=5$ y

$$15/5=3, \text{ por lo tanto } \frac{25}{15} = \frac{5}{3}.$$

La función **math.gcd(n, m)**, disponible desde **Python 3.5**, retorna el común divisor más grande para dos números, y a partir de Python 3.9, lo hace para un número arbitrario de argumentos.

```
>>> from math import gcd
>>> gcd(15, 25)
5
```

Para el manejo de **operaciones aritméticas con números racionales**, se sugiere leer el punto 26.3 «*Manejo de racionales con fractions*» en la página 200.

26.1.3 LOGARITMOS

El logaritmo de un número n con base b , es el exponente al cuál debe elevarse la base b para obtener n . El logaritmo natural es un logaritmo con base e (constante denominada «*número de Euler*»), y se lo denota por $\ln$.

El *número de Euler* es una constante irracional denotada por e y cuyo valor (truncado a los primeros decimales) se aproxima a $e \approx 2.7182818284590\dots$.

La función `math.log(n[, b])` retorna el logaritmo de n con base b . Por defecto, b es el número de Euler por lo que esta función, por defecto, retorna el logaritmo natural.

```
>>> from math import log, e
>>> log(9)
2.1972245773362196
>>> log(9, e)
2.1972245773362196
>>> log(9, 2)
3.1699250014423126
>>> log(9, 10)
0.9542425094393249
```

26.1.4 PRODUCTO, POTENCIA Y RAÍZ CUADRADA

Otras funciones útiles del módulo *math* son *prod*, *pow*, y *sqrt*, las cuáles retornan el producto de un iterable, la potencia y raíz cuadrada de un número, respectivamente. `math.prod(iterable)` se encuentra

disponible desde Python 3.8 mientras que `math.pow(x, potencia)`, y `sqrt(x)` desde la rama 2.

```
>>> from math import prod
>>> prod([2, 4, 5]) # 2 x 4 = 8 x 5 = 40
40

>>> from math import pow
>>> pow(5, 2)
25.0

>>> from math import sqrt
>>> sqrt(25)
5.0
```

26.1.5 RAZONES TRIGONOMÉTRICAS

En geometría, la trigonometría es la disciplina que estudia la relación numérica entre los elementos de un triángulo. La razón trigonométrica es la forma de denominar a los seis cocientes que existen entre los lados de un triángulo rectángulo y uno de sus ángulos agudos.

Así, para una circunferencia de radio r , el **seno** (*sen*) de un ángulo α es el cociente entre la ordenada (distancia vertical [y] hasta el eje horizontal) y el radio, tal que $\text{sen } \alpha = \frac{y}{r}$; el **coseno** (*cos*), el cociente entre la abscisa (distancia horizontal [x] hasta el eje vertical) y el radio, tal que $\text{cos } \alpha = \frac{x}{r}$; y la **tangente** (*tan*), la razón entre seno y coseno (u ordenada y abscisa), tal que $\text{tan } \alpha = \frac{y}{x}$.

Las funciones `math.sin(a)`, `math.cos(a)`, y `math.tan(a)` permiten obtener el *seno*, *coseno* y *tangente* de un ángulo a respectivamente.

Si bien los ángulos pueden medirse en **grados sexagesimales** (donde 1° equivale a $60'$ o $3600''$), matemáticamente se prefieren los **radianes**, y esta es la medida de ángulo que las funciones mencionadas esperan.

Un **radián** es la amplitud de un ángulo cuyo arco mide lo mismo que el radio.

Teniendo en cuenta que la circunferencia se divide en 360° , y que dicha circunferencia equivale a 2π radianes, $\pi = \frac{360^\circ}{2} = 180^\circ$.

Recordar que el **número pi** (π) representa el cociente entre la longitud de la circunferencia y su diámetro.

Por lo tanto, para **convertir los grados a radianes**, se multiplica el ángulo (en grados) por $\pi/180$ y se reduce la fracción a su mínima expresión. Por ejemplo, para un ángulo de 360° (donde el resultado es deducible por lo antedicho):

$$360\left(\frac{\pi}{180}\right) = \frac{360\pi}{180} = \frac{2}{1} = 2\pi$$

Para simplificar la fracción (ángulo sobre 180), se puede recurrir a `gcd`:

```
>>> from math import gcd
>>> mcd = gcd(360, 180)
>>> 360 / mcd
2.0
>>> 180 / mcd
1.0
```

No obstante, en Python pueden convertirse los grados a radianes con la función **`math.radians(grados)`** y de radianes a grados con **`math.degrees(radians)`**:

```
>>> from math import degrees, radians
>>> radians(360)
6.283185307179586
>>> degrees(radians(360))
360.0
```

Al comienzo se mencionó que eran seis las razones trigonométricas de un ángulo. Sin embargo, solo se mencionaron tres. Las otras tres razones trigonométricas son: *cotangente* (razón entre abscisa y ordenada, o entre coseno y seno); *secante* (razón entre 1 y coseno); y *cosecante* (razón entre 1 y seno).

26.2 EL PROBLEMA DE LOS NÚMEROS DE COMA FLOTANTE Y EL MÓDULO DECIMAL

En 1991, David Golberg publicó un artículo denominado (en inglés) «*What Every Computer Scientist Should Know About Floating-Point Arithmetic*» (en español: «*Lo que todo/a informático/a debería saber sobre la aritmética de coma flotante*»), en el cual —entre otras cosas—, explicaba el problema del redondeo con los números de coma flotante, en el sistema binario. Si se tuviese que resumir todo el problema planteado por Goldberg en una única frase, se podría decir que **el problema de los números de coma flotante radica en tener que representar de forma finita un dato que por definición, no lo es**. Pues los números racionales, son infinitos.

Como se explica en «Fundamentos de Ciencias Informáticas para el abordaje de la programación» (Bahit, 2021) al hablar de la diferencia entre las matemáticas discretas y continuas: «(...) [al] pensar en la cantidad de números naturales que existen entre dos números (por ejemplo, entre 1 y 2) y la cantidad de números reales que existen entre los mismos dos

números, enseguida será posible determinar que no existe un número finito para el segundo caso, pero que sí lo existe para el primero».

Entre el número 0.001 y el 0.002, existen infinitos números, pues 0.0009999999999999999999999998... está entre ambos. Así es posible entender que el problema de lo infinito radica en la cantidad de decimales de un número, que puede ser infinito.

En un sistema informático, la información es representada en sistema binario, cuya unidad mínima de información es el **bit** (conjunción abreviada del término «*binary digit*»). Un *bit* es un dígito binario que solo puede asumir uno de dos valores posibles, el **0** o el **1**. Los datos obtenidos a partir de la transformación de la información a datos binarios se denomina código binario. Existen 2^n combinaciones posibles de dígitos binarios para grupos de n bits, por lo que para grupos de 8 bits existen $2^8 = 256$ combinaciones posibles.

Teniendo en cuenta esto, para representar un número, el sistema reserva un conjunto de bits fijo (múltiplo de n , para $n=8$ ya que 8 bits es el tamaño de una palabra). Explica Michael Overton en «*Floating Point Representation*» (1996) que para almacenar un número de coma flotante se emplean, por ejemplo, palabras de 32 bits, divididas en tres: El primer bit para el signo del número, los siguientes 8 bits para el exponente, y los últimos 23 bits para el coeficiente. Por lo tanto, el número debe ser truncado a esta longitud, y en vez de ser una representación exacta será entonces, una **representación aproximada**.

Cuando la representación binaria es aproximada, su inversa también lo es. Así, se pierde precisión en la aritmética de coma flotante. Un ejemplo de ello puede verse a continuación:

```
>>> 0.0001 + 0.0003
0.00039999999999999996
```

El cálculo anterior, ejecutado sobre una arquitectura de 64 bits, arroja un resultado aproximado al esperado **0.0004**, pero no exacto, y la clase **Decimal** del módulo **decimal** de Python, ofrece mayor precisión en la aritmética de coma flotante que el tipo *float*.

Un ejemplo concreto de necesidad de alcanzar mayor precisión, puede darse en el ámbito financiero donde muchas veces se requiere operar con varios decimales y de la exactitud en el redondeo depende la exactitud en, por ejemplo, un balance contable o un cálculo financiero. Sin embargo, no solo se limita a este ámbito. En física y en bioinformática, de la precisión decimal dependen los resultados de un análisis u observación.

Para trabajar con estos números decimales con precisión, se puede recurrir entonces, a la clase **Decimal** del módulo **decimal**:

```
from decimal import Decimal
a = Decimal(0.0001)
```

La **presión aritmética** se podrá establecer mediante el atributo **prec** a través de la función **getcontext**:

```
>>> from decimal import Decimal, getcontext
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.1')
>>> getcontext().prec = 2
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.10')
>>> getcontext().prec = 3
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.100')
>>> getcontext().prec = 4
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.1000')
>>> getcontext().prec = 5
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.10001')
>>> getcontext().prec = 6
>>> Decimal(0.1) + Decimal(0.00001)
```

```
Decimal('0.100010')
>>> getcontext().prec = 20
>>> Decimal(0.1) + Decimal(0.00001)
Decimal('0.1000100000000000055')
```

prec es un atributo (*propiedad*) de un tipo de objeto denominado *Context*, definido en el módulo *decimal*. Estos objetos hacen referencia por “contexto” al hilo (*subproceso*) en ejecución actual. **prec** solo afecta al resultado de un proceso de cómputo.

Es importante notar que **prec** establece la **precisión aritmética**, pero no redondea un número no computado:

```
>>> Decimal(0.1)
Decimal('0.100000000000000005511151231257827021181583404541015625')
```

Así, solo un número previamente computado será retornado con la precisión establecida:

```
>>> getcontext().prec = 2
>>> Decimal(0.1).ln()
Decimal('-2.3')
```

26.3 MANEJO DE RACIONALES CON FRACTIONS

El módulo **fractions** provee soporte para operaciones aritméticas con números racionales.

Un número racional puede crearse con la instancia de un objeto de tipo **Fraction** a través de diferentes mecanismos que van desde obtener el racional a partir de cualquiera de los tipos numéricos hasta escribir la fracción como una cadena con formato '**numerador/denominador**'.

La sola instancia de la clase **Fraction**, crea el objeto racional $\frac{0}{1}$:

```
>>> from fractions import Fraction
>>> Fraction()
Fraction(0, 1)
```

Las siguientes formas, crean el racional $\frac{5}{2}$:

```
>>> Fraction('25/10')
Fraction(5, 2)
>>> Fraction(Decimal(2.5)) # habiendo importado Decimal previamente
Fraction(5, 2)
>>> Fraction(float(2.5))
Fraction(5, 2)
```

Sin embargo, respecto a los últimos dos ejemplos, y debido al problema con la representación de números con coma flotante mencionada en el apartado anterior, **no se recomienda emplear números de coma flotante** para obtener un racional, dado que en ciertos casos, se conseguirán aproximaciones y no el racional exacto:

```
>>> Fraction(1.1)
Fraction(2476979795053773, 2251799813685248)
```

A partir de un entero, se conserva 1 como denominador:

```
>>> Fraction(25)
Fraction(25, 1)
>>> Fraction(10)
Fraction(10, 1)
```

Tanto el **numerador** como el **denominador** de un objeto *Fraction*, son irreducibles.

Los valores que admite el objeto *Fraction* son valores con signo:

```
>>> Fraction(-25, 10)
Fraction(-5, 2)
```

```
>>> Fraction('5e-6')
Fraction(1, 200000)
```

A partir de la creación de los objetos racionales, la aritmética es directa sin necesidad de recurrir a funciones específicas:

```
>>> Fraction(5, 3) * Fraction(3, 3)
Fraction(5, 3)
>>> Fraction(5, 3) + Fraction(3, 3)
Fraction(8, 3)
>>> Fraction(5, 3) - Fraction(3, 3)
Fraction(2, 3)
>>> Fraction(5, 3) / Fraction(3, 3)
Fraction(5, 3)
```

A continuación, puede verse cómo lo anterior puede aplicarse al proceso de demostración inductiva de la base del siguiente teorema:

$$4+9+14+19+\dots+(5n-1)=\frac{n}{2}(3+5n)$$

Primero se presenta la solución manual (paso a paso, para simplificar su comprensión), y finalmente, el código fuente.

Base inductiva del teorema para el número más bajo posible de n (sea $n=1$):

$$\begin{aligned}((5 \times 1) - 1) &= \frac{1}{2}(3 + (5 \times 1)) \\ ((5) - 1) &= \frac{1}{2}(3 + (5)) \\ 4 &= \frac{1}{2}(8) \\ 4 &= \frac{8}{2} \\ 4 &= 4\end{aligned}$$

Código fuente:

```
>>> (5*1)-1 == Fraction(1, 2) * (3+(5*1))
True
```

26.4 NÚMEROS ALEATORIOS CRIPTOGRÁFICAMENTE INSEGUROS

Matemáticamente, la aleatoriedad es un tema complejo. Si se requiere números aleatorios de alta entropía, la versión 3.6 de Python incorpora un nuevo módulo denominado *secrets*, que satisface la necesidad de **números aleatorios criptográficamente seguros**.

El módulo **random**, sin embargo, ofrece aleatoriedad para todo aquello que no requiera niveles de entropía significativos, como puede ser el caso de las operaciones de cómputo relacionadas con la estadística.

Las funciones principales del módulo se agrupan y describen a continuación. Recordar que para hacer uso de cada una de estas funciones, deben ser importadas desde el módulo **random** con la instrucción:

```
from random import <funcion>
```

GENERACIÓN DE NÚMEROS ALEATORIOS

randint, randrange, getrandbits, y choice

Las primeras tres funciones retornan un número entero pseudoaleatorio, mientras que la última, también puede retornar un número no entero. La diferencia entre cada una de ellas se describe en la siguiente tabla:

Función	Descripción
randint(a, b)	Retorna un entero entre <i>a</i> y <i>b</i> .
randrange(n, m, x) n=start m=stop x=step	Retorna un entero de la secuencia de números existentes cada <i>x</i> posiciones entre <i>n</i> y <i>m</i> (por ejemplo, si <i>n</i> =2, <i>m</i> =10, y <i>x</i> =2, la secuencia empleada es 2, 4, 6, 8).

Función	Descripción
<code>getrandbits(k)</code>	Retorna un entero con k bits aleatorios.
<code>choice(secuencia)</code> <code>secuencia=seq</code>	Retorna un número elegido de una secuencia de números.

Algunos ejemplos se muestran a continuación.

Un número entero entre mil y dos mil:

```
>>> randint(1000, 2000)
1927
```

Un entero par entre mil y dos mil:

```
>>> randrange(1000, 2000, 2)
1102
>>> randrange(1000, 2000, 2)
1778
```

Un entero de 64 bits:

```
>>> getrandbits(64)
1439358294053862086L
```

Un número a partir de una secuencia:

```
>>> secuencia = [1025.9, 2765, 91276, 23, 22.09, 76.9862, 75e-7]
>>> choice(secuencia)
2765
```

GENERACIÓN DE SECUENCIAS ALEATORIAS

choices y sample

Estas dos funciones retornan secuencias de elementos aleatorios. La diferencia entre ambas radica en que mientras *sample* retorna una muestra única del conjunto de elementos, *choices* puede retornar una

muestra mayor a la secuencia dada, ya que utiliza la reposición de elementos.

```
>>> secuencia = ['foo', 21545, 'bar', True, (9, 22), 15.5]

>>> choices(secuencia, k=3)
[21545, 15.5, True]
>>> sample(secuencia, 3)
['foo', 'bar', 21545]

>>> choices(secuencia, k=8)
['foo', 21545, 21545, 'bar', True, 15.5, 'bar', 'bar']
```

Por otra parte, la función **shuffle** también permite mezclar los elementos de una secuencia, ya sea la secuencia definida o la retornada por *choices* o *sample*:

```
>>> shuffle(secuencia)
>>> secuencia
[True, 15.5, 'bar', 'foo', (9, 22), 21545]
```

GENERACIÓN DE DISTRIBUCIONES ALEATORIAS

gauss y **gammavariate**

Ambas funciones retornan un valor distribuido aleatoriamente, en el caso de *gauss*, para una media y desvío estándar (distribución Gaussiana), y en el caso de *gammavariate*, la forma de la distribución y el alcance (distribución Gamma).

```
gauss(mu=media, sigma=desvio)
>>> gauss(250, 2)
248.20710079427286

gammavariate(alfa=forma, beta=alcance)
>>> gammavariate(250, 2)
517.2714809214665
```

Otras funciones relativas a la distribución de probabilidades pueden encontrarse en la documentación oficial:

<https://docs.python.org/3/library/random.html#real-valued-distributions>

26.5 EL MÓDULO STATISTIC

A partir de la **versión 3.4 de Python**, el módulo *statistic* provee funciones para el cálculo de promedios, y medidas descriptivas de dispersión y tendencia central.

AVISO IMPORTANTE. Si se requiere retrocompatibilidad con la rama 2, pueden implementarse las soluciones propuestas en el capítulo 25: «*Estadística descriptiva con Python*» de página 182.

Las medidas de dispersión muestral y poblacional, se resumen en la siguiente tabla:

Medida	Muestral	Poblacional
Varianza	<code>variance(data)</code>	<code>Pvariance(data)</code>
Desvío estándar	<code>stdev(data)</code>	<code>pstdev(data)</code>

En cuanto a las medidas de tendencia central, las principales se resumen en la siguiente tabla:

Medida	Función
Media	<code>mean(data)</code>
Meadiana	<code>median(data)</code>
Moda	<code>mode(data)</code>
Mediana más baja	<code>median_low(data)</code>
Mediana más alta	<code>median_high(data)</code>

Media: valor promedio de los datos (sumados y divididos por la cantidad total de los datos).

Mediana: valor que queda en la mitad de los datos ordenados de menor a mayor.

Moda: valor más frecuente de un conjunto de datos.

27. PYTHON COMO CGI PARA APLICACIONES WEB

Cualquier lenguaje de programación, incluso aquellos lenguajes compilados como C, pueden ser empleados para crear aplicaciones Web. Python no es una excepción y al igual que el resto de lenguajes, no requiere de herramientas especiales ni de *Frameworks* para emplearse en la programación de aplicaciones Web.

Para programar aplicaciones Web sobre Apache, en cualquier lenguaje, solo se necesita:

- El intérprete del lenguaje (Python, en este caso).
- El servidor HTTP de Apache.
- El módulo *cgi* para el servidor HTTP de Apache.

Este último requisito, es el que hace posible que las aplicaciones Web puedan ser programadas en cualquier lenguaje soportado por el Sistema Operativo.

27.1 ENTENDER LA INTERFAZ CGI

CGI es una capa intermedia (*gateway*) entre un servidor HTTP y los *scripts* escritos en cualquier lenguaje que pueda ser entendido por el

Sistema Operativo. Esto significa que cualquier *script* que pueda ser ejecutado manualmente en el sistema operativo también puede ser ejecutado por una interfaz cuando se accede desde un servidor HTTP.

Un *script* accesible a través de un servidor Web es conocido como «*Script CGI*».

El único requisito que un *script* CGI debe cumplir, es considerar las especificaciones de entrada y salida del protocolo HTTP. Esto significa:

- Los métodos de entrada del usuario difieren de los métodos de entrada estándar de la línea de comandos. Los métodos de entrada estándar más habituales en el protocolo HTTP, son los métodos GET y POST que se explican más adelante.
- La salida solo difiere en que previo al mensaje (*print* habituales) se deben enviar unas cabeceras.

Antes de profundizar en estos temas se da un ejemplo práctico de código fuente en Python que ayude a entender las diferencias entre un *script* convencional y un *script* CGI.

A continuación se muestran dos versiones de un «Hola Mundo». Una, de un *script* convencional; la segunda, de un *script* CGI.

Script convencional:

```
#!/usr/bin/env python
print("Hola Mundo")
```

Script CGI:

```
#!/usr/bin/env python
print("Content-type: text/html; charset=utf-8")
print("")
print("Hola Mundo")
```

Como puede verse, la única diferencia, en la práctica, para un *script* CGI, es el envío previo de dos líneas: una línea de cabeceras HTTP y una línea en blanco que indica el final de dichas cabeceras y el comienzo del cuerpo del mensaje.

27.2 ENTENDER EL SERVIDOR HTTP DE APACHE

Normalmente, cuando una solicitud HTTP es enviada a un servidor, Apache recibe dicha solicitud y decide qué hacer con ella basándose en la información provista en:

- **El archivo de configuración del servidor HTTP.** En servidores basados en Debian GNU/Linux suele encontrarse en `/etc/apache2/apache2.conf`. Los basados en Fedora o Red Hat, suelen tener este archivo en `/etc/apache2/httpd.conf`.
- **El archivo de configuración del *host* virtual (Virtual Host de Apache).** Se trata del archivo de configuración específico para cada “dominio” (técnicamente, para cada «*host virtual*» almacenado en el servidor. Debe recordarse que un equipo informático es un único *host* y que el servidor HTTP de Apache permite simular el manejo de múltiples *hosts* en un mismo equipo). Estos archivos se localizan en el directorio `/etc/apache2/sites-available`.

Opcionalmente, Apache también puede acceder al **archivo de configuración del directorio** raíz de la aplicación (y al de sus respectivos subdirectorios), denominado `.htaccess`.

El archivo de configuración de Apache, tiene directivas comunes a todos los *host*. Este archivo no suele tener indicaciones particulares. Las indicaciones particulares se encuentran en el *Virtual Host*, y es aquí

donde se establece si los archivos de una aplicación deben ser ejecutados con algún módulo determinado como CGI.

Los módulos de Apache son bibliotecas que otorgan funciones específicas al servidor HTTP. En ocasiones, Apache se vale de estos módulos para realizar determinadas acciones. Una de ellas, es permitir la ejecución de *scripts*. Para permitir la ejecución de *scripts* CGI, el servidor HTTP de Apache debe valerse de un módulo como el módulo *cgi*. También existen otras alternativas (como FastCGI, por ejemplo) pero dado el carácter sintético de este apartado, solo se abarcará *cgi*.

Normalmente, un Virtual Host a la escucha en un puerto específico, define al menos tres parámetros:

- Un nombre de servidor.
- Un directorio raíz para la aplicación.
- La ruta de los archivos de registro.

```
<VirtualHost *:80>
    ServerName eugeniabahit.co.uk
    DocumentRoot "/var/www/torc-web/html"
    ErrorLog "/var/log/torc-web/error.log"
    CustomLog "/var/log/torc-web/access.log" combined
</VirtualHost>
```

Adicionalmente, puede definir indicaciones particulares para un directorio específico, como por ejemplo, la opción de no crear un índice de archivos para evitar exponer el árbol de directorios de la aplicación:

```
<VirtualHost *:80>
    ServerName eugeniabahit.co.uk
    DocumentRoot "/var/www/torc-web"
    ErrorLog "/var/log/torc-web/error.log"
    CustomLog "/var/log/torc-web/access.log" combined

    <Directory "/var/www/torc-web/html">
        Options -Indexes
```

```
</Directory>  
</VirtualHost>
```

Si se desea permitir la ejecución de *scripts* CGI, será necesario:

- Habilitar dicha ejecución de forma concreta.
- Especificar el módulo que se encargará del manejo de solicitudes.

```
<VirtualHost *:80>  
    ServerName eugeniabahit.co.uk  
    DocumentRoot "/var/www/torc-web"  
    ErrorLog "/var/log/torc-web/error.log"  
    CustomLog "/var/log/torc-web/access.log" combined  
  
    <Directory "/var/www/torc-web/html">  
        Options -Indexes +ExecCGI  
        setHandler cgi-script  
    </Directory>  
</VirtualHost>
```

Conclusiones. Como puede observarse, las únicas diferencias que existen en comparación a la creación de un sitio Web estático (sin programación del lado del servidor) son solo dos:

- Una, a nivel de aplicación: la necesidad de enviar las cabeceras HTTP antes de imprimir la salida.
- Otra, a nivel de servidor: la necesidad de definir dos directivas adicionales en el Virtual Host (+ExecCGI y *setHandler*).

27.3 MONTAR UN VIRTUAL HOST LOCALMENTE

En lo sucesivo se darán unas bases de conocimiento mínimas para que estudiantes de programación de nivel medio e inicial puedan introducirse en el tema, pero consideraciones más profundas sobre la Ingeniería de Aplicaciones Web serán tratadas en futuras publicaciones.

A continuación, se explican detalladamente los pasos a seguir para montar un *VirtualHost* local para ejecutar *scripts* CGI mediante el protocolo HTTP.

27.3.1 INSTALACIÓN Y CONFIGURACIÓN DE APACHE

Para instalar Apache en Debian GNU/Linux y derivados, se debe ejecutar:

```
apt install apache2
```

En Fedora, Red Hat y derivados:

```
dnf install httpd  
systemctl enable httpd.service
```

La última línea habilita el inicio automático del servicio. En Debian y derivados el inicio es automático tras la instalación¹³.

Para más información sobre instalación, visitar la documentación oficial en el siguiente enlace:

<http://httpd.apache.org/docs/trunk/en/install.html>

Antes de comenzar, se recomienda habilitar el acceso de Apache al directorio *home* del usuario. Esto solo debe hacerse localmente, nunca en producción.

Para ello, se debe editar el archivo de configuración (*apache2.conf* o *httpd.conf* según aplique) y sustituir */var/www* por */home/NOMBREDEUSUARIO* en el siguiente bloque:

```
<Directory /home/eugenia>  
    Options Indexes FollowSymLinks  
    AllowOverride None
```

13 No se ha comprobado si el inicio es automático también en distribuciones basadas en Fedora.

```
    Require all granted
</Directory>
```

Por favor, notar que todos los cambios que se realicen sobre Apache requieren su ejecución como **root**.

27.3.2 HABILITACIÓN DEL MÓDULO CGI

En algunos sistemas **nix*, la instalación de Apache por defecto utiliza el módulo de multiproceso *event* o *worker*, mientras que en otras, *prefork*. Mientras que para los dos primeros se debe emplear el módulo *cgid*, para el último, será necesario *cgi*. Para un primer acercamiento al *scripting* sobre interfaz CGI, se recomienda configurar Apache para trabajar con el módulo de multiproceso *prefork* y el módulo *cgi*, debido a que con este último se facilita la depuración de errores mediante los registros del sistema. Por lo tanto, por precaución, se deshabilitarán primero *event*, *worker* y *mod_cgid* (si ninguno de ellos estaba habilitado no generará problemas) y luego se habilitarán *prefork* y *mod_cgi* (si estaban habilitados no dará problemas).

Para esto, en Debian y derivados, se deben ejecutar las siguientes instrucciones:

```
a2dismod mpm_event
a2dismod mpm_worker
a2dismod mod_cgid

a2enmod mpm_prefork
a2enmod cgi

systemctl restart apache2
```

En Fedora y derivados, se debe editar el archivo de configuración y agregar o quitar las líneas correspondientes. Estas líneas comienzan con la directiva *LoadModule* y la sintaxis de cada una de ellas es la siguiente:

```
LoadModule <nombre_del_modulo> <ruta>
```

Por ejemplo, la directiva para agregar el módulo *cgi* sería la siguiente:

```
LoadModule cgi_module modules/mod_cgi.so
```

Mientras que la de *prefork* se vería así:

```
LoadModule mpm_prefork_module modules/mod_mpm_prefork.so
```

Para reiniciar Apache en Fedora y derivados, se debe ejecutar:

```
systemctl restart httpd.service
```

27.3.3 DEFINICIÓN DE UN NOMBRE DE HOST NUEVO

Como se comentó previamente, Apache permite emular múltiples *host* en un único equipo. En un servidor público, definir un nombre de *host* requiere:

- Registrar un nombre de dominio público como *eugeniabahit.co.uk*
- Delegar los DNS del dominio al servidor donde será hospedado
- Crear los registros DNS correspondientes («A» para IPv4 y «AAAA» para IPv6)

Sin embargo, cuando el nombre de *host* solo se emplea localmente, basta con agregar una nueva entrada al archivo **/etc/hosts**:

```
127.0.0.1      nuevohost.local
```

Se usa la palabra *local* como TLD de dominio local a modo práctico.

27.3.4 CREACIÓN DE LA ESTRUCTURA DE DIRECTORIOS

Se dará una estructura básica para que pueda probarse el procedimiento. Sin embargo, estructuras más complejas son requeridas en la Ingeniería de Software. Para los fines de este libro, bastarán los siguientes directorios:

- Carpeta del proyecto.
- Carpeta raíz de la aplicación.
- Carpeta para almacenamiento de registros.

Un ejemplo de estructura se vería como la siguiente:

```
nuevohost/      # Carpeta del proyecto
├── app          # Raíz de la aplicación
└── logs         # Registros de la aplicación (logs)
```

Puede crearse la estructura ejecutando el siguiente comando sin necesidad de privilegios:

```
mkdir -p nuevohost/{app,logs}
```

27.3.5 CREACIÓN DEL VIRTUAL HOST

Este archivo debe crearse en el directorio **/etc/apache2/sites-available** y por nombre puede llevar el del nuevo *host* seguido de *.conf*. En negrita se muestran los datos variables que deberán ser modificados:

Archivo: **/etc/apache2/sites-available/nuevohost.conf**

```
<VirtualHost *:80>
    ServerName nuevohost.local
    DocumentRoot "/home/eugenia/nuevohost/app"
    ErrorLog "/home/eugenia/nuevohost/logs/error.log"
    CustomLog "/home/eugenia/nuevohost/logs/access.log" combined
```

```
<Directory "/home/eugenia/nuevohost/app">
    Options -Indexes +ExecCGI
    SetHandler cgi-script
</Directory>
</VirtualHost>
```

27.3.6 HABILITACIÓN DEL NUEVO VIRTUAL HOST

En Debian y derivados se habilita mediante el siguiente comando:

```
a2ensite nuevohost.conf
```

Solo se debe especificar el nombre del archivo de configuración del *Virtual Host*.

En Fedora y derivados se puede crear un enlace simbólico al archivo, dentro de la carpeta *sites-enabled*:

```
ln -s /etc/apache2/sites-available/nuevohost.conf \
/etc/apache2/sites-enabled/nuevohost.conf
```

Este enlace simbólico es el que crea el comando *a2ensite* en Debian. Por lo tanto, también podría realizarse el mismo procedimiento en dicha distribución y sus derivados.

27.3.7 REINICIAR APACHE

Finalmente, para que los cambios se hagan efectivos, se debe reiniciar Apache (normalmente, solo bastaría con volver a cargar los archivos de configuración, sin embargo se reinicia para no tener que comprobar errores de configuración en un paso adicional):

En Debian y derivados:

```
systemctl restart apache2
```

En Fedora y derivados:

```
systemctl restart httpd.service
```

27.3.8 PROBANDO LA NUEVA WEB

Una vez montado el *Virtual Host*, ya puede crearse el primer *hola mundo* en un *script* dentro de la carpeta raíz de la aplicación, recordando colocar las líneas correspondientes al envío de cabeceras HTTP:

Archivo: `hola_mundo.py`

```
#!/usr/bin/env python
print("Content-type: text/html; charset=utf-8")
print("")
print("Hola Mundo")
```

Para probar, se puede ingresar utilizando un navegador Web en el mismo equipo y colocando en la barra de direcciones

`http://nuevohost.local/hola_mundo.py`.

Recordar que este archivo es un *script* como cualquier otro y por lo tanto, debe tener permisos de ejecución.

27.4 SEPARAR EL HTML DEL CÓDIGO PYTHON

A fin de permitir la independencia de las vistas (interfaces gráficas), es posible utilizar archivos HTML como plantillas, por un lado, y capas lógicas por el otro. Solo se requiere reservar en el HTML, los identificadores necesarios para ser sustituidos por valores variables. Por ejemplo, el siguiente es un archivo HTML que solo muestra el saludo «Hola Eugenia»:

```
<!doctype html>
<html>
  <head>
    <title>Archivo de prueba</title>
```

```
<meta charset="utf-8">
</head>
<body>
  <h1>Hola Eugenia</h1>
</body>
</html>
```

Si aquí «Eugenia» fuera el valor variable, se lo debería reemplazar por un identificador.

Un **identificador** es cualquier nombre descriptivo antecedido por el signo dólar.

```
<!doctype html>
<html>
  <head>
    <title>Archivo de prueba</title>
    <meta charset="utf-8">
  </head>
  <body>
    <h1>Hola $nombre</h1>
  </body>
</html>
```

El *script* CGI solo deberá leer el archivo, hacer lo suyo, y a continuación, sustituir los identificadores mediante la clase **Template** de Python. Para ello, se requiere definir un diccionario donde las claves sean los nombres de los identificadores y como valor asociado a cada clave, el de sustitución correspondiente:

```
from string import Template

with open('/path/to/archivo.html', 'r') as f:
    html = f.read()

diccionario = dict(nombre='Eugenia')
# 'Eugenia' podría venir desde un formulario

html = Template(html).safe_substitute(diccionario)
```

Explicación del código anterior:

- El diccionario debe tener una clave por cada identificador. Si el HTML tuviese más identificadores, el diccionario tendría más claves.
- El archivo se leyó en una variable que luego se “pisó” con la sustitución. Esto es útil si no se necesita conservar la plantilla original para usos futuros dentro del *script*.
- La sustitución se realiza mediante la clase *Template*. Esta clase, como parámetro, recibe la cadena que contiene los identificadores a ser sustituidos. A continuación, se invoca al método **safe_substitute**, que recibe como parámetro un diccionario cuyas claves son los nombres de identificadores de la cadena que se pasa a la clase *Template*. Si el número de identificadores de la cadena es distinto al número de claves del diccionario, ningún error será lanzado.

27.5 ENVÍO DE CORREO ELECTRÓNICO

Para el envío de correo electrónico, Python provee una biblioteca llamada **smtpLib**. Para hacer uso de ella (o de cualquier otra biblioteca para envío de correo electrónico) el equipo desde el cual se enviarán los correos debe tener instalado un MTA (*Mail Transport Agent*). Algunos MTA conocidos son *Sendmail* y *POSTFIX*. Se instalará *sendmail* por no requerir configuración en un entorno local.

Para instalar *Sendmail* en Debian y derivados, ejecutar:

```
apt install sendmail
```

En Fedora y derivados:

```
dnf install sendmail
```

El MTA no es requisito de la biblioteca de *smtplib* de Python, sino requisito para el envío de correos electrónicos desde un equipo a otro.

El mensaje de correo electrónico debe ser armado manualmente para cumplir con el formato establecido en las en las [RFC 5322](#)¹⁴.

Dicho formato presenta las siguientes secciones:

```
<cabecera>
[línea en blanco]
<mensaje>
```

El **mensaje** será cualquier mensaje de correo electrónico que se quiera enviar, incluyendo o no formato HTML. Cuando lo incluye, se define el tipo MIME como **text/html** y cuando no, como **text/plain**.

La **cabecera** será un conjunto de líneas formadas, cada una de ellas, por el patrón:

```
<campo de cabecera>: <valor del campo>
```

Las RFC 5322 definen la lista completa de campos de cabecera y sus especificaciones correspondientes.

Los campos de cabecera que habitualmente tendrá un mensaje de correo electrónico serán los siguientes:

Campo de cabecera	Nombre	Valores posibles (ejemplo)
Remitente*	From	usuario@host.tld

14 <https://tools.ietf.org/html/rfc5322>

Destinatario*	To	<u>usuario@host.tld</u>
Versión MIME*	MIME-Version	1.0
Tipo MIME*	Content-Type	text/plain
Dirección de respuesta	Reply-To	usuario@host.tld
Dirección con copia	Cc	usuario@host.tld
Direcciones con copia oculta	Cco	usuario@host.tld
Asunto del correo*	Subject	Cadena de texto

(*) Los cuatro primeros campos deben estar presentes en todo mensaje de correo. El último, se espera que esté presente.

Un ejemplo de mensaje de correo en texto plano podría verse como el siguiente:

```
From: usuario1@host1.tld
To: usuario2@host2.tld
MIME-Version: 1.0
Content-Type: text/plain
Subject: Mensaje de prueba
```

Este es el cuerpo del mensaje de correo electrónico que será enviado desde usuario1 a usuario2.

Una vez formado el mensaje de correo electrónico, se enviará creando una instancia de la clase **smtplib.SMTP**, cuyo constructor recibirá como parámetro el *host* del servidor SMTP (a no ser que se requiera el uso de un servidor externo, este será **localhost**), y enviar el mensaje de correo mediante el método **sendmail** el que recibirá como parámetros, el remitente, el destinatario y el mensaje de correo.

```
from smtplib import SMTP

remitente = 'usuario1@host1.tld'
destinatario = 'usuario2@host2.tld'

mensaje = """From: {desde}
To: {hacia}
```

```
MIME-Version: 1.0
Content-Type: text/plain
Subject: Mensaje de prueba
```

Este es el cuerpo del mensaje de correo electrónico que será enviado desde usuario1 a usuario2.

```
"".format(desde=remiteinte, hacia=destinatario)
```

```
smtp = SMTP('localhost')
smtp.sendmail(remiteinte, destinatario, mensaje)
```

El mismo mensaje con formato HTML, solo requiere modificar el valor del campo de cabecera **MIME-Type** y dar formato con HTML al mensaje, en vez de dejarlo en texto plano:

```
from smtplib import SMTP
```

```
remiteinte = 'usuario1@host1.tld'
destinatario = 'usuario2@host2.tld'
```

```
mensaje = """From: {desde}
To: {hacia}
MIME-Version: 1.0
Content-Type: text/html
Subject: Mensaje de prueba
```

Este es el cuerpo del mensaje de correo electrónico
que será enviado desde usuario1 a usuario2.

```
"".format(desde=remiteinte, hacia=destinatario)
```

```
smtp = SMTP('localhost')
smtp.sendmail(remiteinte, destinatario, mensaje)
```

27.6 MÉTODOS GET Y POST DE HTTP

POST y GET son dos de los ocho métodos de solicitud (*request methods*) definidos en la versión 1.1 del protocolo HTTP. Una especificación completa puede obtenerse en la sección 4 de las RFC 7231¹⁵.

15 <https://tools.ietf.org/html/rfc7231#section-4>

En el desarrollo de aplicaciones Web, POST y GET son los métodos más habituales de recupero de información. En el desarrollo de APIs, le siguen PUT, PATCH y DELETE.

27.6.1 RECEPCIÓN Y ANÁLISIS DE SOLICITUDES GET

GET es el método principal por el que se envían la mayor parte de solicitudes en HTTP. Los datos enviados por GET se pasan como cadena de consulta (*query string*) en la URI. El diccionario **environ** del módulo **os** permite recuperar tanto la URI como los datos enviados como parte de esta:

```
from os import environ

uri = environ['REQUEST_URI']
get_data = environ['QUERY_STRING']
```

La URI completa puede analizarse empleando el módulo **parse**¹⁶ de la biblioteca **urllib**. El método **urlparse** de este módulo, devolverá la URL dividida en seis partes. Esto es útil cuando la URL que se desea analizar es una URL foránea, distinta a la disponible en **environ**:

```
from urllib.parse import urlparse

url = "http://www.eugeniahahit.co.uk/search?q=una+cadena"
print(urlparse(url))
```

La salida de lo anterior se verá así:

```
ParseResult(
  scheme='http',
  netloc='www.eugeniahahit.co.uk',
  path='/search',
  params='',
  query='q=una+cadena',
  fragment='')
```

16 <https://docs.python.org/3/library/urllib.parse.html>

Cada una de las partes se encuentra disponible en un atributo del objeto **ParseResult** retornado por **urlparse**:

```
result = urlparse(url)
print(result.scheme) # Salida: http
```

Este mismo módulo permite **analizar los datos recibidos por GET**, mediante la función **parse_qs** (*qs* son las iniciales de *Query String*):

```
from os import environ
from urllib.parse import parse_qs

qs = environ['QUERY_STRING']
params = parse_qs(qs)
```

Esta función retorna los parámetros recibidos como claves de un diccionario. Dado que un mismo parámetro puede tener una colección de valores, los valores de cada parámetro serán almacenados en una lista, incluso aunque sea solo uno:

```
print(qs) # Salida: nombre=Eugenia&apellido=Bahit
print(params['nombre']) # Salida: ['Eugenia']

print(qs) # Salida: nombre=Eugenia&nombre=Juan
print(params['nombre']) # Salida: ['Eugenia', 'Juan']
```

Notar que los **datos enviados por GET** son recibidos en una cadena de consulta (*Query String*) cuyo formato sigue el patrón **<clave>=<valor>**. Cuando más de un par clave/valor son transmitidos, se concatenan con el símbolo *ampersand* (&).

El método GET es útil para transmisión y recepción de **datos no sensibles** por HTTP. Para datos sensibles, puede emplearse el método POST.

27.6.2 RECEPCIÓN Y ANÁLISIS DE SOLICITUDES POST

Los datos enviados a través del método POST pueden ser recibidos en Python, utilizando la clase **FieldStorage** del módulo **cgi**:

```
from cgi import FieldStorage

form = FieldStorage()
campo = form['nombre_del_campo'].value
```

27.6.3 CARGA DE ARCHIVOS CON PYTHON

Cuando lo que se desea enviar y recibir son archivos, debe tenerse la precaución de definir el tipo MIME del formulario como **multipart/form-data**:

```
<form action='/upload.py' enctype='multipart/form-data'
  method='POST' id='uploader'>
  <input type='file' name='foto'>
  <button type='submit'>Enviar</button>
</form>
```

Los archivos serán recibidos como cualquier otro campo enviado mediante POST, mediante **FieldStorage** y podrán guardarse mediante la estructura **with open**:

```
from cgi import FieldStorage

form = FieldStorage()
foto = form['foto'].value

with open('/path/to/uploaded_files/foto.png', 'wb') as f:
    f.write(foto)
```

27.6.3.1 CONSIDERACIONES SOBRE LA SEGURIDAD

Al cargar archivos desde una interfaz Web, el directorio donde se vaya a crear el archivo debe tener permisos de escritura (777). No es aconsejable que un directorio en el que puedan escribir los usuarios sea accesible por HTTP (ni mucho menos, propiedad del usuario de Apache que equivale a que cualquier usuario de la aplicación sea el propietario), ya que implicaría que se tuviese acceso a archivos subidos con propósitos diferentes a los previstos. Este problema de seguridad, plantea entonces la necesidad de tener un directorio con permisos de escritura pero con acceso restringido a través de HTTP. Es decir, se necesita un directorio que no esté servido.

La **dificultad que plantea tener un directorio no servido**, es que no se tendrá acceso a los archivos desde HTML. Por ejemplo, si el directorio se utiliza para crear imágenes que luego deben ser mostradas o enlazadas desde archivos HTML, no se podría resolver sin servir el directorio.

Por lo tanto, las alternativas se reducen a:

1. Servir el archivo con Python (requiere leer el archivo, obtener su tipo MIME y enviarlo a Apache modificando las cabeceras).
2. Codificar el archivo con Base 64 para servirlo como parte del esquema de URL "data" (se explica más adelante).

La primera alternativa era relativamente simple de llevar a cabo hasta Python 2. Sin embargo, en Python 3, su complejidad la hace menos viable que la segunda alternativa. Por lo tanto, se elegirá la segunda opción.

27.6.3.2 SERVIR ARCHIVOS ESTÁTICOS CON PYTHON

Para entender mejor lo que aquí se propone, puede ser útil responder primero a la pregunta «¿Qué significa servir un archivo?».

Cuando un archivo HTML es visualizado en un navegador Web y este contiene imágenes, hojas de estilo en cascada, bibliotecas de funciones JavaScript y otro tipo de formatos embebidos, por cada uno de ellos se está realizando una solicitud al servidor HTTP. Es decir, que cada uno de ellos se procesa en realidad, de forma independiente.

Por ejemplo, se puede pensar en un archivo HTML (*archivo.html*) que enlaza una hoja de estilos en cascada (*estilos.css*) y como parte de su diseño, contiene una imagen PNG (*imagen.png*). Si este archivo se hospedase en la raíz del dominio *eugeniabahit.co.uk*, por cada usuario que ingresase a *eugeniabahit.co.uk/archivo.html* se producirían en realidad, tres solicitudes al servidor HTTP de Apache:

eugeniabahit.co.uk/archivo.html
eugeniabahit.co.uk/estilos.css
eugeniabahit.co.uk/imagen.png

Cuando cada una de esas solicitudes se produce, el servidor HTTP de Apache (o cualquier otro servidor HTTP), enviará diferentes cabeceras al navegador Web. Esto implica que en cada caso, el tipo MIME será diferente:

- **text/html** para el caso de **archivo.html**
- **text/css** para el caso de **estilos.css**
- **image/png** para el caso de **imagen.png**

Algo que puede parecer muy evidente, en ocasiones no se tiene en cuenta a la hora de servir archivos con Python o con cualquier otro lenguaje.

Cuando se trabaja con *scripts* CGI, las solicitudes no se hacen a */archivo.html* sino al *script* CGI (*script.py*). Este *script*, además de realizar unas operaciones propias, se encarga de mostrar el contenido HTML (que puede estar en un archivo HTML o embebido en el propio *script*).

Si se considera que:

- a) aquello que sería una solicitud a */archivo.html* pasará a ser una solicitud a */script.py*;
- b) que el *script* CGI será quien muestre el contenido HTML; y
- c) que el HTML requiere mostrar una imagen (*imagen.png*) no servida,

la pregunta que se suscita entonces es ¿qué sucede si la imagen a mostrar no está servida? La respuesta inmediata es que un segundo *script* CGI (*script_imagen_no_servida.py*) es requerido para procesar y servir dicha imagen.

Esto implicará, por lo tanto, dos solicitudes: una a */script.py* y otra a */script_imagen_no_servida.py*. Por esta razón, el HTML ya no deberá enlazar la fuente de la imagen (atributo *src*) a *imagen.png* sino al segundo *script* CGI, como se muestra a en la comparación siguiente:

```
  

```

La forma habitual de resolver esto, incluso en Python hasta la versión 2.7 del lenguaje, es haciendo que el segundo *script* ejecute cuatro instrucciones:

1. Leer la imagen en modo binario.
2. Obtener el tipo MIME de la imagen.
3. Enviar las cabeceras HTTP con el tipo MIME de la imagen.
4. Enviar la imagen leída en el cuerpo del mensaje.

Esto se podía hacer con la simplicidad relativa con la que se hace habitualmente en otros lenguajes. Con la incorporación de los objetos de tipo *byte* en Python 3, no se puede realizar con la misma simplicidad (no bastan cuatro instrucciones) por lo que se necesita una forma alternativa.

Una opción viable (pero con limitaciones de longitud según el navegador) es emplear el esquema *data* de URL definido en las [RFC 2397](https://tools.ietf.org/html/rfc2397)¹⁷. Mediante este esquema es posible codificar un mensaje en Base 64 y transmitirlo como parte del esquema de la URL. Esto tiene una limitación de longitud determinada por cada navegador pero como solución inicial es viable para servir archivos estáticos en Python 3.

El **esquema *data* de URL** presenta el siguiente formato:

```
data:[<mediatype>][;base64],<data>
```

Recordando el formato BNF: los corchetes indican que el dato es opcional y los signos menor qué y mayor qué se emplean para representar datos variables.

Los dos primeros parámetros, opcionales, corresponden al tipo MIME y a la especificación de que los datos se encuentran codificados con Base 64, respectivamente. El valor final, es el objeto codificado en Base 64.

17 <https://tools.ietf.org/html/rfc2397>

El siguiente es un ejemplo sencillo y fácil de replicar manualmente. Se debe tener la precaución de respetar mayúsculas y minúsculas al escribir la siguiente línea en la barra de direcciones del navegador:

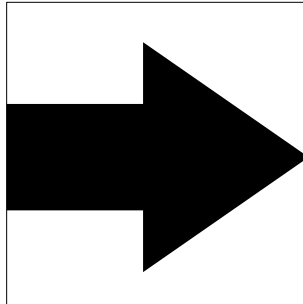
```
data:text/plain;base64,aG9sYQ==
```

Un ejemplo algo más complejo pero a la vez realista, podría ser el siguiente:

```
data:image/
png;base64,iVBORw0KGgoAAAANSUHEugAAAJcAAACXCAMAAAvQTlLAAAAZlBMV
EX///8BAQEAAAChoAHR0dG1tbXg4OD7+/scHBy4uLj39/
fX19fu7u7y8vLd3d2mpqZiYmIJCQno60iRkZEyMjI/
Pz98fHzBwcFYWFhJSUkpKS12dnbLy8tpawLRUVEWFhaZmZmGhoYtF+8ZAAADHULE
QVR4n02ba1PqMBCGy2KVix7o4Q6i+P//5MkFS5umzY5C3p0z+
+AHBnHm5Skm22RTFIqiKIqiKIqIKMp/x/
sTOKGcklZv6AwXSqlD0zpEBJOLaDtDx+hgco2I1lN0jhDrywSj0ys6SRvry0Bule
goLbwv+6D9X3SYBldfThktXtBxam6+bLD9H3Seb26+vLKLKCGj6cs508iYmFq+vL
IvCUNG6MsGW+/Qqbq+vLITfMjo+hqJmJgivrzy/
TM6V8SxvZbHMTXzJf7liELsz5fzlllgU1Mvb68swlqlL3yZR/
VCZar35dXBinMUR7szxfgjlnly/9nTrN//
xm+XGGWwXnD1lc2zLuYTTm+3JMle2Ia/
57FlpvLP0WuGNBdYFzG74vJrGUHP2jy4//gD4i2nFqW/
UnvBtGGUctm9+VeOCdr2fy+nLJjamKC+GLc/
kJ80WwThUBf7uXNXJ4vp6xaSfTlbn/
7RlmgR5FfMYjP5VBf7peT6CiL9eWVxSYmuC8brCo7tSxal8Uk04eFmQBf7i3VYib
0l8VMTHOBvuy1pNVMni+/yLKT58s7q9dl0ZbamMJsJ9CXe/PyTZ4vP/y/
yPNV9hoPy0ae5iifNXfekm+qLVIIMaXKfebtSva0hUrqiXrCPFFm+B2Ei3KYnef
w7pQgC8zxHfvvdGu7L5b7MYb7ov00eUwtKy+tTCor4G1Q6Srou1AmC8axpvG2YoM
Dnhfyb0PjCtaphZ+Ab4M3fUIuC/i7UXear+G74smrHaPO+Xiy9pyUhXF0+
+ZXxLGGt+srL0BU54w20uRMRVzH3kUFPScgF6dvj9AB/
5e5sYviC9Y0l+E0zv3LAV3v7sg3INjqSo3szB/q/
DwGbe43P1yroAT1j09xfy+0seLcsua4ztLe/pX92je5Hj/
b7ortp4fzRscGjnCmQl2zayEPqqmDXpownOK+Qus3ppne/
4lHKkouHL5PuQIqu4+Ur130SmPm91FnNEX3E9n7YwMTg0c0CMBZznCLG5jgLOv4S
U/
LbUrJRCzleFPIsZSRVFURRRFURRRFBP8A4vg01XWKWT8AAAAASUVORK5CYII=
```

Si lo anterior se colocase en la barra de direcciones de un navegador como cualquier esquema de URL habitual, se podría visualizar una imagen cuadrada con fondo blanco y una flecha negra cuya punta se orienta hacia la derecha, similar a la que se muestra a continuación

(tener en cuenta que el borde de la imagen solo se presenta a modo orientativo para señalar los límites de la misma):



Como *data* es un esquema de URL similar a **http** o a **file**, puede emplearse para sustituir la llamada a cualquier esquema URL de cualquier tipo de archivo estático.

Para emplear este esquema se requieren dos operaciones:

- Obtener el tipo MIME del archivo que se desea servir.
- Obtener la codificación Base 64 de dicho archivo.

Ambas operaciones se describen a continuación.

27.6.3.3 OBTENER EL TIPO MIME DE UN ARCHIVO

Si bien Python ofrece un módulo para trabajar con los tipos MIME llamado *mimetypes*¹⁸, la forma más segura de obtener el tipo MIME real de un archivo es mediante la biblioteca **file** del sistema operativo, ya que *mimetypes* solo intenta "adivinar" el tipo MIME utilizando la extensión del archivo. Esto supone un riesgo de seguridad significativo, puesto que la extensión de un archivo es solo parte de su nombre pero no

18 <https://docs.python.org/3/library/mimetypes.html>

necesariamente una representación de su tipo MIME como se demuestra a continuación:

```
..$ ls private/  
-rw-r--r-- 1 www-data www-data 210 ago  6 18:32 foto.png  
..$ file private/foto.png --mime-type  
private/foto.png: image/png  
..$ cp private/foto.png private/foto.txt  
..$ file private/foto.txt --mime-type  
private/foto.png: image/png
```

Aquí se ve claramente que el archivo *foto.png*, incluso aunque se copie como *foto.txt*, es de tipo *image/png*. Sin embargo, *mimetypes* responderá que *foto.txt* es **text/plain**:

```
>>> from mimetypes import guess_type  
>>> guess_type('foto.txt')  
('text/plain', None)
```

Demostrada la inexactitud, se determina entonces que la forma más segura de obtener el tipo MIME de forma precisa, es a través del comando **file**, el cual se puede ejecutar desde **Popen** como se muestra a continuación:

```
from subprocess import Popen, PIPE  
  
img = '/path/to/uploaded_files/una_imagen'  
commands = ['file', '--mime-type', img]  
process = Popen(commands, stdout=PIPE)  
mime = process.stdout.read().decode('utf-8')  
mime = mime.split(': ')[1].replace('\n', '')
```

27.6.3.4 CODIFICAR UN ARCHIVO EN BASE 64

Para codificar un archivo en Base 64 se empleará la biblioteca *base64* de Python. Dado que el valor retornado será de tipo *bytes*, se decodificará su resultado en UTF-8:

```
from base64 import b64encode
```

```
with open(img, "rb") as f:
    contenido = f.read()

contenido = b64encode(contenido).decode('utf-8')
```

A partir de aquí, el esquema *data* de URL se podrá utilizar como valor de los atributos *src* del objeto *img* de HTML, o *href* del objeto *a*, completándolo con los valores *mime* y *contenido* obtenidos en los pasos previos.

El siguiente ejemplo se utiliza el esquema *data* para mostrar una imagen:

```

```

En este otro ejemplo, las variables se utilizan para crear un enlace a un archivo PDF:

```
<a href="data: $mime; base64,$contenido">Ver PDF</a>
```

En los ejemplos precedentes, las variables *\$mime* y *\$contenido* serán reemplazados mediante `Template.safe_substitute`:

```
diccionario = dict(mime=mime, contenido=contenido)
html = Template(html).safe_substitute(diccionario)
```

28. CONSUMO Y PRODUCCIÓN DE APIs CON ARQUITECTURAS REST/JSON

Una **API** (*Application Programming Interface*, o Interfaz de programación de aplicaciones) es un programa informático empleado para permitir la interacción entre dos programas independientes. Se denomina interfaz ya que actúa como intermediaria para que un programa pueda acceder a otro sin hacer un uso directo de este y por consiguiente, tampoco de sus

datos. En el mundo actual, las APIs son implementadas en todo tipo de sistemas informáticos. Un uso tecnológico habitual y bien conocido de este tipo de interfaces de aplicación, es el que las redes sociales ofrecen a otros programas para facilitar la función de autenticación de usuarios.

Por sus siglas en inglés, **REST** significa *Representational State Transfer* (transferencia de estado representacional). Es un estilo arquitectónico presentado en el año 2000¹⁹ por el Dr. R. T. Fielding de la Universidad de California, y confundador de la Apache Software Foundation. Fielding define a REST como un *estilo arquitectónico híbrido para sistemas hipermedia distribuidos*.

28.1 EL FORMATO JSON

JSON es el formato de intercambio de datos utilizado por APIs con arquitecturas REST. Por sus siglas en inglés, JSON significa *JavaScript Object Notation* (notación de objetos JavaScript). Es un conjunto de reglas de formato de texto para el intercambio de datos estructurados, e independiente del lenguaje de programación.

Brevemente, el formato propuesto por JSON es algo similar a los diccionarios de Python, y la biblioteca **json** de Python —entre otras cosas— se encarga de codificar diccionarios Python en formato JSON, y decodificar datos en formato JSON como diccionarios Python.

Un conjunto de datos en formato JSON —denominado Objeto JSON—, se describe como un diccionario formato por claves y valores:

```
/* Código JavaScript */  
var libros = {  
    "Python para principiantes": {  
        "ISBN": 9781838190101,  
        "Edición": 2,  
        "Páginas": 178,  
        "Autora": "Eugenia Bahit"
```

19 https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

```
    },  
    "Python Aplicado": {  
        "ISBN": 9781838190125,  
        "Edición": 3,  
        "Páginas": 202,  
        "Autora": "Eugenia Bahit"  
    }  
};
```

En el ejemplo anterior, el valor de la variable *libros* es un objeto JSON.

Al recibir un objeto JSON en Python, o en cualquier otro lenguaje, inicialmente se recibe como una cadena de texto. Debe ser convertida, entonces, a un formato interpretable por el lenguaje. Las funciones **loads** y **dumps** de la biblioteca **json** de Python, decodifican y codifican el formato JSON respectivamente:

```
# Código Python  
from json import loads  
libros_json = """"{  
    "Python para principiantes": {  
        "ISBN": 9781838190101,  
        "Edición": 2,  
        "Páginas": 178,  
        "Autora": "Eugenia Bahit"  
    },  
    "Python Aplicado": {  
        "ISBN": 9781838190125,  
        "Edición": 3,  
        "Páginas": 202,  
        "Autora": "Eugenia Bahit"  
    }  
}""""  
libros_dict = loads(libros_json)
```

Si se verifica la salida se comprobará que la cadena con formato JSON ha sido convertida a diccionario:

```
>>> type(libros_dict)  
<class 'dict'>  
>>> print(libros_dict)  
{'Python para principiantes': {'ISBN': 9781838190101, 'Edición':  
2, 'Páginas': 178, 'Autora': 'Eugenia Bahit'}, 'Python
```

```
Aplicado': {'ISBN': 9781838190125, 'Edición': 3, 'Páginas': 202,
'Autora': 'Eugenia Bahit'}}
>>> libros_dict['Python Aplicado']['Autora']
'Eugenia Bahit'
```

De igual forma, un diccionario Python puede convertirse a JSON:

```
# Código Python
from json import dumps
libros_dict = {
    'Python para principiantes': {
        'ISBN': 9781838190101,
        'Edición': 2,
        'Páginas': 178,
        'Autora': 'Eugenia Bahit'
    },
    'Python Aplicado': {
        'ISBN': 9781838190125,
        'Edición': 3,
        'Páginas': 202,
        'Autora': 'Eugenia Bahit'
    }
}

libros_dict = dumps(libros_dict)
```

Igual que en el caso anterior, se comprueba la salida para verificar que el diccionario se alla convertido:

```
>>> libros_dict = dumps(libros_dict)
>>> libros_dict
'{"Python para principiantes": {"ISBN": 9781838190101, "Edici\\
u00f3n": 2, "P\\u00e1ginas": 178, "Autora": "Eugenia Bahit"},
"Python Aplicado": {"ISBN": 9781838190125, "Edici\\u00f3n": 3,
"P\\u00e1ginas": 202, "Autora": "Eugenia Bahit"}}'
>>> type(libros_dict)
<class 'str'>
```

28.2 EL PROTOCOLO HTTP

Para entender cómo consumir y producir interfaces de programación de aplicaciones, es necesario comprender que la base de una API está en el conocimiento del protocolo HTTP. Si se entiende el protocolo, para

consumir una API solo resta emplear una biblioteca que permita realizar solicitudes HTTP, y para producirla, bastará con crear recursos que en vez de vistas, retornen datos, y que por otra parte, constaten que la solicitud HTTP haya sido correcta.

El **protocolo HTTP** está basado en una **arquitectura cliente-servidor**. Esta arquitectura plantea dos componentes (programas) conectados sobre una red informática:

- un *cliente* que consume los recursos provistos por un servidor;
- y un *servidor*, que provee de recursos a los clientes.

Los clientes solicitan al servidor un recurso, el servidor acepta la solicitud, y retorna al cliente los resultados de la ejecución.

Los recursos provistos por el servidor (funcionalidades que el servidor puede llevar a cabo), son ofrecidos mediante un *identificador uniforme de recurso* (URI), también conocido como *localizador uniforme de recursos* (URL).

En lo que a HTTP concierne, los identificadores uniformes de recursos, es decir, los URI o URL, son cadenas de texto simple que permiten identificar mediante un nombre o localización, un recurso específico (lo que a los ojos del software, sería una funcionalidad concreta o un paso en una funcionalidad).

Un URI HTTP puede indicarse en uno de dos formatos: absoluto o relativo. Es absoluto cuando incluye el esquema “*http*” seguido de dos puntos, y es relativo cuando provee el nombre del recurso sobre una base conocida (host). El esquema “*http*” se utiliza para localizar recursos bajo el protocolo homónimo (HTTP) empleando el siguiente formato:

`http://<host>[:<puerto>][<ruta absoluta del recurso>][?<cadena de consulta>]`

Un ejemplo completo podría verse como el siguiente:

```
http://www.elidabahit.co.uk:8080/ruta/absoluta?  
consulta=una+cadena+cualquiera
```

Cuando el puerto no es indicado en el URI, por defecto, el protocolo entiende que el acceso será a través del puerto 80.

Las solicitudes HTTP se envían en una sola línea sin saltos (excepto al final). Una solicitud HTTP presenta el siguiente formato:

<método><SP><URI><SP><HTTP-Version><CRLF>

En la línea anterior, **<SP>** significa espacio en blanco y **<CRLF>**, salto de línea.

Estas solicitudes HTTP se envían empleando un método específico. El protocolo HTTP ofrece diferentes métodos de solicitud. Actualmente, la versión HTTP/1.1 define ocho métodos. Las denominaciones así como su descripción, se presentan en la siguiente tabla:

Método	Descripción
GET	Obtiene cualquier tipo de información
HEAD	Igual a GET pero el servidor no debe retornar una respuesta en el cuerpo del mensaje.
POST	Mientras que GET se recupera mediante la URI, POST solicita al servidor aceptar la información encapsulada como una entidad dentro de la propia solicitud
PUT	Mientras que POST se limita a enviar la información encapsulada, PUT agrega a la solicitud, que la información enviada (encapsulada como en POST) sea almacenada en el recurso que se está solicitando. Si el recurso solicitado existe, HTTP interpreta que debe ser modificado con la nueva entidad recibida. Si no existe, podría crear la nueva entidad (siempre y cuando el recurso tenga la capacidad de hacerlo).
DELETE	Solicita al servidor eliminar el recurso identificado en el URI.

Otros dos métodos son **TRACE** y **CONNECT** definidos en las secciones 9.8 y 9.9 de las RFC 2616, respectivamente.

Tras recibir la solicitud, y dependiendo tanto del método empleado por el cliente para efectuar la solicitud, como del resultado obtenido por el servidor al llevarla a cabo, este responderá al cliente con un mensaje compuesto de una línea de estado.

La línea de estado estará formada por la versión del protocolo, un código de estado de 3 dígitos, y su correspondiente explicación en una cadena de texto.

Existen cinco clases de respuesta que pueden ser identificadas por el primer número del código de estado. Las clases de respuesta se describen en la siguiente tabla:

Clase	Tipo	Descripción
1xx	Información	la solicitud ha sido recibida y se continúa el proceso
2xx	Éxito	la solicitud fue recibida y ejecutada con éxito
3xx	Movida	la solicitud requiere de acciones adicionales para ser completada
4xx	Error de cliente	la solicitud o bien estaba incompleta, o bien, contenía errores de formato
5xx	Error de servidor	la solicitud no pudo completarse debido a un error del servidor

Las secciones 10.1 a 10.5 de las RFC 2616, definen la lista completa de los códigos de estado para las cinco clases de respuesta. Algunos de estos estados se describen en la siguiente tabla:

240

Eugenia Bahit. Python Aplicado.

Código		Descripción
200	Ok	La solicitud ha sido exitosa
301	Moved Permanently	El recurso solicitado ha sido movido de forma permanente y se le ha asignado un nuevo URI
304	Not Modified	Cuando se realiza una nueva solicitud mediante el método GET y el recurso no ha sufrido cambios desde la última consulta del mismo cliente, el servidor debería responder con este código y no con '200 Ok'.
307	Temporary Redirect	Redirección temporal a un nuevo recurso (con nuevo URI asignado, temporalmente)
400	Bad Request	La solicitud no ha sido entendida por el servidor
403	Forbidden	La solicitud ha sido rechazada por el servidor a pesar de haber sido entendida, y no debe ser reintentada
404	Not Found	El servidor no encontró el recurso solicitado
405	Method Not Allowed	El método de la solicitud no corresponde al método (o métodos) exigidos por el recurso
408	Request Time Out	La solicitud del cliente ha tomado más tiempo del que el servidor puede esperar
500	Internal Server Error	El servidor ha encontrado un problema interno y ha debido finalizar abruptamente la solicitud
503	Service Unavailable	El servidor no se encuentra disponible temporalmente para manejar la solicitud

De esta forma, una API definirá no solo el recurso a través del cual será accedida, sino además, el método, y del método dependerá la respuesta. No obstante, si se va a consumir una API, se deberá tener en cuenta que una mala implementación del protocolo HTTP (o una mala interpretación del protocolo), pueden generar especificaciones incoherentes en la API, las cuales habrá que seguir a fin de no obtener fallos.

Generalmente, cuando se trate de leer datos de acceso público (como un listado de productos, o un catálogo de cualquier tipo que sea accesible por cualquier persona), el método del recurso será GET, y las posibles respuestas, serán:

- 200, si todo ha ido bien, y en ese caso, la respuesta irá acompañada de un JSON con los datos solicitados;
- 400, si la solicitud es incomprensible; o,
- 405, si el método recibido no es GET.

Y, cuando se desee escribir, dependerá de si se pueden o no modificar datos existentes. Cuando los datos puedan ser modificados, se empleará PUT. Si solo pueden agregarse, se empleará POST. Y si se desea eliminar datos existentes, DELETE. Generalmente, las respuestas de estado serán las mismas que en el caso de GET, y en el caso de un código 200, el JSON contendrá una confirmación del dato agregado, modificado ,o eliminado, acompañado de un identificador único para el dato en cuestión (ID).

28.3 CONSUMO Y PRODUCCIÓN DE APIs

Para consumir una API se puede optar por uno de dos mecanismos posibles:

- Conectar mediante una herramienta del sistema operativo, como cURL, a través de la biblioteca **Popen** de Python.
- Conectar mediante la biblioteca **http** de Python. Para realizar la conexión se seguirán los pasos descritos en el Capítulo 21, a partir de la página 133.

En el caso de emplear cURL y **Popen**, la solicitud mediante cURL se hará considerando las especificaciones de cURL, dentro de una cadena de texto que será dividida luego con **shelx** o directamente en una lista, como es requerida por **Popen**. Para más información sobre el uso de Popen y shelx, dirigirse al capítulo 20.3 «Ejecución de comandos del sistema mediante Popen y shlex.split» en página 127.

El siguiente es un ejemplo rústico de producción de una API que acepta una solicitud mediante GET al URI `http://api-server.local/api-server.py`

Esta API acude a:

- `os.environ` para verificar el método de la solicitud
- `json.dumps` para convertir el diccionario con los datos de respuesta a un formato JSON válido.

Como puede observarse se trata de un código rústico, simple, que utiliza los recursos mínimos para demostrar la forma de implementar una API en el componente servidor. Si verifica que el método de la solicitud no ha sido GET, sobrescribe el estado y la salida JSON.

```
#!/usr/bin/env python3
from json import dumps
from os import environ

metodo = environ['REQUEST_METHOD']

estado = "200 Ok"
json = dumps(dict(listado=[1, 2, 3, 4]))

if not metodo == "GET":
    estado = "405 Method Not Allowed"
    json = "{}"

print("Content-type: application/json")
print("Status: {}".format(estado))
print("")
print(json)
```

Se demuestra a continuación, mediante un ejemplo rústico, cómo esta API podría ser consumida por un cliente implementando cURL y **Popen**.

```
#!/usr/bin/env python3
from subprocess import Popen, PIPE

comando = [
    "curl",
    "-i", # Incluir encabezados en la respuesta
```

```
    "http://api-server.local/api-server.py"  
]  
  
p = Popen(comando, stdout=PIPE)  
salida = p.stdout.read()  
  
print("Content-type: text/html; charset=utf-8")  
print("")  
print(salida)
```

Lo anterior genera como salida un objeto de tipo *Byte* con el contenido siguiente:

```
b'HTTP/1.1 200 Ok\r\nDate: Tue, 28 Sep 2021 01:24:24 GMT\r\nServer: Apache/2.4.38 (Debian)\r\nTransfer-Encoding: chunked\r\nContent-Type: application/json\r\n\r\n{"listado": [1, 2, 3, 4]}\r\n'
```

El mismo podría convertirse a string decodificando la salida:

```
salida = p.stdout.read().decode("utf-8")
```

Arrojando como salida:

```
HTTP/1.1 200 Ok  
Date: Tue, 28 Sep 2021 01:26:57 GMT  
Server: Apache/2.4.38 (Debian)  
Transfer-Encoding: chunked  
Content-Type: application/json  
  
{"listado": [1, 2, 3, 4]}
```

Misma, que podría ser fácilmente analizable:

```
from email.parser import Parser  
...  
  
salida = p.stdout.read().decode("utf-8").split("\r\n")  
estado = salida[0].split('HTTP/1.1 ')[-1] # 200 Ok  
codigo_estado = estado.split(" ")[0] # 200  
headers = "\n".join(salida[1:-2])  
headers = Parser().parsestr(headers) # Diccionario de encabezados  
json = salida[-1] # {"listado": [1, 2, 3, 4]}
```

Se presenta a continuación el ejemplo completo, un poco menos rústico que el anterior, incluyendo la conversión de JSON a diccionario:

```
#!/usr/bin/env python3
from email.parser import Parser
from json import loads
from subprocess import Popen, PIPE

comando = [
    "curl",
    "-i", # Incluir encabezados en la respuesta
    "http://api-server.local/api-server.py"
]

p = Popen(comando, stdout=PIPE)
salida = p.stdout.read().decode("utf-8").split("\r\n")
estado = salida[0].split('HTTP/1.1 ')[-1] # 200 OK
codigo_estado = estado.split(" ")[0] # 200
headers = "\n".join(salida[1:-2])
headers = Parser().parsestr(headers) # Diccionario de encabezados
json = salida[-1] # {"listado": [1, 2, 3, 4]}

print("Content-type: text/html; charset=utf-8")
print("")
print(codigo_estado)
print(headers['Content-type'])
print(loads(json))
```

El mismo ejemplo podría haber sido consumido con la biblioteca **http.client**:

```
#!/usr/bin/env python3
from http.client import HTTPConnection
from json import loads

http = HTTPConnection('api-server.local')
http.request('GET', '/api-server.py')

respuesta = http.getresponse()
codigo_estado = respuesta.status
json = respuesta.read().decode('utf-8')

print("Content-type: text/html; charset=utf-8")
print("")
print(codigo_estado)
print(respuesta.getheader('Content-type'))
print(loads(json))
```

Como se puede observar, incluso en una versión tan rústica como la presentada, el código empleando la biblioteca **http**, resulta más simple y legible. El consumo de recursos, probablemente sea menor o igual, puesto que internamente ejecutará pasos similares, solo que sin el intérprete como intermediario, por lo cual, la segunda implementación, parecería ser la más viable.

29. CONEXIONES A BASES DE DATOS CON MYSQL® Y MARIADB

Para trabajar con bases de datos, en todo lenguaje se deben reunir dos requisitos:

1. Un servidor de bases de datos (como MySQL®, MariaDB, PostgreSQL, entre otros).
2. Un conector de base de datos para el lenguaje, que es el que permitirá al lenguaje interactuar con la base de datos.

Tener en cuenta que **MariaDB es un fork de MySQL®**. Por lo tanto, si se tiene por costumbre utilizar MySQL®, probablemente no se noten diferencias utilizando MariaDB.

En el caso de MySQL® existe un conector desarrollado por el fabricante, denominado *MySQL Connector/Python*²⁰. Se trata de una biblioteca privativa, no libre. Una versión libre con licencia GPL v3.0 es la biblioteca *python-MySQLdb*²¹. Esta biblioteca sirve no solo para conexiones MySQL® sino también a MariaDB.

20 <https://dev.mysql.com/doc/connector-python/en/>

21 <https://mysqlclient.readthedocs.io/>

Recientemente en **Junio de 2020**, MariaDB lanzó su propio conector para Python, **MariaDB Connector/Python**. Hasta tanto transcurra el tiempo suficiente para considerarlo estable se recomienda utilizar MySQLdb. No obstante, se sugiere estudiarlo con la documentación oficial y mantenerse al día en cuanto a su avance:

<https://mariadb.com/resources/blog/how-to-connect-python-programs-to-mariadb/>

Los paquetes a instalar tanto para Debian como para Fedora y distribuciones derivadas de ambos, son los siguientes:

1. mariadb-server
2. python3-mysqldb

29.1 CONFIGURACIÓN DE MARIADB

Como usuario **root**, conectarse vía *Shell* a MariaDB:

```
..# mariadb -u root
```

El objetivo es crear un usuario nuevo con permisos para crear bases de datos, y que pueda conectarse desde los *scripts* CGI:

```
CREATE USER <nombre> IDENTIFIED BY '<clave>';  
GRANT ALL PRIVILEGES ON *.* TO <nombre>;
```

<nombre> y <clave> pueden ser cualquier cadena de texto que se desee, por ejemplo, el usuario *eugenia* con la clave *secret*:

```
CREATE USER eugenia IDENTIFIED BY 'secret';  
GRANT ALL PRIVILEGES ON *.* TO eugenia;
```

A partir de aquí, es posible entrar al administrador de MariaDB con el nuevo usuario (y sin necesidad de hacerlo con el usuario *root* del sistema), y crear la nueva base de datos. La conexión (ahora con clave) se realizará con la instrucción:

```
..$ mariadb -u eugenia -psecret
```

Y para crear una nueva base de datos se ejecutará:

```
CREATE DATABASE <nombre>;
```

Donde *<nombre>* será el que se le quiera dar a la nueva base de datos. Por ejemplo, *nueva_db*:

```
CREATE DATABASE nueva_db;
```

Algunas **sentencias SQL para gestionar las bases de datos**, y que podrían ser útiles de recordar, se describen en la siguiente tabla:

Objetivo	Sentencia
Crear base de datos	CREATE DATABASE <nombre>;
Ver todas las bases de datos	SHOW DATABASES;
Eliminar base de datos	DROP DATABASE <nombre>;
Elegir una base de datos	USE <nombre>
Crear una tabla con el motor InnoDB (para bases de datos relacionales)	CREATE TABLE <nombre> (<campo ₁ > <características ₁ >, <campo _n > <características _n >) ENGINE=InnoDB;
Ver todas las tablas de una base de datos	SHOW TABLES;
Ver la estructura de una tabla	DESCRIBE <tabla>;
Eliminar una tabla	DROP TABLE <nombre>;
Crear usuario	CREATE USER <nombre>

IDENTIFIED BY '<clave>';

La lista completa puede verse en la documentación oficial de MariaDB y en la de MySQL:

- MariaDB: <https://mariadb.com/kb/en/sql-statements/>
- MySQL: <https://dev.mysql.com/doc/refman/8.0/en/sql-statements.html>

29.2 TRABAJANDO CON MARIADB Y MYSQL® DESDE PYTHON

Teniendo el servidor de bases de datos y el conector correspondiente instalados, el acceso y consulta a bases de datos se realizará importando el conector:

```
from MySQLdb import connect
```

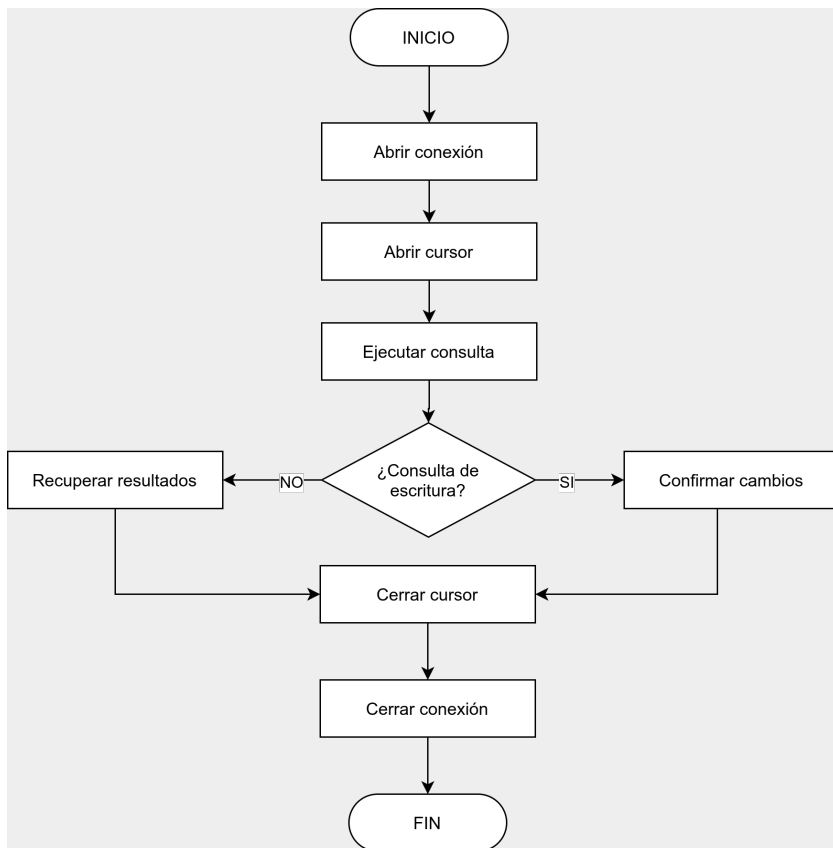
Para entender cómo funciona el acceso y consulta a una base de datos independientemente del servidor y del lenguaje, considerar que los pasos a seguir siempre serán los mismos:

1. **Abrir la conexión a la base de datos.** Esto implica: conectarse a una *host*, autenticarse con un usuario y contraseña en el servidor de base de datos, y elegir la base de datos sobre la cual trabajar. La apertura de la conexión, la autenticación y la selección de la base de datos, suelen ser tres operaciones para las que los conectores ofrecen una sola que los ejecuta. En algunos lenguajes, pueden llevarse a cabo en tres operaciones independientes.

Por otra parte, esta conexión abierta al comienzo es como un programa que se abre: necesita cerrarse al finalizar el trabajo.

2. **Abrir un cursor.** Un cursor es una operación que se abre dentro de la conexión establecida, con la finalidad de almacenar los resultados devueltos por una consulta. Al igual que en el caso anterior, todo aquello que se abre debe cerrarse al finalizar el proceso.
3. **Ejecutar una consulta.** Una consulta es la sentencia SQL que será ejecutada sobre la base de datos especificada.
4. **Confirmar cambios.** Esta operación se realiza cuando se ejecutan instrucciones de escritura como **INSERT**, **UPDATE** o **DELETE**, entre otras.
5. **Recuperar los resultados de una consulta.** Esta operación se realiza cuando se ejecutan consultas de lectura como **SELECT**, **SHOW** o **DESCRIBE**, entre otras.
6. **Cerrar el cursor** abierto.
7. **Cerrar la conexión** abierta.

El siguiente diagrama de flujo (*flowchart*) puede ayudar a memorizar y entender las operaciones necesarias para escribir el código fuente de funciones de manejo de bases de datos en cualquier lenguaje y para cualquier servidor:



1. Ilustración: Diagrama de flujo de operaciones necesarias para acceso y consulta a bases de datos, independiente del lenguaje y del servidor de bases de datos.

Cada una de estas operaciones tienen un nombre específico en Python, definido en las PEP 249²². Sus nombres se describen en la siguiente tabla.

22 <https://www.python.org/dev/peps/pep-0249>

Operación	Nombre en Python
Abrir conexión	<code>connect</code>
Abrir cursor	<code>cursor</code>
Ejecutar consulta	<code>execute</code>
Confirmar cambios	<code>commit</code>
Recuperar resultados (todos)	<code>fetchall</code>
Cerrar cursor	<code>(connect) close</code>
Cerrar conexión	<code>(cursor) close</code>

Cada una de estas operación será implementada según el conector elegido. En el caso de *MySQLdb*, la implementación se realiza como se indica a continuación.

Abrir una conexión: requiere los datos de conexión a la base de datos como *host*, usuario, contraseña y nombre de la base de datos.

```
conexion = connect(  
    host='localhost',  
    db='una_db',  
    user='un_usuario',  
    password='una_clave'  
)
```

Otras opciones como número de puerto, la tabla de codificación de caracteres a utilizar, entre otras, también están disponibles²³.

Abrir un cursor: no tiene especificaciones particulares. Se abre sobre la conexión.

```
cursor = conexion.cursor()
```

Ejecutar la consulta: requiere que se le pase como parámetro la sentencia SQL a ejecutar. Se ejecuta sobre el cursor. Opcionalmente, si

23 https://mysqlclient.readthedocs.io/user_guide.html#mysqlldb

la sentencia SQL recibirá datos externos, estos se pasarán como segundo parámetro. Pero esto se explica en el apartado de seguridad relativo a las inyecciones SQL.

```
cursor.execute(sql) # la variable sql es la sentencia a
ejecutar
```

La sentencia SQL podría haber sido cualquier sentencia de las mencionadas previamente, o las más habituales de lectura y escritura. Algunos ejemplos se dan a continuación:

Seleccionar los campos nombre y apellidos de la tabla personas:

```
sql = "SELECT nombre, apellidos FROM personas"
```

Agregar a Juan Pérez en la tabla personas:

```
sql = """
    INSERT INTO personas
            (nombre, apellidos)
    VALUES      ('Juan', 'Pérez')
    """
```

Modificar el apellido de la persona cuya ID es 15:

```
sql = """
    UPDATE      personas
    SET          apellidos = 'Pérez García'
    WHERE       persona_id = 15
    """
```

Eliminar de la tabla personas a la persona cuya ID es 15:

```
sql = "DELETE FROM personas WHERE persona_id = 15"
```

Confirmar los cambios: no tiene especificaciones particulares. Se confirman los cambios sobre la conexión (ya que la función del cursor solo es almacenar el resultado de las consultas).

```
conexion.commit()
```

Recuperar la ID del último registro sobre el que se confirmó una operación, suele ser útil cuando se está insertando un nuevo registro y su ID se genera de forma automática por la propia base de datos. En este caso, puede recuperarse a partir del cursor:

```
ultima_id = cursor.lastrowid
```

Recuperar todos los resultados: no tiene especificaciones particulares. Los resultados se recuperan a partir del cursor, en una tupla conformada internamente, por una tupla por cada registro encontrado.

```
resultados = cursor.fetchall()
```

La variable **resultados** será una tupla. Los elementos de esta tupla serán otras tuplas (una tupla por cada registro). Cada una de ellas corresponderá a un registro, y el valor de las columnas será cada uno de los elementos de esta tupla. Por ejemplo, para una tabla como esta:

tipodocumento_id	denominacion
1	Carta
2	Comunicado
3	Informe

La tupla de resultados se vería como la siguiente:

```
(  
    (1L, 'Carta'),
```

```
(2L, 'Comunicado'),  
(3L, 'Informe')  
)
```

Cerrar el cursor y cerrar la conexión: ninguna de las dos tiene especificaciones particulares.

```
cursor.close()  
conexion.close()
```

29.3 SEGURIDAD: PREVENCIÓN DE INYECCIONES SQL

Se denomina inyección SQL al ataque informático producido a una base de datos, mediante el cual se logra «inyectar» en una sentencia SQL, código SQL que altere la sentencia original.

Esto suele producirse cuando una sentencia SQL se completa con datos aportados por el usuario. Un ejemplo sencillo de entender es si se tiene una sentencia SQL como la siguiente:

```
sql = """  
    SELECT campo  
    FROM   tabla  
    WHERE  dato = '{}'  
    """.format(un_dato)
```

Esta sentencia se completa con el valor de una variable llamada **un_dato**, la cual almacena información aportada por el usuario. La sentencia podría estar esperando una cadena de texto genuina pero el usuario podría “cortar y extender” la sentencia con solo ingresar comillas simples. Si la variable **un_dato** tuviese un valor como este:

```
un_dato = "' or '='"
```

La sentencia SQL final se vería afectada de la siguiente forma:

```
SELECT campo FROM tabla WHERE dato = '' or ''=''
```

La condición `''=''` siempre será verdadera por lo que la consulta SQL siempre retornaría como resultados todos los registros de la tabla (dejará expuestos el total de los registros).

La forma más directa de **prevenir inyecciones SQL** es no emplear *format* para completar las sentencias SQL con datos externos, sino preparar la sentencia con modificadores de parámetro y pasar los datos variables como segundo parámetro de **execute**.

Una sentencia SQL se prepara con modificadores de parámetro identificados por el signo de porcentaje y una letra **s** (independientemente del tipo de datos).

En el nuevo conector de MariaDB, el modificador de parámetro utilizado es el signo de interrogación **?**.

Sentencia NO preparada:

```
sql = """
    SELECT      campo
    FROM        tabla
    WHERE       dato = '{} '
    """.format(un_dato)
```

Sentencia preparada:

```
sql = "SELECT campo FROM tabla WHERE dato = %s"
```

*NOTA: Con el nuevo conector de MariaDB para Python se escribiría un **?** en vez de **%s**.*

No se utilizan las comillas ya que el propio conector se encargará de ello.

Como segundo parámetro de la operación *execute*, se pasarán los datos externos dentro de una tupla:

```
conexion.execute(sql, (un_dato,))
```

De esta forma, el propio conector se encarga de limpiar los datos y prevenir que pueda inyectarse código fuente en las sentencias SQL.

29.4 FUNCIÓN PARA AUTOMATIZAR CONSULTAS SQL

La siguiente es una función sencilla que recibe tres parámetros:

1. Un diccionario con los datos de acceso a la base de datos.
2. La sentencia SQL preparada, si es necesario, con sus modificadores de parámetro correspondientes.
3. Opcionalmente, una tupla con datos a ser enlazados a la sentencia preparada.

Esta función sigue el algoritmo del diagrama de flujo explicado más arriba. Utiliza la verificación por el opuesto (afirmación de la negación) para saber si la sentencia ejecutada ha sido de escritura. Si lo ha sido, confirma los cambios y recupera la ID del último registro sobre el que se confirmaron cambios. En caso contrario, recupera los resultados.

Retorna o bien la ID del último registro sobre el que se confirmaron cambios, o bien, una tupla de resultados.

```
from MySQLdb import connect
```

```
def sql_execute(datos_acceso, sql, parametros=()):
    conexion = connect(**datos_acceso)
    cursor = conexion.cursor()
    cursor.execute(sql, parametros)

    if not sql.upper().find('SELECT') > -1:
        conexion.commit()
        data = cursor.lastrowid
    else:
        data = cursor.fetchall()

    cursor.close()
    conexion.close()

    return data
```

La variable **datos_acceso** será un diccionario con las claves *host* , *db* , *user* y *passwd* :

```
datos_acceso = dict(
    host='localhost',
    db='una_db',
    user='un_usuario',
    passwd='una_clave'
)
```

30. GENERACIÓN Y LECTURA DE COOKIES HTTP

Una cookie HTTP es un dato que el servidor envía al navegador del cliente, y este almacena a fin de ponerlo a disposición del servidor en cualquier momento, o hasta que el usuario decida eliminarlo.

Las cookies HTTP permiten a un servidor reconocer a un usuario y guardar información no crítica sobre este, de forma temporal, y localmente en el dispositivo del usuario en vez de hacerlo en el propio servidor.

30.1 CREACIÓN DE COOKIES HTTP CON PYTHON

Las cookies se generan por medio de cabeceras HTTP, enviando al navegador una instrucción con la siguiente estructura:

Set-Cookie: <nombre de la cookie>=<valor de la cookie>

Por ejemplo, para crear una cookie cuyo nombre sea *color*, y cuyo valor sea *azul*, la instrucción se vería como la siguiente:

Set-Cookie: color=azul

En Python, las cookies pueden crearse imprimiendo simplemente la línea anterior antes del cuerpo:

```
print("Content-Type: text/html; charset=utf-8")
print("Set-Cookie: color=azul")
print("")
print("<b>Cuerpo del documento</b>")
```

O empleando la biblioteca **http.cookies**:

```
from http.cookies import SimpleCookie
cookie = SimpleCookie()
cookie['color'] = 'azul'
print(cookie) # Debe enviarse ANTES de enviar el cuerpo del documento
```

30.1.1 ATRIBUTOS ADICIONALES DE UNA COOKIE HTTP

Además del nombre y valor, una cookie puede tener otros atributos definidos en las **RFC 6265**²⁴. Estos atributos pueden emplearse de forma combinada en una misma cookie, o utilizarse individualmente. Los principales atributos se describen y ejemplifican en la tabla de página 259.

²⁴ <https://www.rfc-editor.org/rfc/rfc6265#section-5.2>

Atributo	Descripción
Expires	Establece una fecha y hora de caducidad para la cookie. Ejemplo: Set-Cookie: foo=bar; Expires=Mon, 1 Nov 2021 09:00:00 GMT
Max-Age	Establece el tiempo de vida máximo de la cookie, en segundos. Ejemplo: Set-Cookie: foo=bar; Max-Age=300
Domain	Establece el dominio para el cual la cookie es válida. Ejemplo: Set-Cookie: foo=bar; Domain=elidabahit.co.uk
Path	Establece las rutas de directorio para las cuales la cookie es válida. Ejemplo: Set-Cookie: foo=bar; Path=/static
Secure	Establece que la cookie solo puede operar sobre un protocolo HTTP cifrado (HTTPS). Ejemplo: Set-Cookie: foo=bar; Secure
HttpOnly	Establece que la cookie no puede ser accesible por la API de JavaScript, a fin de prevenir la inyección de código fuente. Ejemplo: Set-Cookie: foo=bar; HttpOnly

Para agregar atributos mediante **SimpleCookie**, se puede recurrir a la siguiente sintaxis:

```
<variable>['<nombre>']['<atributo>'] = '<valor del atributo>'
```

Por ejemplo, para agregar el atributo *Max-Age* a la cookie *color* del ejemplo anterior:

```
cookie['color']['Max-Age'] = 300
```

30.2 LECTURA DE COOKIES HTTP DESDE PYTHON

En Python, las cabeceras HTTP pueden ser leídas desde el diccionario **environ** del módulo **os**:

```
from os import environ
cookies = environ.get('HTTP_COOKIE', '')
```

El valor de la variable cookies será la cadena de consulta:

```
color=azul
```

Para probar lo anterior, se sugiere montar un host virtual mediante CGI, enviar las cookies, recargar el recurso HTTP, y recuperarlas:

```
from os import environ
print("Content-Type: text/html; charset=utf-8")
print("Set-Cookie: color=azul")
cookies = environ.get('HTTP_COOKIE', 'Recargar esta página')
print("")
print(cookies)
```

Ya que la cadena recuperada desde `environ` debe ser manipulada de forma manual, emplear `SimpleCookie` para recuperar una cookie, resultará en un proceso más optimizado:

```
from http.cookies import SimpleCookie
from os import environ

cookie_string = environ.get('HTTP_COOKIE', '')
cookie = SimpleCookie()
cookie.load(cookie_string)

print("Content-Type: text/html; charset=utf-8")
print("")
print(cookie['color'].value)
```

Para recuperar el valor de un atributo específico de la cookie, se emplea la sintaxis:

```
<variable>['<nombre>']['<atributo>'].value
```

Por ejemplo, para recuperar el atributo *Max-Age* de la cookie color:

```
cookie['color']['Max-Age'].value
```

31. SISTEMAS DE CONTROL DE ACCESO EN ENTORNOS HTTPS

Los sistemas de control de acceso son mecanismos de seguridad empleados para garantizar que el acceso a un programa informático (o parte de dicho programa), se realiza por personas autorizadas.

Estos mecanismos se implementan en dos etapas:

- I. Una primera etapa de **autenticación**, donde se asegura que la persona que solicita el acceso a un programa es efectivamente quien dice ser.
- II. Una segunda etapa de **autorización**, donde se verifica que la persona previamente autenticada, efectivamente tiene derecho a acceder al programa (o a parte de él), y a realizar la acción para la cual ha solicitado la autorización.

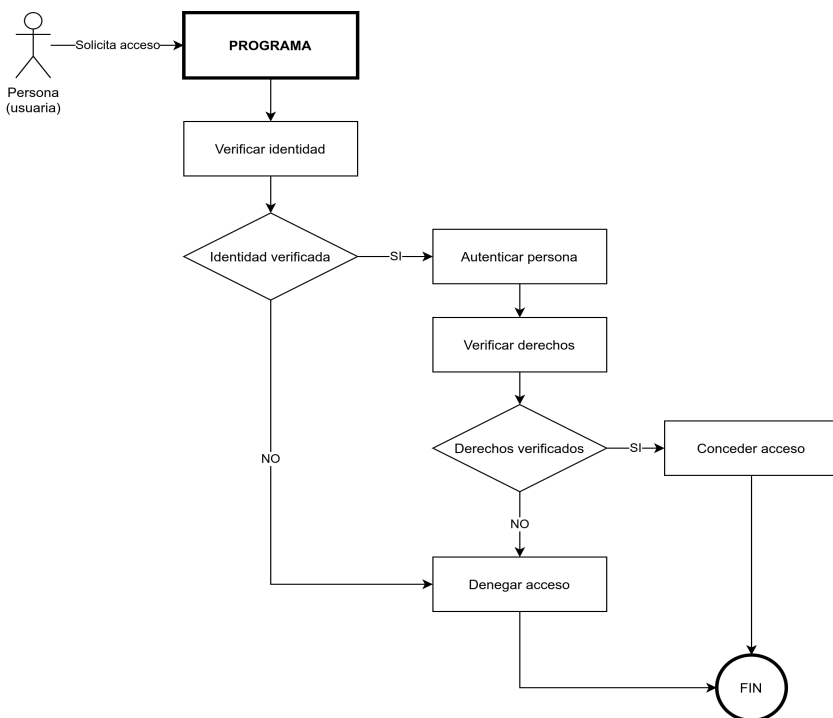
La **autorización** es el procedimiento mediante el cual se concede acceso a la persona autenticada, mientras que la **autenticación**, es el procedimiento mediante el cual se constata la identidad de una persona.

En líneas generales, un sistema de control de acceso se inicia con la solicitud de una persona, y a partir de allí, el programa procede a realizar las siguientes acciones:

1. Verificar la identidad de la persona.
2. Si la identidad no puede verificarse se niega el acceso.

3. Si la identidad puede verificarse, se constatan los derechos de la persona.
4. Si la persona no posee derechos de acceso, se niega el acceso.
5. Si la persona posee derechos de acceso, se concede acceso.

La siguiente imagen recoge estas acciones gráficamente:



Como puede verse, un programa informático solo puede conceder acceso a una persona cuya identidad haya sido verificada (proceso de autenticación) y cuyos derechos de acceso, hayan sido constatados (autorización).

Lo que parece sencillo a simple vista, presenta un gran inconveniente, y es que una persona no solicita acceso a una única parte del programa, una única vez a lo largo de su visita, si no que lo hace de forma constante.

Cuando una persona ingresa a un programa, podría requerir hacer uso de diversas funcionalidades, incluso más de una vez cada una, a lo largo de su visita. Y esto requiere estar constantemente verificando la identidad de la persona (es decir, autenticando a la persona), y sus derechos de acceso (es decir, autorizando a la persona).

Teniendo en cuenta que tanto para autenticarse como para recibir autorización, una persona aporta unos datos de acceso al programa²⁵ (como usuario y contraseña), si en cada solicitud debe autenticarse y autorizarse a la persona, sería necesario entonces, pedirle sus datos de acceso cada vez que solicitase autorización. Sin lugar a dudas, sería engorroso y poco práctico.

Para resolver este inconveniente, se requiere un modo de hacer persistir temporalmente, a lo largo de una misma sesión, los resultados de la autenticación.

Uno de los mecanismos de persistencia de sesiones más simple y seguro de llevar a cabo consiste en la generación de *archivos de*

25 Aunque este no es el único modo de autenticación, será el que se abarcará a la largo del capítulo.

sesión en el servidor, y el empleo de *cookies* —mencionadas en el capítulo 30—para recuperación de los mismos.

31.1 TRABAJAR CON ARCHIVOS DE SESIÓN

En el escenario más básico posible, este sistema consiste en crear un archivo temporal en el servidor, cuyo nombre (o parte de él) se almacene como valor en una cookie, una vez que la persona haya sido autenticada. Así, en cada nueva solicitud, se leerá el valor de la cookie y se utilizará para recuperar el archivo del servidor. Cuando ambas acciones sean posibles (leer el valor de la cookie y recuperar el archivo de sesión del servidor) se habrá constatado la vigencia de autenticación previa. En caso contrario, se solicitará a la persona, autenticarse nuevamente.

Por ejemplo, si un usuario llamado Juan se autentica de forma satisfactoria, se podría crear un archivo de sesión en el servidor llamado **archivo_de_sesion_de_juan** dentro del directorio temporal **/tmp/**, y almacenar el nombre de usuario en una cookie:

Set-Cookie: usuario=juan

De esta forma, en cada nueva solicitud de Juan durante una misma sesión, se estaría leyendo la cookie:

```
cookie_string = environ.get('HTTP_COOKIE', '')
cookie = SimpleCookie()
cookie.load(cookie_string)
usuario = cookie['usuario'].value
```

y empleando el valor recuperado para localizar el archivo de sesión:

```
from os.path import isfile
if isfile('/tmp/archivo_de_sesion_de_{}'.format(usuario)):
    # El usuario está autenticado
```

Dentro de un archivo de sesión se puede guardar información que no sea sensible, como por ejemplo, la ID del usuario o el nivel de acceso otorgado:

```
archivo = '/tmp/archivo_de_sesion_de_{}'.format(usuario)
with open(archivo, 'a') as f:
    f.write(nivel_de_acceso)
```

El mecanismo de operación en un sistema de archivos de sesión es simple. Lo crítico es hacerlo de forma segura.

31.2 CONSIDERACIONES DE SEGURIDAD

Si bien las consideraciones sobre la seguridad de un sistema de control de acceso HTTP, son un tema de ingeniería de software que excede el alcance de este libro, se abarcarán aspectos básicos a fin de orientar a aquellas personas con particular interés en el tema.

Un punto de partida para consideraciones básicas sobre la seguridad de un manejador de sesiones, puede encontrarse en el sitio Web del *Open Web Application Security Project (OWASP)* visitando https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

La premisa de todo sistema de seguridad debe —necesariamente— consistir en definir lo que un sistema y sus componentes son, y no lo que no son. Esta premisa se basa en el **principio epistemológico de la definición** que establece que *no puede definirse un elemento por lo que no es, sino que debe definirse por lo que el elemento es*.

Dicho de otro modo, quien programa no puede centrarse en especificar lo que está prohibido, sino que debe enfocarse en lo que está absolutamente permitido. A partir de allí, todo aquello que no está

permitido, se supone prohibido. Si se define con precisión cada uno de los componentes del sistema de archivos de sesión, se estará definiendo un sistema seguro.

A continuación, se recorren los aspectos más básicos a tener en cuenta en un sistema de manejo de sesiones, en lo que a seguridad respecta.

Aspecto	Observaciones
Directorio	• Se requiere un directorio con permisos de escritura. • El directorio no debe ser servido por el servidor HTTP de Apache. • Se sugiere emplear un directorio temporal oculto, de difícil acceso.
Archivo de sesión	• Definir un sistema para almacenar variables de sesión en el archivo (requiere diseñar la estructura del archivo, un mecanismo para almacenar las variables de sesión, y otro para recuperarlas). • Evitar el uso de las bibliotecas shelve y pickle.
Nombre del archivo de sesión	• Se sugiere emplear algún tipo de prefijo o sufijo combinado con una ID de sesión de alta entropía²⁶. • Actualizar la fecha de modificación del archivo conforme la sesión del usuario se mantenga activa.
Cookie	• Emplear como valor de la cookie solo la ID de la sesión. • Establecer en HttpOnly y Secure. • Actualizar la fecha de caducidad conforme la sesión del usuario se mantenga activa.
Recolección de basura	• Establecer un tiempo de vida máximo para sesiones inactivas, acorde a los requerimientos particulares de la aplicación, o configurable (en promedio, entre 5 y 30 minutos). • Automatizar el proceso de verificación de archivos de sesión caducos, y eliminar aquellos cuya fecha de modificación sea anterior al período máximo de inactividad establecido.

26 Para profundizar en este aspecto, se sugiere leer el capítulo «Criptografía» del libro «Fundamentos de Ciencias Informáticas para el abordaje de la Programación» de Eugenia Bahit.

32. PROGRAMACIÓN ORIENTADA A OBJETOS CON PYTHON

En esta sección se hará una breve introducción al paradigma de la programación orientada a objetos, aspectos más elementales y cómo estos han sido implementados por Python. No se abarcarán conceptos avanzados o teóricos relativos al pensamiento computacional aplicado al análisis de objetos.

32.1 BREVE INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Un paradigma de programación es aquello que define la forma en la que la estructura y los elementos de un programa se deben construir. El paradigma de la programación orientada a objetos establece que un programa debe ser construido sobre la base de unos elementos que se denominan *objetos*.

Este paradigma propone la agrupación de datos y operaciones —relativas a esos datos—, en elementos que los mantienen aislados del entorno.

Hasta aquí se ha visto cómo los datos (variables) y operaciones (funciones) se mantienen dispersas a lo largo de uno o más *scripts*. A partir de ahora, se verá que en la programación orientada a objetos estos datos y operaciones mantienen agrupados en estructuras de control denominadas *clases*.

De esta manera, mientras que en la programación estructurada un dato ***a*** y un dato ***b*** procesados mediante una operación ***c*** se veían de esta forma:

```
a = 25  
b = 5
```

```
def c(a, b):  
    return a * b
```

```
result = c(a, b)
```

En la programación orientada a objetos se encapsulan de esta otra:

```
class X:  
  
    a = 25  
    b = 5  
  
    def c(self):  
        return self.a * self.b
```

```
x = X()  
result = x.c()
```

32.2 ELEMENTOS Y CARACTERÍSTICAS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

La programación orientada a objetos define entonces, unos elementos y características que los lenguajes de programación que soportan este paradigma deben ofrecer. En lo sucesivo se explican estos elementos y características junto al código Python que los implementa.

32.2.1 CLASES

Se define como clase a aquella estructura de control cuyo objetivo es definir los datos —denominados *propiedades*— y operaciones —denominadas *métodos*— necesarios para crear un objetos.

32.2.2 MÉTODOS Y PROPIEDADES

Los *métodos* son las operaciones definidas dentro de una clase mientras que las *propiedades*, los datos. Un método se ve como una función pero dentro de la clase, mientras que una propiedad, como una variable.

```
class A:
    b = 15
    c = 10

    def d(self):
        return self.b * self.c
```

En el ejemplo anterior, **A** es una clase; **b** y **c** , son sus propiedades; y **d** , su único método. **self** es una característica particular de Python de la que se hablará posteriormente.

32.2.3 OBJETOS

Se define como objeto a un conjunto de datos y operaciones encapsulados. Técnicamente, un objeto es la instancia de una clase (donde el término «instancia» se refiere a que la clase ha sido invocada). En términos prácticos, un objeto es una variable cuyo valor es la instancia de una clase.

```
class A:
    pass
```

```
a = A()
```

En el ejemplo previo, **a** es un objeto y **A()** es la instancia de la clase denominada **A** .

32.2.4 POLIMORFISMO

Se denomina polimorfismo a la característica que posibilita que un único objeto pueda estar formado por más de un tipo de datos.

En el siguiente ejemplo, se dice que ***a*** es polimórfico debido a que posee más de un tipo de datos:

```
class A:
    pass

a = A() # "a" es un objeto polimórfico
a.tipo_entero = 12
a.tipo_string = 'cadena de texto'
a.tipo_boolean = False
```

32.2.5 ENCAPSULADO

Se denomina encapsulado a la característica que permite mantener aisladas dentro de un objeto, a sus propiedades y métodos. Esto significa que dos clases diferentes pueden definir datos y operaciones con, por ejemplo, los mismos nombres, pero estas no se superponen ni colapsan ni interfieren entre sí.

En el siguiente ejemplo se puede ver como ***B.a*** está encapsulado mientras que ***a*** no lo está:

```
a = 15

class B:
    a = 7

a = a + 1
print(a)      # 16
print(B.a)    # 7
```

Solo el valor de la variable global a se incrementa mientras que el de la propiedad $B.a$ permanece inalterado.

32.2.6 HERENCIA

Se denomina herencia a la característica que permite que un objeto adquiera las propiedades y métodos de otro objeto del cual hereda.

```
class A:
    b = 15

class B(A):
    pass

c = B()
print(c.b) # 15
```

En el ejemplo anterior, B es una clase heredada de A —esto, en Python se denota mediante $B(A)$ —. Esta clase, B , no define ninguna propiedad. Sin embargo, al crearse el objeto c , este hereda la propiedad b de la clase A .

32.2.7 COMPOSICIÓN

Se denomina composición a la característica que permite que un objeto se convierta en un subconjunto de otro objeto.

En la composición existe un objeto que compone a otro (*compositor*) y uno que incluye a este entre sus propiedades (*compuesto*). Mientras que en la herencia los elementos de un objetos son elementos de otro objeto, en la composición, es un objeto quien pasa a ser el elemento de otro objeto.

```
class A:
    b = 15
```

HERENCIA

```
class B(A):  
    pass
```

COMPOSICIÓN

```
class C:  
    a = A()
```

```
print(B.b)  
print(C.a.b)
```

Para entender la **diferencia entre herencia y composición**, puede llevarse a un plano más tangible, imaginando una pizza de tomate y otra de tomate y queso. Si se asume que la base de una pizza es la masa con tomate, entonces, la pizza de tomate y queso es una “evolución” de la pizza de tomate y por lo tanto «*hereda*» de ella, la masa y el tomate de los cuáles se «*compone*» (adicionalmente, también se compone de queso pero este no lo hereda).

Si se toma la analogía de la pizza como ejemplo, se obtienen las siguientes clases:

```
class Queso:  
    pass
```

```
class Tomate:  
    pass
```

```
class Masa:  
    pass
```

```
class PizzaDeTomate:  
  
    masa = Masa()  
    tomate = Tomate()
```

```
class PizzaDeTomateQueso(PizzaDeTomate):  
    queso = Queso()
```

32.2.8 VISIBILIDAD Y OCULTACIÓN

Otra de las características de la programación orientada a objetos es que estos pueden ocultar sus métodos y propiedades. La visibilidad de los métodos y propiedades se maneja en tres niveles que permiten controlar el acceso a estos atributos:

- *Pública*: los métodos y propiedades son accesibles desde cualquier ámbito.
- *Protegida*: los métodos y propiedades son accesibles desde la propia clase y las clases que heredan.
- *Privada*: los métodos y las propiedades solo son accesibles desde el ámbito de la clase que los define.

En lenguajes de programación como C++, Java o PHP, esta característica de la POO se implementa mediante el uso de las palabras clave reservadas, *public*, *protected* y *private* como se muestra en los siguientes ejemplos:

Ejemplo en C++	Mismo ejemplo en PHP
<pre>class A { public: int b; float c; protected: char d[10]; int e; private:</pre>	<pre>class A { public \$b; public \$c; protected \$d; protected \$e; private \$f; }</pre>

```
};          float f;
```

Python no ofrece un mecanismo que satisfaga esta característica de la programación orientada a objetos, por lo que no existe un control de acceso nativo a propiedades y métodos.

Según se informa en la documentación oficial²⁷, quienes deseen programar en Python pueden apelar a una convención informal de escribir el código de propiedades y métodos que se desea considerar protegidos, agregando como prefijo un guión bajo. Pero esto es solo una convención informal de escritura, que no tiene efecto real en el código. Por ejemplo, esta propiedad podría ser accedida desde fuera de la clase:

```
class A:
    _b = 15

print(A._b) # 15
```

Al agregar al menos dos guiones bajos como prefijo, dado que Python reserva esta nomenclatura para nombres propios, adiciona un prefijo extra con el nombre de la clase antecedido de un guión bajo:

```
class A:
    __b = 15
```

Aquí `__b` es convertido por Python en `_A__b`, lo que podría entenderse como una forma de escribir propiedades y métodos privados, debido a que cuando se intenta acceder a una propiedad cuyo nombre difiere del definido, un error será lanzado:

```
>>> class A:
```

27 <https://docs.python.org/3/tutorial/classes.html#private-variables>

```
...     __b = 15
...
>>> A.__b
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: type object 'A' has no attribute '__b'
```

Sin embargo, este efecto colateral, no ofrece un control de acceso nativo real, puesto que se puede acceder al atributo modificado empleando la misma regla que Python:

```
>>> A._A__b
15
```

32.3 SOBRE EL USO DE SELF EN PYTHON

Cuando se define una clase se describen unas propiedades y unos métodos que en realidad hacen referencia al objeto que se creará a partir de ellas, y no a la clase en sí misma. Analizándolo en un pseudo código *ad hoc*, la interpretación sería la siguiente:

```
class UnaClase:
    una_propiedad = 1

    funcion una_funcion():
        objeto.una_propiedad = 2
```

La palabra *self* —sí mismo, en inglés— (también utilizada en otros lenguajes), es la forma en la que Python se refiere al objeto. En PHP, por ejemplo, la palabra empleada es *this* (*self* se utiliza pero en otro contexto).

En el caso particular de Python, *self* debe pasarse como parámetro obligatorio en la primera posición de todos los métodos definidos en una clase, para que esta pueda disponer del objeto creado a partir de su

instancia. Este requerimiento es un *requerimiento particular del lenguaje* y no, una característica de la programación orientada a objetos.

32.4 ARTILUGIOS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

En esta sección se presentarán algunos recursos ofrecidos de forma nativa por el lenguaje para el desarrollo de objetos, y otros, no disponibles pero sí resueltos empleando los recursos nativos.

32.4.1 EL MÉTODO CONSTRUCTOR

El método constructor es aquel que se ejecuta de forma automática para inicializar las propiedades de un objeto cuando éste es creado. El nombre del método constructor se encuentra reservado, siendo `__init__()` para Python:

```
class A:
    def __init__(self):
        self.a = 10
        self.b = 25
```

32.4.2 RECORRIDO DE PROPIEDADES

Mediante la función `vars()` de Python es posible recorrer sus propiedades de forma iterativa. Esta función retorna un diccionario cuyas claves son los nombres de las propiedades y los valores, el valor de las propiedades:

```
>>> a = A()
>>> vars(a)
{'a': 10, 'b': 25}
```

Si la clase `A` definiese sus propiedades fuera del método `__init__` la salida de `vars()` sería distinta:

```
>>> a = A()
```

```
>>> vars(a)
{}
```

32.4.3 PATRÓN DE DISEÑO COMPUESTO Y AGREGACIÓN

Cuando una propiedad está compuesta por un solo objeto es posible restringir el tipo de datos de la propiedad, en un lenguaje de tipado dinámico, empleando el *patrón de diseño compuesto*. En algunos lenguajes de tipado dinámico, es el lenguaje quien ofrece este soporte de forma nativa. Por ejemplo, en PHP esto se haría de la siguiente forma:

```
class A {
    function __construct(B $b) {
        $this->b = $b;
    }
}
```

Si al crear un nuevo objeto de tipo *A* se pasara como parámetro una variable que no fuese de tipo *B*, un error sería lanzado:

```
php > $a = new A(25);
PHP Warning: Uncaught TypeError: Argument 1 passed to
A::__construct() must be an instance of B, int given (...)
```

Esto es similar en la mayoría de los lenguajes, sin embargo en Python, no se ofrece este tipo de restricción para ser impuesta a variables y funciones en tiempo de ejecución²⁸. El mismo comportamiento puede ser logrado de forma manual, verificando que el parámetro recibido sea una instancia del tipo esperado, y lanzando un error cuando no lo sea:

```
class A(object):
```

²⁸ Notar que módulos como *typing* están disponibles en Python desde la versión 3.5, pero no impone restricción de tipo en tiempo de ejecución.

```
def __init__(self, b):
    if isinstance(b, B):
        self.b = b
    else:
        tipo_recibido = type(b)
        msg = "Tipo B esperado, {} recibido".format(tipo_recibido)
        raise TypeError, msg
```

Este mismo mecanismo puede emplearse para implementar un *patrón de agregación*:

```
class A(object):

    def __init__(self):
        self.b_collection = []

    def add_b(self, b):
        if isinstance(b, B):
            self.b.append(b)
        else:
            tipo_recibido = type(b)
            msg = "Tipo B esperado, {} recibido".format(tipo_recibido)
            raise TypeError, msg
```

33. PRUEBAS UNITARIAS

Existen varias bibliotecas para realizar pruebas unitarias en Python. Sin embargo, este libro se centrará en módulos de la biblioteca estándar.

33.1 DOCTEST

En Python existe una forma bastante simple y rudimentaria de realizar pruebas unitarias, cuya curva de aprendizaje es prácticamente imperceptible. Se trata de cadenas de texto insertadas en los comentarios, cuyo contenido se equipara al de una sesión interactiva de la *shell* de python. *Doctest*, localiza dichos fragmentos, los ejecuta y verifica que la salida obtenida sea igual a la mostrada en los

comentarios. Solo se requiere ejecutar el *script* con el argumento **-v** para ver los resultados.

El siguiente fragmento de código pertenece a un archivo llamado *prueba.py*:

```
#!/usr/bin/env python3
"""
>>> calcular_edad(1978)
42

>>> calcular_edad("1978")
42

>>> calcular_edad("xxx")
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: 'xxx'
>>> calcular_edad()
Traceback (most recent call last):
...
TypeError: calcular_edad() missing 1 required positional
argument: 'nacimiento'
"""

def calcular_edad(nacimiento):
    aaaa = int(nacimiento)
    edad = 2020 - aaaa
    return edad

if __name__ == "__main__":
    from doctest import testmod
    testmod()
```

Al ejecutar el archivo, solo el argumento **-v** hará correr las pruebas:

```
eugenia@bella:~/Documentos$ python prueba.py
eugenia@bella:~/Documentos$ python3 prueba.py -v
Trying:
    calcular_edad(1978)
Expecting:
    42
ok
Trying:
```

```
    calcular_edad("1978")
Expecting:
    42
ok
Trying:
    calcular_edad("xxx")
Expecting:
    Traceback (most recent call last):
        ...
    ValueError: invalid literal for int() with base 10: 'xxx'
ok
Trying:
    calcular_edad()
Expecting:
    Traceback (most recent call last):
        ...
    TypeError: calcular_edad() missing 1 required positional
argument: 'nacimiento'
ok
1 items had no tests:
    __main__.calcular_edad
1 items passed all tests:
    4 tests in __main__
4 tests in 2 items.
4 passed and 0 failed.
Test passed.
```

La documentación oficial se encuentra disponible en <https://docs.python.org/3/library/doctest.html>

33.2 UNITTEST

Menos rudimentario, para quienes ya poseen experiencia programando con pruebas unitarias, existe el módulo *unittest*, quien tampoco necesita ser instalado ya que desde la versión 2.1 forma parte de la biblioteca estándar de Python.

33.2.1 MÉTODOS ASSERT

Una lista completa de los métodos *assert* se puede encontrar en la documentación oficial:

<http://docs.python.org/library/unittest.html#assert-methods>

Una diferencia particular con otros marcos de trabajo para pruebas unitarias, es que se pueden llevar a cabo pruebas con una sintaxis bastante simple, sin necesidad de recurrir a métodos *assert* específicos:

```
assert resultado == valor_esperado
```

Un ejemplo más concreto podría verse como el siguiente:

```
assert self.coverage.sumar_dos_numeros(5, 15) == 20
```

Donde *coverage* será la instancia al objeto del SUT.

Generalmente, por cada método *assert* existe su opuesto:

```
asserEqual(a, b)  
asserNotEqual(a, b)
```

Cada método *assert* deberá recibir al menos un parámetro que será el resultado de ejecutar el código del SUT y opcionalmente, como último parámetro, puede recibir un mensaje personalizado para ser arrojado en caso de error:

```
asserEqual(a, b, msg='ERROR al comprobar igualdad')
```

Los métodos *assert* que requieren el paso de dos parámetros obligatorios (valores que deben compararse entre sí), generalmente guardan el siguiente orden:

```
metodoAssert(valor_recibido, valor_esperado)
```

Es decir, que el primer parámetro siempre será el valor recibido por la ejecución del código SUT y el segundo, el valor esperado.

A continuación, se muestra un ejemplo sencillo:

```
# -*- coding: utf-8 -*-
# Archivo TestCase: test_balance_contable.py
from unittest import main, TestCase
from balance_contable import BalanceContable

class BalanceContableTestCase(TestCase):

    # setUp()
    def setUp(self):
        self.coverage = BalanceContable()
        self.coverage.alicuota_iva = 21

    # assertEquals(valor_recibido, valor_esperado)
    def test_calcular_iva(self):
        self.coverage.importe_bruto = 2500
        result = self.coverage.calcular_iva()
        self.assertEqual(result, 525)

    # AssertTrue(valor_recibido)
    def test_alcanzado_por_impuesto_de_importacion_con_160(self):
        self.coverage.importe_bruto = 160
        result = self.coverage.alcanzado_por_impuesto_de_importacion()
        self.assertTrue(result)

    # AssertIsNone(valor_recibido)
    def test_alcanzado_por_impuesto_de_importacion_con_143(self):
        self.coverage.importe_bruto = 143
        result = self.coverage.alcanzado_por_impuesto_de_importacion()
        self.assertIsNone(result)

# Necesario para correr los test si es llamado por línea de comandos
if __name__ == "__main__":
    main()
```

A continuación, el código del SUT:

```
# -*- coding: utf-8 -*-
# Código SUT: balance_contable.py

class BalanceContable(object):

    def __init__(self):
        self.importe_bruto = 0
        self.alicuota_iva = 0
```

```
# Calcular IVA sobre un importe bruto
def calcular_iva(self):
    iva = self.importe_bruto * self.alicuota_iva / 100
    return iva

# Determinar si un importe paga impuesto de importación
def alcanzado_por_impuesto_de_importacion(self):
    # importes mayores a 150 USD pagan impuesto
    if self.importe_bruto > 150:
        return True
```

33.2.2 DESCUBRIENDO PRUEBAS

Todas las pruebas de una aplicación pueden ejecutarse empleando el comando *discover*:

```
eugenia@host:~/proyectos$ python -m unittest discover
```

discover, “descubrirá” todas las pruebas, identificándolas por el nombre del archivo, el cual debe comenzar por el prefijo *test* (*discover* utiliza la expresión regular **test*.py** para identificar las pruebas). Además, debe tenerse en cuenta que el nombre de los métodos de prueba, también deben comenzar por el prefijo *test*.

En caso de querer ejecutar las pruebas de un único archivo, basta con correr dicho archivo:

```
eugenia@host:~/proyectos$ python test_balance_contable.py
```

Para una prueba en particular, en la línea anterior habrá que invocar al método de prueba que se desee ejecutar:

```
eugenia@host:~/proyectos$ python test_balance_contable.py \
    BalanceContableTestCase.test_calcular_iva
```

También es posible, pasar el parámetro **-v** a fin de obtener un reporte más detallado:

```
eugenia@host:~/proyectos$ python -m unittest discover -v
test_alcanzado_por_impuesto_de_importacion_con_143
(Test.test_balance_contable.BalanceContableTestCase) ... ok
test_alcanzado_por_impuesto_de_importacion_con_160
(Test.test_balance_contable.BalanceContableTestCase) ... ok
test_calcular_iva
(Test.test_balance_contable.BalanceContableTestCase) ... ok
```

```
-----
Ran 3 tests in 0.001s
```

OK

